

Owl - Optimal wealth lab:

Mathematical Foundations of a Retirement Planning Optimizer

A mixed-integer linear programming approach to US federal and state tax-aware retirement wealth management

2026 Edition

Martin-D. Lacasse

October 28, 2026

© 2024–2026 Martin-D. Lacasse

Contents

1 Introduction **1**

2 US Retirement Tax Rules and Their Implementation in Owl **3**
2.1 Overview . . . 3
2.2 Savings Accounts . . . 3
2.3 Household Financial Profile and Cash Flow . . . 5
2.4 Social Security Benefits . . . 10
2.5 Pension Income . . . 13
2.6 Single Premium Immediate Annuity (SPIA) . . . 15
2.7 Roth Account Rules . . . 16
2.8 Health Savings Accounts . . . 17
2.9 ACA Marketplace Premiums (Pre-65) . . . 20
2.10 Medicare and IRMAA Premiums . . . 22
2.11 Long-Term Capital Gains Tax . . . 23
2.12 Net Investment Income Tax . . . 25
2.13 Ordinary Income Tax . . . 26
2.14 State Income Tax . . . 27

3 Model Architecture and Solution Approach **29**
3.1 Overview . . . 29
3.2 Planning Horizon and Time Steps . . . 29
3.3 Account Dynamics . . . 31
3.4 Asset Allocation and Glide Paths . . . 32
3.5 Spending Profile and Objectives . . . 32
3.6 The Self-Consistent Iteration Loop . . . 34
3.7 Filing Status and the Death of a Spouse . . . 36
3.8 Solvers . . . 36
3.9 MIP Strategies: Branch-and-Bound and Local Search . . . 37

4 Formulation with imposed asset allocation ratios **39**
4.1 Constraints . . . 40
4.1.1 Required minimum distributions (RMDs) . . . 40
4.1.2 Qualified charitable distributions . . . 40
4.1.3 Income tax brackets . . . 42
4.1.4 Standard exemption . . . 42
4.1.5 The OBBBA senior bonus and its phase-out . . . 42
4.1.6 Withdrawal limits . . . 43
4.1.7 Withdrawal ordering . . . 43
4.1.8 Posthumous account activities . . . 44
4.1.9 Roth conversions . . . 44
4.1.10 Minimum taxable balance . . . 45
4.1.11 Initial balances . . . 45
4.1.12 Roth 5-year maturation and conversion ladder . . . 45
4.1.13 Cash-flow surplus . . . 47
4.1.14 Account balances . . . 47
4.1.15 Net spending . . . 51
4.1.16 LTCG bracket constraints . . . 53
4.1.17 Net investment income tax . . . 55
4.1.18 Taxable ordinary income . . . 56
4.1.19 State income tax . . . 56
4.1.20 Social Security taxability . . . 58
4.1.21 Medicare brackets and costs . . . 60
4.1.22 ACA marketplace premium constraints . . . 64

5 Objective Functions and Their Formulation **66**
5.1 Maximizing net spending . . . 66
5.2 Maximizing bequest . . . 68
5.3 Lexicographic tie-breaking . . . 68

6 Rate models for return scenarios **70**
6.1 Historical data . . . 70
6.2 Constant rate methods . . . 71
6.3 Deterministic varying rates: `historical` . . . 72
6.4 Stochastic models . . . 72
6.4.1 Inflation skewness correction . . . 72
6.4.2 Mean anchoring (`constrain_mean`) . . . 73
6.4.3 Multivariate normal: `historical_gaussian` and `gaussian` . . . 74

6.4.4 Log-normal models: `lognormal` and `historical_lognormal` . 75
6.4.5 Bootstrap sequence-of-returns: `historical_bootstrap` 76
6.4.6 Vector autoregression: `vector_ar` 77
6.4.7 DCC-GARCH(1,1): `garch_dcc` 78
6.4.8 Gaussian mixture model: `gmm` 80
6.4.9 Hidden Markov model: `hmm` 81
6.4.10 Gaussian copula: `historical_copula` 83
6.5 User-supplied rates: `dataframe` 85
6.6 Monte Carlo and stress testing 85

7 Stochastic Spending Optimization and Longevity Risk 87
7.1 Scenario bases 87
7.2 The commitment LP and efficient frontier 88
7.3 Longevity risk 90
7.4 Scenario generation 90
7.5 Implementation in Owl 91

8 The Spending–Bequest Trade-off 93
8.1 Why the bequest is swept and spending is not 93
8.2 Deterministic trade-off 93
8.3 Stochastic trade-off 95
8.4 What reaches heirs 95
8.5 Implementation in Owl 96

9 Optimization of Social Security Claiming Ages 97
9.1 The claiming-age grid 97
9.2 Claiming-age adjustment factors 98
9.3 The search over fixed-age solves 99
9.4 Why the claiming age is not a MILP variable 100
9.5 Implementation in Owl 100

A Indices, variables, and parameters 102
A.1 Indices 102
A.2 Variables 103
A.3 Parameters 108
A.4 Intermediate variables 117

B Mapping of decision variables 123
B.1 Reverse mapping of indices 128

C Building constraint matrices **129**
C.1 Inequality constraints 129
C.2 Equality constraints 132
C.3 Other considerations 137
C.4 Constraint tags and the explanation engine 139

D Solver Options **143**
D.1 Objective and goals 143
D.2 Roth conversion controls 144
D.3 Tax-breakpoint modes 144
D.4 Solver selection and effort 145
D.5 Convergence of the self-consistent loop 146
D.6 Local search 147
D.7 Inputs, reporting, and units 148
D.8 Retired and removed options 148

E Source File Reference **150**

Abstract

This document describes the mathematical foundations of **Owl** - Optimal wealth lab, a Python application that optimizes US retirement planning using mixed-integer linear programming. Owl models federal tax laws, Medicare and ACA marketplace premiums, Social Security rules, pension income, Roth conversions, Health Savings Accounts, and account dynamics jointly within a single optimization framework. The self-consistent iteration loop and MIP reformulations handle nonlinear tax interactions. Chapters 1–3 present the conceptual model for a general audience. Chapters 4–8 provide the formal constraint structure, the objective functions, the rate models, and the two outer searches — stochastic spending and Social Security claiming ages. Five appendices hold the implementation reference: the notation, the variable mapping, the construction of the constraint matrices, the solver options, and a guide to the source files.

1. Introduction

This document describes the mathematical model underlying the optimization algorithms implemented in Owl, which is a Python application optimizing retirement planning using mixed-integer linear programming. Owl is designed for US retirees as it considers US federal tax laws, Medicare premiums, rules for 401k including required minimum distributions, maturation rules for Roth accounts and conversions, Social Security rules, etc. The goal of these calculations is to optimize the financial aspects of retirement planning, considering the types of savings accounts, federal income tax, contributions, return rates, Medicare premiums, Roth conversions, and desired income among many other things.

The approach is described here mathematically and the Python implementation follows the structure and notation presented in this document. The intent of this document is to provide a guide to the source code for any individual desiring to extend the model to other cases. The mathematical description is not tied to any specific mathematical programming language so that it remains as generic as possible and can be solved using different linear solvers. The source code, installation instructions, and usage examples are available on the project's GitHub repository [1].

This document is intended for two audiences. Chapters 1 through 3 require only basic financial literacy: familiarity with retirement account types, tax-deferred vs. tax-free savings, Social Security, and Medicare is sufficient to follow the discussion. Chapters 4 through 9 require mathematical maturity: comfort with linear algebra, index notation, and the basics of linear and mixed-integer programming is assumed, and Chapter 6 additionally assumes familiarity with probability distributions and time-series models. Readers interested only in the conceptual model may stop after Chapter 3; readers seeking to extend or verify the implementation will need the later chapters and the appendices.

The remainder of this document is organized as follows. Chapter 2 surveys the US retirement tax rules that Owl models—Social Security benefits and the earnings test, pension income, Roth account rules, Health Savings Accounts, ACA marketplace premiums, Medicare premiums, capital gains tax, the Net Investment Income

Tax, and federal and state ordinary income tax—with notes on how each is implemented and where known limitations apply. Chapter 3 describes the overall model architecture: how account balances evolve over time, how asset allocation glide paths are specified, how spending objectives are formulated, and how the self-consistent iteration loop and the mixed-integer reformulations resolve the nonlinear tax interactions that arise within a linear programming framework. Chapter 4 presents the formal constraint structure, and Chapter 5 defines the objective functions. Chapter 6 describes the rate models available for generating annual return sequences, from simple fixed-rate assumptions to fully stochastic simulations. The last three chapters cover the outer searches that solve the plan repeatedly rather than once: Chapter 7 commits to a spending level across many return scenarios and samples longevity, Chapter 8 traces how much spending each dollar of bequest costs, and Chapter 9 chooses Social Security claiming ages.

Five appendices hold the implementation reference. Appendix A defines every index, variable, and parameter used in the formulation, and is the notation key for Chapter 4 onwards. Appendix B gives the mapping of decision variables into the solver's one-dimensional array, and Appendix C the construction of the constraint matrices, including the constraint tags that drive the explanation engine. Appendix D documents every solver option, with its default and the options that have been retired or removed, and Appendix E is a reference guide to the Python source files.

2. US Retirement Tax Rules and Their Implementation in Owl

2.1 Overview

Retirement income in the United States comes from four broad account types: taxable brokerage accounts, tax-deferred accounts such as traditional 401(k)s and IRAs, tax-free Roth accounts, and Health Savings Accounts (HSAs). Withdrawals from these accounts interact with multiple simultaneously computed taxes and premiums: ordinary income tax (with bracket stacking), long-term capital gains (LTCG) tax, the Net Investment Income Tax (NIIT), ACA Premium Tax Credit, Medicare Part B premium surcharges, and the taxability fraction of Social Security (SS) benefits. Because all these quantities depend on the same underlying income variables, optimizing any one strategy in isolation—such as Roth conversion timing or Social Security claiming age—can inadvertently worsen another.

Owl models all of these rules *jointly* within a single linear program (LP) or mixed-integer program (MIP) so that strategies such as Roth conversions, SS claiming age, and withdrawal order are optimized together rather than sequentially. Many rules are nonlinear (piecewise, min/max); Owl handles them within a linear framework using a self-consistent (SC) iteration loop for quantities that can be lagged by one iteration, and MIP reformulations for quantities that require exact simultaneous computation.

2.2 Savings Accounts

US retirement savings are held in four fundamentally different account types, each with its own tax treatment, contribution rules, and withdrawal constraints. Owl models all four types and optimizes across them jointly. For married couples, each spouse holds their own independent set of accounts; balances, contribution histories, claiming ages, and Required Minimum Distribution (RMD) schedules are tracked

separately for each individual while the optimizer considers the combined household portfolio.

Taxable brokerage accounts

Taxable accounts hold securities purchased with after-tax dollars. Dividends and interest are taxed as ordinary income (or at preferential qualified-dividend rates) in the year received, regardless of whether the cash is withdrawn. Capital gains are recognized and taxed only when securities are sold. There is no tax deduction for contributions, no contribution limit from a federal tax standpoint, and no RMD requirement.

Owl models the taxable account as generating a dividend and interest yield each year proportional to the balance. Capital gains arise when the optimizer elects to sell holdings; all realized gains are treated as long-term (see the limitation noted in Section 2.11). When an initial cost basis is provided via `setCostBasis()`, the unrealized gain fraction is computed from the actual basis and tracked throughout the plan via a pro-rata update rule (see Eqs. (A.25)–(A.27)); this reduces the taxable gain recognized on withdrawals in the early plan years.

Tax-deferred accounts (traditional IRA, 401(k), and 403(b))

Tax-deferred accounts accept pre-tax contributions that reduce current taxable income, subject to annual IRS limits. Growth inside the account is not taxed until withdrawal. Every dollar withdrawn is taxed as ordinary income in the year taken. Required Minimum Distributions (RMDs) apply starting at age 70–75 depending on the account owner's birth year (see Section 2.13). A 10% early withdrawal penalty applies to withdrawals taken before age $59\frac{1}{2}$, subject to IRS exceptions not modeled here.

Owl supports contributions to traditional 401(k) and IRA accounts as entered in the Wages and Contributions table described in Section 2.3. The optimizer determines the optimal annual withdrawal amount from these accounts subject to RMD floors and other constraints. The pro-rata rule (IRS Form 8606) is not modeled: Owl treats the entire tax-deferred balance as pre-tax, so every dollar converted or withdrawn is fully taxable (see the limitation in Section 2.7).

Tax-free Roth accounts (Roth IRA and Roth 401(k))

Roth accounts accept after-tax contributions. Qualified withdrawals of both principal and earnings are entirely tax- and penalty-free. Roth accounts are not subject to RMDs during the original owner's lifetime. Roth conversions—transfers from a tax-deferred account to a Roth account—are fully taxable as ordinary income in the year of conversion but produce tax-free growth thereafter. The detailed five-year seasoning and age-$59\frac{1}{2}$ rules governing Roth withdrawals and conversions are described in Section 2.7.

Owl supports contributions to Roth 401(k) and Roth IRA accounts, and optimizes Roth conversion amounts annually subject to user-specified constraints and tax-bracket considerations.

Health Savings Accounts (HSA)

HSAs are the only triple tax-advantaged savings vehicle in the US tax code: contributions are pre-tax (reducing ordinary income and MAGI), growth is tax-free, and qualified medical withdrawals are entirely tax-free at any age. After age 65 (Medicare enrollment), non-qualified withdrawals are taxed as ordinary income, effectively making the HSA a second traditional IRA without Required Minimum Distributions. A surviving spouse inherits the account intact, preserving its full tax-advantaged status. Medicare Parts B and D premiums, IRMAA surcharges, and out-of-pocket medical costs all qualify as HSA withdrawals, so these expenses can be funded entirely tax-free from the account; Medigap premiums do not qualify. Annual contribution limits are set by the IRS (approximately $4,400 for self-only and $8,750 for family coverage in 2026) and must cease at Medicare enrollment. See Section 2.8 for the detailed rules and implementation.

2.3 Household Financial Profile and Cash Flow

Owl takes two primary inputs. The first is the **case configuration file** (a `.toml` file), which specifies all plan parameters: account balances, Social Security claiming ages, spending objectives, rate assumptions, solver options, and so on. The second is the **Household Financial Profile (HFP)**, an Excel workbook that captures the year-by-year financial data for each individual and the household as a whole. The HFP consists of three tables: the *Wages and Contributions* table (one tab per individual), the *Fixed Assets* table, and the *Debts* table.

Wages and Contributions table

Each individual has their own sheet with the following year-indexed columns:

- *year* — calendar year;
- *anticipated wages* — earned income expected for that year, as it enters AGI: pay net of the pre-tax contributions (traditional 401(k) deferrals, deductible IRA and payroll HSA contributions; employer contributions, entered in *employer ctrb*, are not subtracted, as they were never part of wages). This single amount serves both as cash available to the plan and as taxable employment income, and the employee's payroll tax is charged on it (plus the year's *401k ctrb*, and its *IRA ctrb* in a year with pay; Section 2.4) as a cash outflow. An after-tax contribution paid from pay (Roth 401(k), Roth IRA, taxable) therefore stays in the wages, where it is taxed, and is paid for by a negative *big-ticket items* entry of the same amount;
- *months worked* — months of work at the start of the year, from 0 to 12 (12 when absent in a year with wages), read by the earnings test (Section 2.4);
- *other inc* — other ordinary income not captured elsewhere (enter 0 if not applicable); a negative amount is an above-the-line deduction paid from cash, such as an HSA contribution made without wages;
- *net inv* — rent and trust distributions that count as net investment income for NIIT (enter 0 if not applicable);
- *taxable ctrb* — contributions to the taxable brokerage account;
- *401k ctrb* — the employee's own traditional 401(k) deferrals;
- *employer ctrb* — the employer's contributions (match, profit sharing) to the traditional 401(k), deposited in the tax-deferred account like *401k ctrb* but never wages, so neither payroll-taxed nor counted by the earnings test;
- *Roth 401k ctrb* — Roth 401(k) contributions;
- *IRA ctrb* — traditional IRA contributions;
- *Roth IRA ctrb* — Roth IRA contributions;
- *HSA ctrb* — Health Savings Account contributions (enter 0 if not applicable); the year of Medicare enrollment keeps the months before it, and later entries are ignored;
- *Roth conv* — planned or historical Roth conversion amounts; the five years preceding the plan start date are used to implement the five-year lookback rule (Section 2.7);
- *Roth conv fixed* — a checkbox, not an amount, deciding whether the *Roth conv* figure beside it binds. Unticked, the default, the amount is documentation only and the year's conversion is optimized as usual; ticked, the conversion is held at exactly that amount, bypassing the maximum annual conversion cap and the

start and stop years, so a ticked 0 means no conversion that year. Keeping the flag separate from the amount lets a year's figure stay in place while it is unticked. The flag has no effect on the five lead-in rows, where past conversions always count;
- *QCD* — qualified charitable distributions paid directly from the tax-deferred account to a charity (Section 4.1.2);
- *big-ticket items* — large planned expenses such as a home purchase or college tuition that must be funded from the portfolio in a specific year (negative), or inflows such as an inheritance (positive); a negative entry also pays for an after-tax contribution made from pay.

Only *year* is required. Any other column may be omitted from a person sheet, in which case it is read as zero for every year. The five rows before the plan start serve two rules: the Roth contributions and conversions feed the five-year rule (Section 2.7), and, for someone who claimed Social Security before the plan and before full retirement age, the *anticipated wages*, *months worked*, *401k ctrb*, *employer ctrb*, and *IRA ctrb* feed the earnings test (Section 2.4). The other lead-in entries are not read.

Fixed Assets table

The Fixed Assets table describes illiquid assets held outside the investment accounts: a primary residence, other real estate, collectibles, precious metals, individual stocks, or a lump-sum fixed annuity. Assets may be already held or acquired in a future year. Each row has the following columns:

- *active* — a boolean turning the asset on or off without deleting the row, which is useful for comparing cases; absent or true, the asset is included;
- *name* — a unique identifier for the asset, which is also what a loan in the Debts table names to be paid off by its sale;
- *type* — one of *residence*, *real estate*, *collectibles*, *precious metals*, *stocks*, or *fixed annuity*. The type governs both how the growth *rate* is read and how the proceeds are taxed at disposition, as described below;
- *year* — the reference year, which is the first January in which the asset is held: an asset already owned carries the current year, while one bought during a year is entered as the year after, since it was not there in January. A year already past is reset to the current year on read;
- *basis* — the cost basis, in the nominal dollars of the year it was or will be paid, against which the gain at disposition is measured;
- *value* — the value at the beginning of the reference year, in reference-year

dollars, from which the asset grows to its disposition year;

- *rate* — the annual growth rate, in percent, read as *real* for a *residence*, *real estate*, *collectibles*, and *precious metals*, and as *nominal* for *stocks* and a *fixed annuity*. The default of zero therefore means two different things: for a physical asset it tracks inflation and preserves purchasing power, which long-run US data make a defensible starting point, while for a nominal-rate asset it means no growth at all and a steady loss of real value;
- *yod* — the year of disposition, at the start of which the asset is sold. A non-positive value counts back from the end of the plan, so -1 is the final plan year and -2 the year before it, while 0 is one year past the end. A *yod* landing beyond the plan—the 0 case included—leaves the asset unsold, to be liquidated into the bequest at the end of the plan, untaxed, since property passing to heirs takes a stepped-up basis (IRC §1014);
- *commission* — the sale commission, in percent of the grown value, which reduces both the proceeds and, through them, the taxable gain.

When an asset *is* sold within the plan, the proceeds are the grown value net of commission, the gain is measured against the cost basis from those net proceeds, and the proceeds are split into tax-free, ordinary-income, and capital-gain portions by type:

- *fixed annuity*: any gain over basis is ordinary income, and the basis itself returns tax-free;
- *residence*: the gain is reduced by the principal-residence exclusion of $250,000, or $500,000 for a couple filing jointly (IRC §121(b)(1)–(2)); only the excess is a capital gain, while the basis plus the excluded part of the gain is tax-free. A surviving spouse keeps the $500,000 limit for a sale no later than two years after the death and takes $250,000 after that (IRC §121(b)(4)); with Owl's convention of placing the death at the end of year $n_d - 1$ and a sale at the start of its year, the joint limit applies for $n \leq n_d + 1$;
- *real estate*, *collectibles*, *precious metals*, and *stocks*: the whole gain over basis is a capital gain and the basis returns tax-free.

A sale at a loss makes the entire proceeds tax-free; losses are not carried against other gains. The 28% collectibles rate is not modeled: a collectibles gain is taxed at the long-term rates of Section 2.11.

Debts table

The Debts table lists outstanding loan and mortgage obligations, whose payments are not part of the net spending amount. Each row has the following columns:

- *active* — a boolean turning the debt on or off without deleting the row; absent or true, the debt is included;
- *name* — a unique identifier for the debt, such as "Primary mortgage" or "Car loan";
- *type* — *loan* or *mortgage*. The two are classified separately for the reader's benefit but are treated identically;
- *year* — the start year, in which the loan was taken out or payments begin;
- *term* — the term in years, so that regular payments run from the start year through the year before *year* + *term*;
- *amount* — the principal outstanding at the start year, which is the balance amortized over the term;
- *rate* — the annual interest rate, in percent, from which the constant payment is computed by standard monthly amortization;
- *property* — optionally, the *residence* or *real estate* row of the Fixed Assets table that the loan finances. Naming one ties the loan to it: the loan is paid off in the year that property is sold, out of the sale proceeds, and nothing is paid afterwards. Left empty, the loan runs to its term.

Annual debt-service payments enter the cash-flow constraint as fixed outflows. No optimization takes place on a debt: whether to pay a mortgage down or stay invested turns on risk tolerance rather than on a comparison of rates, which is outside what a linear program over returns can express. A debt left outstanding at the end of the plan is settled from the savings accounts, so a bequest of zero still leaves enough to discharge it—which also means that selling a property late in a plan with a zero bequest can leave the problem infeasible. Mortgage interest is not deducted, since Owl takes the standard deduction.

Cash-flow balance

In every plan year, Owl enforces an annual cash-flow balance. Spending plus all taxes (the employee's payroll tax on wages included), Medicare premiums, and debt payments must equal income from wages, Social Security benefits, account withdrawals, fixed-asset sale proceeds, and other income. Account contributions do not appear in this balance: anticipated wages are entered net of pre-tax contributions, an after-tax contribution paid from pay is offset by a negative big-ticket item (see the Wages and Contributions table above), and the contributed amounts are deposited directly into their accounts through the account-evolution equations. Big-ticket expenses shift additional spending into specific years. The optimizer determines the combination of account withdrawals and Roth conversions—across all account types and both

spouses—that satisfies this constraint while maximizing the chosen objective.

2.4 Social Security Benefits

Rules

Monthly Social Security retirement benefits equal the Primary Insurance Amount (PIA) multiplied by an age-based adjustment factor. The Full Retirement Age (FRA) depends on birth year: 65 for those born before 1938; 66 for those born between 1943 and 1954; and 67 for those born in 1960 or later, with two-month-per-year interpolation for the transition cohorts (born 1938–1942 and 1955–1959). A special Social Security Administration (SSA) rule treats individuals born on January 1st as attaining age on the last day of the prior month, shifting their effective FRA by one month. Reference: SSA POMS RS 00615.003 [2].

Benefits claimed before FRA are permanently reduced: by 5/9 of 1% per month for the first 36 months before FRA, and by 5/12 of 1% per month for any additional months beyond that, yielding a minimum of approximately 70% of PIA at age 62 when FRA is 67. Benefits deferred beyond FRA earn delayed retirement credits of 8% per year up to age 70. Reference: SSA POMS RS 00615.015 [2].

The spousal benefit is up to 50% of the higher-earning spouse's PIA, reduced by the claiming spouse's own PIA (floored at zero), with an additional reduction of 25/36 of 1% per month for the first 36 months before the spousal FRA and 5/12 of 1% for each further month (the spousal FRA mirrors the own FRA schedule) and no bonus for deferral beyond FRA. Delayed credits in the claiming spouse's own benefit do not add to the spousal excess: the combined payment is computed without them and the own benefit with them is subtracted from it, so the credits come out of the spousal part, down to zero (POMS RS 00615.694; 42 U.S.C. 402(k)(3)(A)).

The survivor benefit starts from the deceased spouse's PIA plus the delayed retirement credits earned up to death, fixed at death. It is reduced if the survivor claims before their own survivor FRA (minimum 71.5% at age 60, interpolated linearly in the claiming month). If the deceased claimed before their own FRA, the reduced amount is then capped at the greater of the deceased's reduced benefit and 82.5% of their PIA (Social Security Act § 202(e)(2)(D)); the cap applies after the survivor's age reduction and is not itself reduced. Unlike the retirement benefit, it earns no delayed retirement credits past the survivor FRA, so deferring beyond that date is never advantageous. The survivor benefit and the survivor's own retirement benefit are distinct entitlements with independent claiming dates, and only the larger of the two is paid in any given month; claiming one does not forfeit the other. A

survivor may therefore claim the survivor benefit at 60 and let their own benefit accrue delayed credits to age 70, or the reverse. Reference: CFR § 404.338 [2].

Benefits claimed before FRA are also subject to the *retirement earnings test*, which withholds benefits while the beneficiary works. Below the year of FRA attainment, \$1 of benefit is withheld for every \$2 of earnings above an annual exempt amount; in the FRA year the rate softens to \$1 per \$3 above a much higher exempt amount, and only earnings in the months *before* the FRA month count. From the FRA month on the test no longer applies, whatever the earnings. For 2026 the exempt amounts are \$24,480 below the FRA year and \$65,160 in it. Earnings here are wages before pre-tax deferrals (W-2 box 5) plus net self-employment earnings, not AGI. The excess is charged against the family benefit on the worker's record, month by month from the first month of entitlement in the year, and an auxiliary's own excess is then charged against whatever remains of their benefit. A beneficiary's first year of entitlement also gets a *grace year*: a month in which no work is done, or in which wages do not exceed one twelfth of the annual exempt amount, is paid in full regardless of the annual total. References: SSA POMS RS 02501.021, .025, .030, .080, .095; 20 CFR 404.430–404.439 [17].

Benefits withheld by the test are *not* forfeited. At FRA the SSA performs an *adjustment of the reduction factor* (ARF): the months in which any benefit was withheld, the *crediting months*, are subtracted from the original reduction factor, which is equivalent to recomputing the factor as though the claim had been made that many months later. Writing c_i for the crediting months accumulated by individual i, the factor applied from the FRA month on is

$$\Phi_i^{\text{arf}} \;=\; \min\Big(1,\; \Phi\big(\min(\alpha_i + c_i/12,\, \text{FRA}_i),\, \text{FRA}_i\big)\Big), \tag{2.1}$$

with α_i the claiming age and Φ the adjustment factor of Eq. (9.3). The cap at FRA is what makes the ARF a partial, not a full, restitution: a withheld month raises the benefit for the rest of the beneficiary's life, but never above the unreduced PIA, so an individual who claims at FRA or later and works earns no credit back. For someone born on the first of a month, whose SSA age runs a month ahead, the cap falls one conventional month before FRA_i. A spouse accumulates crediting months from deductions caused by their own work and by the worker's, counted separately on each record. References: SSA POMS RS 00615.480, .482, .490.

The fraction of SS benefits subject to federal income tax ranges from 0% to 85% based on "provisional income" (PI), defined as adjusted gross income (excluding Social Security) plus tax-exempt interest plus half of total SS benefits. Thresholds are \$25,000/\$34,000 (single) and \$32,000/\$44,000 (MFJ), frozen since 1983 and 1994 respectively. Reference: IRS Publication 915 [3].

Owl also accepts a `trim_pct` and `trim_year` to model a projected reduction in SS benefits, for example to reflect a potential trust fund depletion scenario.

Implementation in Owl

SS benefit amounts, FRA schedules, spousal benefits, and survivor benefits are computed from the PIA inputs and each individual's date of birth. Own, spousal, and survivor entitlements are built as three separate time series and then combined as SSA pays them: while both spouses are alive, each receives their own benefit plus any excess spousal amount; from the year of the first death n_d the spousal add-on ends and the survivor receives $\max(\zeta^{\text{own}}_{i_s n}, \zeta^{\text{surv}}_{n})$. The survivor claiming date is set by the parameter `survivor_claim_age`, which accepts `"immediate"` (as soon as eligible), `"FRA"`, or an explicit age; it is never earlier than age 60 or n_d, and is capped at the survivor FRA. When claiming ages are searched (`withSSAges`), each candidate age is a complete, ordinary solve, so all three entitlements—own, spousal, and survivor—are recomputed exactly for that age rather than approximated; see Chapter 9. SS taxability is computed iteratively using the provisional income formula, updated at each pass until it converges; a more exact method is also available via the option `withSSTaxability="optimize"` [3].

Owl always applies the earnings test. The earnings it counts in year n are the wages before pre-tax deferrals (W-2 box 5): the *anticipated wages* plus that year's *401k ctrb*, which those wages are entered net of and which counts even when it took all of the year's pay, plus the *IRA ctrb* in a year with pay (wages or a 401(k) deferral; without pay, the contribution is taken as a spousal IRA funded from the other spouse's pay). *Employer ctrb* is never earnings. The earnings are deflated to today's dollars to match the exempt amounts, and taken as spread evenly over the *months worked* at the start of the year. For someone who claimed before the plan and before FRA, the same quantities are read from the five lead-in rows of the HFP, and the test runs from entitlement on: they decide whether the grace year was already used and which months withheld before the plan are credited at FRA. That month count is what decides whether a month qualifies as a non-service month for the grace-year rule, and, in the FRA year, what share of the year's earnings falls before the FRA month. The excess is floored to the whole dollar and charged month by month against the family's monthly entitlements, splitting a partial last month in proportion to the original entitlement shares; the number of months carrying any deduction is accumulated as the crediting months c_i and applied from the FRA month through Eq. (2.1). Everything the test does is, for a given claiming month, a fixed function of that month: the withheld amounts and the raised post-FRA benefit

are numbers computed before the LP is built, so the test adds no variable and no constraint, and a claiming-age search sees its effect on every candidate. Exempt amounts come from the POMS chart for 2021–2027 and are held at the 2027 level in today's dollars thereafter.

Limitations and assumptions

The SSA family maximum benefit cap (CFR § 404.403) is not modeled; this may overstate benefits when multiple beneficiaries claim on one worker's record. The earnings test is not applied to checks paid from the year of the first death on: survivor benefits under the test, and the survivor ARF of RS 00615.580, are not modeled. Its exempt amounts are indexed to the national average wage with a two-year lag, which Owl does not track, so holding them flat in real terms after 2027 slightly overstates the withholding of later years. Employer contributions belong in *employer ctrb*; entered in *401k ctrb* instead, they are counted as earnings and make them slightly high. Pay that went entirely to a deductible IRA is missed, as the contribution is then taken as a spousal IRA. A payroll HSA contribution is correctly left out, as it is not in W-2 box 5. Self-employment earnings count only where they are entered in *anticipated wages*: the test does not read *other inc.* The monthly test is implemented only as the initial grace year; the substantial-services test for self-employment is out of scope. The survivor claiming date is a user setting rather than an optimized decision: the model evaluates whichever date is specified, but does not search for the one that maximizes the objective. The Windfall Elimination Provision (WEP) and Government Pension Offset (GPO) were repealed by the Social Security Fairness Act (signed January 5, 2025) and therefore require no modeling.

2.5 Pension Income

Overview

A pension is a fixed periodic payment from an employer-sponsored defined-benefit plan, paid for the life of the retiree. Unlike Social Security, pension amounts are set by the plan sponsor and are not subject to federal phase-in rules; they begin in full at the elected commencement age. Some plans offer cost-of-living adjustments (COLAs) that partially or fully index payments to inflation; many do not.

Implementation in Owl

Each individual may have a monthly pension amount and a commencement age. Owl converts the monthly amount to an annual figure and determines the first plan year in which payments begin, using the individual's birth year and birth month to compute the age reached in each calendar year. In the first year of receipt, the annual amount is prorated by the fraction of the year remaining after commencement. For subsequent years the full annual amount (twelve times the monthly benefit) is credited.

When indexed, the annual pension payment grows with inflation each year so that its purchasing power remains constant in real terms. When not indexed, the nominal dollar amount is fixed for the entire duration of the pension. In both cases no pension income is credited before the commencement year. Pension income enters the cash-flow constraint as ordinary income alongside wages, other income, and Social Security benefits.

Joint-and-survivor option

When a pension holder elects a joint-and-survivor (J&S) option, the surviving spouse receives a fraction (e.g., 50%, 75%, or 100%) of the primary's pension after the primary's death. The user specifies the *actual* monthly amount received (already actuarially reduced if J&S was elected) and the survivor fraction as a number between 0 and 1. Starting in the year after the first spouse passes away, the surviving spouse receives that fraction of the deceased's pension each year for the remainder of their horizon. If the primary's pension is inflation-indexed, the survivor benefit follows the same cost-of-living adjustment, so both the primary's pension (while alive) and the survivor benefit grow at the same rate over time. The resulting survivor benefit is added to the surviving spouse's pension income and enters the cash-flow as ordinary income.

Limitations and assumptions

Early-commencement reductions and late-commencement credits are not modeled.

2.6 Single Premium Immediate Annuity (SPIA)

Overview

A Single Premium Immediate Annuity (SPIA) converts a one-time lump-sum premium into a guaranteed income stream that begins immediately and continues for the annuitant's lifetime. Unlike a pension, the income rate (premium-to-income ratio) is market-determined and must be supplied by the user; Owl does not contain an actuarial pricing model. The premium is funded from the individual's tax-deferred account via an IRA rollover—a non-taxable transfer—so no ordinary income is recognized at the time of purchase. Annual payments begin in the year of purchase and are fully taxable as ordinary income for the remainder of the plan horizon.

Implementation in Owl

Each individual may hold one or more SPIAs; each is added independently via `addSPIA()`. In the purchase year, the premium is deducted from the annuitant's tax-deferred balance as a non-taxable IRA rollover; no ordinary income is generated at that time. If the purchase year precedes the plan's start year (an already-purchased SPIA), the premium deduction is skipped and income begins at plan year zero.

Annual SPIA income equals twelve times the specified monthly benefit. When the *indexed* flag is set, the nominal benefit grows with the inflation series each year so that its real purchasing power remains constant; otherwise the nominal dollar amount is fixed for the life of the annuity.

An optional *survivor fraction* ($0 \leq f_s \leq 1$) directs a specified share of the annuitant's benefit to continue to the other individual after the annuitant's death.

SPIA income enters the cash-flow constraint as ordinary income alongside wages, pension payments, and Social Security benefits. It therefore raises Modified Adjusted Gross Income (MAGI) and affects Medicare IRMAA surcharges and the Social Security taxability fraction.

Limitations and assumptions

- The premium is assumed drawn entirely from a tax-deferred account (IRA rollover); funding from taxable or Roth accounts is not modeled.
- Full annual income is credited from the purchase year; partial-year proration in the purchase year is not modeled.
- Annuity pricing (premium-to-income ratio) is user-supplied; Owl contains no actuarial pricing model.

- Payments cease at the annuitant's plan horizon; term-certain and period-certain riders are not modeled.

2.7 Roth Account Rules

Rules

Roth IRA contributions are made with after-tax dollars; qualified withdrawals are entirely tax- and penalty-free. Reference: IRC § 408A [4]. Contribution *principal* may be withdrawn at any time without tax or penalty. Roth *earnings* on contributions require both (a) the account must be at least five years old and (b) the owner must be at least age $59\frac{1}{2}$ (or meet another IRS exception) for a penalty-free withdrawal.

Each Roth *conversion* carries its own independent five-year clock. Conversion principal withdrawn before age $59\frac{1}{2}$ is subject to a 10% early withdrawal penalty if taken within five years of that conversion. Roth conversions are fully taxable as ordinary income in the year of conversion. Reference: IRC § 408A(d)(3); IRS Publication 590-B [5].

Roth IRA accounts are not subject to Required Minimum Distributions (RMDs) during the original owner's lifetime. Roth 401(k) RMDs were eliminated by SECURE 2.0 Act § 325 effective for tax years beginning after December 31, 2023. A 10% early withdrawal penalty also applies to tax-deferred account withdrawals before age $59\frac{1}{2}$ (subject to exceptions not modeled here).

Implementation in Owl

The five-year maturation rule is enforced as a set of minimum-balance constraints that retain recent conversions at compounded value, plus gains-only on recent contributions, as a floor on the Roth account balance. Before age $59\frac{1}{2}$, a second set of constraints keeps cumulative Roth withdrawals within the starting Roth balance, the contributions made since, and the conversions past their five-year period, since by the ordering rules of IRC § 408A(d)(4)(B) anything beyond those comes from earnings or from conversions still in their period. The age-$59\frac{1}{2}$ threshold is computed per individual from their birth month. Conversion amounts are added to taxable ordinary income in the year of conversion. Conversion behavior can be further controlled via the options `maxRothConversion`, `noRothConversions`, `startRothConversions`, `stopRothConversions`, and `swapRothConverters` [4, 5].

Limitations and assumptions

A single unified five-year lookback is applied conservatively: recent conversions at compounded value plus gains-only on contributions are retained as a minimum balance floor. Contribution principal is not separately tracked, so some valid early withdrawals of contribution principal are conservatively disallowed. The pro-rata rule (IRS Form 8606) is not modeled: Owl treats the entire tax-deferred balance as pre-tax, so every dollar converted or withdrawn is fully taxable ordinary income. Users with significant IRA after-tax (nondeductible) basis should be aware of this limitation. Roth conversions are also disallowed in the last two years of the plan horizon.

2.8 Health Savings Accounts

Rules

A Health Savings Account (HSA) is a tax-advantaged account available to individuals enrolled in a High-Deductible Health Plan (HDHP). HSAs offer a unique triple tax advantage: contributions are pre-tax (above the line, reducing AGI), investment growth is tax-free, and withdrawals used for qualified medical expenses are tax-free at any age. After age 65, non-qualified withdrawals are taxed as ordinary income—making the HSA functionally equivalent to a traditional IRA for general use, but without Required Minimum Distributions. Reference: IRC § 223 [6]; IRS Publication 969 [7].

Eligibility requires enrollment in a High-Deductible Health Plan (HDHP) and no disqualifying coverage (e.g., Medicare, non-HDHP insurance). Enrollment in any part of Medicare (Part A, B, C, or D) immediately disqualifies one from contributing; existing HSA balances may still be withdrawn for qualified expenses. Annual contribution limits for 2026 are $4,400 for self-only coverage and $8,750 for family coverage, with an additional $1,000 catch-up for individuals age 55 and older (each spouse with family coverage may add $1,000 to their own account if both are 55+). Contributions must cease once the account owner enrolls in Medicare, which typically occurs at age 65. Reference: IRC § 223(b)(7) [6]; IRS Publication 969 [7].

A surviving spouse who is the named beneficiary inherits the HSA intact and may treat it as their own HSA, preserving all three tax advantages. A non-spouse beneficiary must include the full account balance in taxable income in the year of inheritance. Reference: IRC § 223(f)(8)(B) [6].

Implementation in Owl

Owl models the HSA as a fourth savings account type alongside taxable, tax-deferred, and Roth accounts. Because the dominant retiree use is funding qualified medical expenses, all HSA withdrawals are treated as tax-free (simplified model); see Limitations below. The key modeling choices are as follows.

- *Contributions are pre-tax.* Made through payroll, HSA contributions are netted out of the anticipated wages, like the other pre-tax contributions (Section 2.3), which excludes the contributed dollars from ordinary taxable income, from provisional income used to compute Social Security taxability, and from the modified adjusted gross income (MAGI) used in the two-year Medicare IRMAA lookback. Made from savings without wages, they are entered as a negative *other inc* as well, an above-the-line deduction (IRC 223(a)) that pays for them; in the year of Medicare enrollment, only for the months before it. Contributions stop automatically at Medicare enrollment: the year it begins keeps the months before it, and later HFP entries are ignored.
- *Withdrawals are tax-free and capped by qualified expenses.* HSA withdrawals enter the annual cash-flow balance at full dollar value, identical to Roth account withdrawals, and are excluded from taxable income. Medicare Parts B and D premiums, IRMAA surcharges, and other out-of-pocket medical costs are qualified expenses under IRS Publication 969; Medigap premiums are not. Each year, the household's HSA withdrawals are capped at that year's Medicare costs m_n plus the other qualified medical expenses supplied by the user (`other_medical_expenses`, in today's dollars, indexed by γ_n^{med}). Before Medicare, only the latter count, so without them no HSA withdrawal can be made before Medicare starts.
- *No Required Minimum Distributions.* Unlike tax-deferred accounts, HSAs are never subject to mandatory annual distributions.
- *Cannot exceed account balance.* As with taxable and Roth accounts, the optimizer may not withdraw more than the current HSA balance in any year.
- *Bequest at heirs' tax rate.* The HSA balance remaining at the end of the plan is valued after the heirs' assumed marginal income tax rate, identical to the treatment of tax-deferred accounts. A surviving spouse inherits the HSA intact with no tax (the full balance transfers to their own HSA), but a non-spouse beneficiary must include the full amount in ordinary income. The reported estate value applies the same discount.
- *Asset allocation.* In account-specific allocation mode, the HSA defaults to the same allocation as the Roth account, reflecting that both grow tax-free. Users may also specify a separate HSA allocation.

- *Spousal transfer at death.* At the death of the first spouse, the full HSA balance transfers to the surviving spouse's HSA account, preserving its tax-advantaged status.

Limitations and assumptions

Withdrawals. All HSA withdrawals are treated as qualified medical expenses and therefore entirely tax-free. Non-medical withdrawals after age 65—which would be taxable as ordinary income—are not separately tracked. Before age 65, non-qualified withdrawals incur ordinary income tax plus a 20% penalty; Owl does not model this penalty because all withdrawals are assumed qualified. This simplification is conservative for the dominant retiree use case: funding Medicare premiums and out-of-pocket health costs. Future work may introduce a more detailed model that distinguishes medical from non-medical withdrawals.

Contributions. The annual IRS contribution limit ($4,400 self-only and $8,750 family for 2026) is not enforced as an optimizer constraint; users are responsible for not exceeding it in the *HSA ctrb* column of the Household Financial Profile. The catch-up contribution ($1,000 for age 55+) is not automatically applied; users aged 55+ must add it manually if desired. Self-only vs. family coverage is not distinguished; the applicable limit is the user's responsibility. The mandatory contribution stop at Medicare enrollment is enforced month by month. `setHSA()` is invoked either explicitly or automatically when loading from configuration (e.g., TOML). In the year of enrollment, the IRS allows prorated contributions for the months before it [8], and Owl keeps that share of the *HSA ctrb* entry; entries for later years are overwritten with zero.

Eligibility. Owl assumes the user is HSA-eligible (enrolled in an HDHP with no disqualifying coverage). HDHP status, disqualifying coverage, and dependent status are not verified.

Bequests. Non-spouse beneficiary inheritance is modeled by applying the heirs' marginal income tax rate to the terminal HSA balance, identical to the treatment of tax-deferred accounts. The simplified qualified-withdrawal model (all HSA withdrawals tax-free during the plan horizon) is not extended to non-qualified withdrawals after age 65; those remain deferred to a future path.

2.9 ACA Marketplace Premiums (Pre-65)

Rules

Before Medicare eligibility at age 65, individuals and couples may obtain health coverage through the Health Insurance Marketplace (Exchange) established by the Affordable Care Act. The Premium Tax Credit (PTC) reduces the net premium cost for Marketplace plans based on household modified adjusted gross income (MAGI) and the Federal Poverty Level (FPL). Reference: IRC § 36B [9].

The applicable percentage—the share of income a household is expected to contribute toward the benchmark plan—varies with the ratio of MAGI to FPL. The IRS publishes the applicable percentage table annually. For 2025, Rev. Proc. 2024-35 provides the table under the Inflation Reduction Act (IRA), which suspends indexing and caps the contribution at 8.5% for income above 400% FPL. Reference: Rev. Proc. 2024-35 [10].

For 2026 and beyond, Rev. Proc. 2025-25 provides the applicable percentage table when the IRA enhanced subsidies have expired: indexing resumes, contribution percentages are higher, and there is no subsidy for income above 400% FPL (the "subsidy cliff" returns). Reference: Rev. Proc. 2025-25 [11].

Federal Poverty Level guidelines are published annually by the Department of Health and Human Services (HHS). A coverage year uses the guideline *published the year before*, since the statute takes the most recently published poverty line as of the first day of the open-enrollment period. The guidelines published in 2025, which price coverage year 2026, are $15,650 (one person) and $21,150 (two) for the 48 contiguous states and DC; those published in 2026, pricing coverage year 2027, are $15,960 and $21,640. Reference: HHS Poverty Guidelines [20].

Household size for the poverty line is the *tax household* — two for a couple filing jointly while both are alive, even when one spouse has reached Medicare (IRC 36B(d)(1); 26 CFR 1.36B-1(d)(2)). Who is *enrolled* is a separate question, settled month by month: an individual is covered in the months of a year before Medicare begins, and where only one partner of a couple is enrolled the benchmark is the couple's premium scaled by the CMS age-rating fraction for that partner. Up to the state's Medicaid limit the household is covered by Medicaid at *no* premium: 138% of FPL in expansion states, 100% in Wisconsin, and no Medicaid at all in the remaining states. Where that limit falls below 100% of FPL, income below 100% receives neither Medicaid nor the credit, which starts at 100% (IRC 36B(c)(1)(A)): the *coverage gap*, in which the household pays the full benchmark premium. Income of exactly 400% of FPL still qualifies, since the statute covers income that "does not

exceed 400 percent."

Implementation in Owl

Owl models ACA net premium costs for pre-65 years when the user supplies the annual benchmark Silver plan (SLCSP) premium via `setACA()`. Two computation modes are available.

In the default `withACA="loop"` mode, ACA costs are computed after each solver pass using `acaCosts()`, which applies the IRS applicable-percentage table to MAGI and FPL for each plan year. Rules are year-aware, by taxable year: 2025 uses Rev. Proc. 2024-35, 2026 uses Rev. Proc. 2025-25, 2027 uses Rev. Proc. 2026-26, and a year past the last published table reuses it. Within Medicaid the premium is zero; in a coverage gap the loop charges the lowest applicable percentage as though from 100% of FPL, because the exact cost jumps to the full benchmark below 100% and the loop would oscillate across that cliff. Optimize mode prices the gap exactly, with a bracket of its own.

In `withACA="optimize"` mode, ACA bracket selection and cost are folded into the optimization. The formulation uses 8 MAGI brackets — 7 intervals up to 400% of FPL plus one above — with SOS1 binary selection, MAGI decomposition, and a piecewise cost. Bracket 0, below the first threshold, is Medicaid at no premium or the coverage gap at the full benchmark; brackets 1–6 follow the sliding scale, each priced from below by the maximum of four tangent lines to the convex cost within the bracket; bracket 7, at 400% and above, imposes the full SLCSP. The bracket thresholds are set by the year's Medicaid limit, so bracket 1 has zero width wherever Medicaid reaches past the lowest band. ACA uses current-year MAGI (no two-year lag like Medicare IRMAA). Unlike IRMAA and the NIIT, the ACA MAGI is the full-Social-Security variant $\mathbb{G}_n^{\text{aca}}$ (IRC §36B adds the non-taxable portion of Social Security back to AGI).

Limitations and assumptions

The SLCSP benchmark premium is user-supplied and inflated by the plan's inflation factor. FPL and applicable-percentage tables are updated annually in the code; manual updates are required when HHS or IRS publish new values. If Congress extends IRA subsidies (e.g., the 8.5% cap above 400%), parameters must be updated accordingly. Cost-sharing reductions and per-year benchmark premiums by band are not modeled: the benchmark is one user-supplied figure, inflated.

2.10 Medicare and IRMAA Premiums

Rules

Medicare Part B eligibility begins at age 65. The standard 2026 monthly premium is $202.90 ($2,434.80/year). Reference: Centers for Medicare & Medicaid Services (CMS) 2026 Medicare Part B premium announcement [21].

The Income-Related Monthly Adjustment Amount (IRMAA) adds a surcharge based on MAGI from two years prior. In 2026, the five incremental per-person per-month surcharges are $81.20, $121.70, $121.70, $121.70, and $40.70, applied at MAGI thresholds of $109k, $137k, $171k, $205k, and $500k (single); for married filing jointly (MFJ) the first four are doubled, to $218k, $274k, $342k, and $410k, and the top threshold is $750k. Reference: CMS 2026 IRMAA tables [21].

MAGI for IRMAA purposes is the beneficiary's adjusted gross income (AGI, IRS Form 1040 line 11) plus tax-exempt interest income (line 2a). It therefore includes all ordinary taxable income, LTCG, and dividends, but only the *taxable* portion of Social Security benefits (the up-to-85% already in AGI); the non-taxable remainder is *not* added back. This differs from the Affordable Care Act definition (Section 2.9), which does add back non-taxable Social Security. Reference: SSA POMS HI 01101.010 [2]. For couples, premiums reflect both individuals when both are Medicare-eligible. The table that applies is that of the filing status on the lookback return (20 CFR 418.1115(c)): a couple files jointly in the year of the first death, so the survivor's first two years after it still use the joint table, and the single table starts in the third. A survivor's request to use a more recent year (418.1205(a)) is not modeled.

Implementation in Owl

Owl determines which IRMAA bracket applies to each year's MAGI and computes the exact cumulative premium for that bracket. Two computation modes are available. The default, `withMedicare="loop"`, computes IRMAA costs after each solver pass. Setting `withMedicare="optimize"` folds bracket selection directly into the optimization for an exact result. The two-year MAGI lookback is fully implemented. Because IRMAA in plan year n depends on MAGI from year $n-2$, the first two plan years require MAGI values from before the plan start. These are supplied by the user via the `previousMAGIs` option as a list of the two MAGI values from two and one years before the plan start. When not supplied, both default to zero, which may underestimate IRMAA surcharges in the first two plan years for higher-income households. Reference: [21].

Part D and assumptions

Medicare Part D is modeled using the same MAGI brackets and two-year lookback as Part B. Part D IRMAA surcharges (CMS 2026) are included by default; the optional Part D base premium is configurable (default zero). Users can disable Part D via `includeMedicarePartD=false` when they have other drug coverage (e.g., employer or VA). IRMAA thresholds in Owl are assumed to scale with the general inflation rate; in practice, CMS adjusts brackets annually but not necessarily by the Consumer Price Index (CPI). The top tier is the exception: the Bipartisan Budget Act of 2018 froze it at $500,000 ($750,000 joint) in nominal dollars, and Owl holds it there until indexing resumes in 2028.

Medical price inflation

Medical prices have historically outgrown general inflation, and a plan may say so. The parameter `setMedicalInflation(excess)` takes a yearly growth above inflation, in percent and bounded to the interval $[0, 10]$, and medical prices then grow by the compounded factor

$$\gamma_n^{\text{med}} \;=\; \gamma_n \,(1 + \epsilon^{\text{med}})^{\,n}, \tag{2.2}$$

with ϵ^{med} the excess as a decimal and γ_n the general inflation factor. The default of zero keeps medical prices in step with inflation, recovering $\gamma_n^{\text{med}} = \gamma_n$.

What γ_n^{med} scales is a deliberate, asymmetric choice: the Medicare Part B and Part D base premiums and their IRMAA surcharges, the ACA benchmark premium, and the household's other qualified medical expenses. What it does *not* scale is anything statutory—the tax brackets, the standard deduction, the IRMAA bracket thresholds, and the poverty line all keep following γ_n alone. That asymmetry is the point of the parameter: a positive excess does not merely raise a cost, it pushes medical costs *through* thresholds that are not moving with them, so IRMAA tiers and the ACA bands are crossed earlier than general inflation alone would suggest.

2.11 Long-Term Capital Gains Tax

Rules

Long-term capital gains (LTCG) and qualified dividends are taxed at preferential federal rates of 0%, 15%, or 20% based on total taxable income (ordinary income plus LTCG combined). Reference: IRS Topic 409 [22]; IRS Publication 550. LTCG

"stacks on top of" ordinary income: the portion of gains that pushes combined taxable income above each threshold is taxed at the corresponding higher rate. For 2026, the 0%/15% threshold is $49,450 (single) or $98,900 (MFJ); the 15%/20% threshold is $545,500 (single) or $613,700 (MFJ). These thresholds are indexed for inflation annually.

Implementation in Owl

LTCG bracket allocation is always active (no option required). Total gains are split across the three rate brackets; because the rates increase (0%, 15%, 20%), the optimizer naturally uses the lowest-rate brackets first. The upper limit of each bracket depends on how much ordinary income already occupies the corresponding threshold; these limits are refined at each solver pass until they converge. The total LTCG tax is derived from the bracket allocation after solving. Reference: [22].

An optional MIP formulation is available via `withLTCG="optimize"`. In this mode, the ordinary taxable income G_n is embedded as a continuous LP variable together with binary variables that select the applicable LTCG bracket in each year, replacing the self-consistent loop refinement of bracket thresholds with an exact mixed-integer constraint.

Limitations and assumptions

Short-term capital gains (assets held one year or less) are taxed as ordinary income. Owl does not separately model short-term versus long-term gains within the taxable account; all realized gains are treated as long-term. The last line of the Qualified Dividends and Capital Gain Tax Worksheet takes the lesser of the stacked tax and the regular tax on all taxable income (IRC 1(h)(1)); Owl keeps the stacked tax. The two differ only because the 0% breakpoint is below the top of the 12% bracket ($49,450 and $50,400 single, $98,900 and $100,800 joint in 2026): gains in that gap pay 15% where the regular tax charges 12%. The overstatement is at most 3% of the gap, $28.50 single or $57 joint a year, and only in a year whose gains fall in it, while taking the minimum exactly would need a binary variable per year.

2.12 Net Investment Income Tax

Rules

The Net Investment Income Tax (NIIT) is a 3.8% surtax imposed by the Affordable Care Act on net investment income (NII) for taxpayers whose MAGI exceeds $200,000 (single) or $250,000 (MFJ). Reference: IRC § 1411 [23]; IRS Form 8960. The thresholds are *not* indexed for inflation. The tax equals 3.8% of the lesser of net investment income and the amount by which MAGI exceeds the applicable threshold. NII includes dividends, interest, LTCG, rents, and royalties, but excludes wages, Social Security benefits, tax-deferred retirement distributions, and Roth withdrawals.

Implementation in Owl

The NIIT is computed annually from MAGI and net investment income (dividends, interest, LTCG, and any rent or trust income entered in the `netinv` column). Rent and trust income from `netinv` is included in NII and also flows into cash-flow and taxable-income as ordinary income. The tax uses the IRS formula described above. Because the NIIT depends on the solution, it is held fixed during each solver pass and updated from the new income values until it converges. The threshold is fixed in nominal dollars; its real impact over time is correctly reflected in the model. Reference: [23].

An optional MIP formulation is available via `withNIIT="optimize"`. In this mode, MAGI is embedded as a continuous LP variable together with a binary variable z_n^j that selects, each year, which term of the IRS $\min(\cdot, \cdot)$ bounds the tax: the excess of MAGI over the threshold, or net investment income. Because the optimizer minimizes taxes, it picks the smaller, reproducing the IRS value over every income range.

Limitations and assumptions

None.

2.13 Ordinary Income Tax

Rules

The US federal income tax is a progressive tax applied to taxable ordinary income (adjusted gross income, or AGI, minus deductions). Reference: IRS Publication 17 [24]. Under the One Big Beautiful Bill Act (OBBBA, signed July 4, 2025), seven marginal brackets apply at rates of 10%, 12%, 22%, 24%, 32%, 35%, and 37%. Standard deductions for 2026 under the OBBBA are $16,100 (single) and $32,200 (MFJ). An additional deduction of $2,050 (single) or $1,650 (MFJ) per eligible individual applies at age 65 or older.

OBBBA § 1002 provides an extra $6,000 per-person senior bonus deduction for taxpayers aged 65 or older, phasing out at $6 per $100 of MAGI above $75,000 (single) or $150,000 (MFJ) and fully phased out at $175k/$250k. This bonus expires after tax year 2028. Reference: [25].

Required Minimum Distributions (RMDs) are mandatory annual withdrawals from tax-deferred accounts, beginning at age 70 for those born before 1949, age 72 for those born 1949–1950, age 73 for those born 1951–1959, and age 75 for those born in 1960 or later (effective 2033), per SECURE 2.0 Act § 107 [26]. RMD fractions are taken from the IRS Uniform Lifetime Table III (effective 2022), as published in IRS Publication 590-B [5], except when the sole designated beneficiary is a spouse more than ten years younger, in which case IRS Table II (Joint and Last Survivor) applies. Roth accounts are exempt from RMDs. Filing status transitions from MFJ to single in the calendar year after the first spouse's death, as a couple files jointly in the year of death.

Owl can also model a future reversion to pre-OBBBA tax rates (10%, 15%, 25%, 28%, 33%, 35%, 39.6%) from a user-specified calendar year (`yOBBBA` parameter), enabling scenario analysis of potential future tax law changes.

Implementation in Owl

Ordinary income is allocated across the seven tax brackets, with bracket widths and rates updated each year for inflation and for the OBBBA sunset schedule. RMD fractions are computed per individual and year from the IRS Uniform Lifetime Table. The OBBBA 65+ bonus deduction phaseout is computed from the current MAGI and refined at each solver pass, or carried inside the optimization with `withSeniorBonus="optimize"` (Section 4.1.5). Reference: [24, 26, 25].

Limitations and assumptions

State income taxes are modeled separately; see the next section. Itemized deductions are not modeled; only the standard deduction (plus the 65+ and OBBBA bonus deductions) is applied. By default, the OBBBA 65+ bonus deduction phaseout is computed outside the LP (as a post-solution update) because it is a nonlinear function of MAGI; under `withSeniorBonus="optimize"`, a binary per bonus year carries it in the MIP instead. The IRS Uniform Lifetime Table III is used for standard cases. When the spouse is the sole designated beneficiary and more than ten years younger than the account owner, IRS Table II (Joint and Last Survivor, Pub. 590-B) is applied instead. Inherited IRA and beneficiary RMD rules are not modeled.

2.14 State Income Tax

Rules

Most US states levy a personal income tax on top of the federal tax, each with its own bracket structure, standard deduction, and filing statuses. The treatment of retirement income varies widely: the majority of states exempt Social Security benefits, several exempt all retirement income, and many others offer capped exclusions for pension or retirement-account income. A few states exempt Social Security only below an income threshold.

Implementation in Owl

A plan may specify a two-letter state code. Bracket rates and widths, standard deductions, Social Security treatment, and retirement-income and pension exemption caps for all 50 states and the District of Columbia are read from a data table (`data/taxes_state.toml`) with separate entries for Single and Married Filing Jointly filers; the filing status switches to Single in the year after the passing of a spouse, and bracket widths and deductions are indexed to inflation through γ_n only where the state indexes them: most state brackets are fixed in nominal dollars. State taxable income is derived from quantities already present in the model: federal ordinary taxable income with the federal standard deduction added back, plus capital gains (which most states tax as ordinary income), minus the exempted portion of Social Security, pension exemptions, the state standard deduction, and any capped retirement-income exemption. A plan can change its state of residence once (option `moves`): the new state taxes the year of the move and every year after it, as the state

of residence on December 31 taxes the whole year. The resulting state tax joins the cash-flow constraint alongside its federal counterpart. The LP formulation is given in Sec. (4.1.19). States without an income tax simply generate no state-tax variables.

Limitations and assumptions

State AGI is approximated from federal quantities; state-specific additions and subtractions beyond those listed above are not modeled. Capital gains are taxed as ordinary income, which is accurate for most but not all states. Each state either exempts Social Security in full or taxes its federally taxable part; no AGI threshold for the exemption is modeled. Kansas and Missouri, which had such thresholds, exempt the benefit in full from tax year 2024 (Kansas SB 1, 2024 special session; Missouri for filers 62 and older), and are treated as exempting it. Personal and senior credits are modeled where a state offers them, capped by the year's state tax (Eq. (4.53)); other state credits are not. Local and county income taxes, and itemized deductions, are not modeled.

3. Model Architecture and Solution Approach

3.1 Overview

Owl formulates retirement planning as a linear program (LP) or, when binary variables are required, a mixed-integer program (MIP). The plan is divided into annual time steps covering the entire household planning horizon. In each year, the optimizer simultaneously determines withdrawal amounts from all accounts, Roth conversion amounts, and spending levels, subject to tax laws, account balance dynamics, and user-specified constraints. The resulting solution determines the optimal sequence of financial decisions across the full horizon in a single solve—not year by year.

A complication arises because several tax quantities depend nonlinearly on income, yet also influence the cash-flow balance that income must satisfy. Owl resolves this circularity through a self-consistent (SC) iteration loop: nonlinear quantities are fixed at their previous-iteration values during each LP solve, then recomputed from the new solution, and the process repeats until convergence. Threshold quantities can instead be carried inside the optimization as binary variables (MIP reformulations); the loop still iterates the quantities that remain nonlinear in the solution.

3.2 Planning Horizon and Time Steps

The plan horizon runs from the current calendar year (year 0) through the year of the last individual's assumed death. Each year index corresponds to one calendar year; all decision variables are resolved at annual granularity.

For a single individual, the horizon spans the years from the plan start through the assumed year of death, which is derived from a user-supplied life expectancy or target age. For married couples, the shorter-lived spouse's death marks the transition from a joint plan to a survivor plan. Both death years are inputs to Owl; they are

treated as certain within a given scenario. Monte Carlo and historical stress-test simulations run the optimizer repeatedly across many return scenarios. When the longevity-risk option is enabled in stochastic spending optimization (Chapter 7), ages at death are drawn for each scenario from a selectable actuarial period life table (Section 7.3), so that the planning horizon itself becomes a random variable.

The partial first year

Every plan year runs from January 1, so year 0 is a full calendar year even when the plan is built in the middle of one. Balances, however, are known as of a date the user supplies—today by default—and Owl reconciles the two by *back-projecting* the balances to January 1, dividing out the growth that the elapsed part of the year would have produced at year-0 rates and allocations:

$$b_{ij0} \;=\; \frac{\beta_{ij}}{1 + (1-\varphi)\sum_k \alpha_{ijk0}\,\tau_{k0}}, \qquad \varphi \;=\; 1 - \frac{d-1}{365+\ell}, \tag{3.1}$$

where β_{ij} is the balance as entered, d is the day of the year of the balance date, ℓ is 1 in a leap year, and φ is the fraction of the year still ahead. After a first-year loss the back-projected balance exceeds the balance entered, as it should.

The back-projection removes *growth* only. It does not add back the money that has already moved through the accounts since January 1, so year 0 counts a full year of spending, income and contributions whatever the balance date. The consequences run in opposite directions: spending already paid this year, net of income already received, is charged again, which is conservative for a retiree drawing down savings; contributions already made are deposited again, which is generous for someone still accumulating. On one retiree case the net effect of an October 1 date rather than January 1 is a result about 2% lower.

Two remedies are available, and neither requires a model change. Entering January 1 balances—from a year-end statement—with a January 1 date makes the first year exact. Otherwise the first-year row of the Wages and Contributions table should carry only the contributions, QCDs, and big-ticket items *still to come*, since those already made have already moved the balance that was entered; wages, Social Security, and pensions stay at their full-year amounts, because income received so far is what offsets the spending already paid. Spending is the one flow that cannot be split this way, being an annual target rather than a schedule. To offset it, add to the taxable balance the spending and taxes paid since January 1 *minus* the wages, Social Security, and pensions received since January 1: the net, not the spending alone, because the year-to-date income is counted again too.

Because the balance date shifts every opening balance through φ, re-stating balances must not silently re-date them: `setAccountBalances()` leaves `startDate` unchanged unless it is given one explicitly. Reports show the whole first year like any other, and the case summary adds the row *Net spending remaining in year*, $g_0\,\varphi$, which is what is left to spend from the balance date on.

3.3 Account Dynamics

Each individual maintains up to four savings accounts: a taxable brokerage account, a tax-deferred account, a Roth account, and a Health Savings Account (HSA). A married couple therefore has up to eight accounts in total, tracked and optimized jointly.

At the start of each plan year each account balance grows by the year's investment return for that account's asset allocation. During the year the following flows are applied:

- contributions from the Wages and Contributions table are deposited (modeled at mid-year so they earn half a year of return);
- withdrawals are made to fund spending, pay taxes, and cover other expenses including debt service and big-ticket items;
- Roth conversions transfer a chosen amount from the tax-deferred account to the Roth account, adding the converted amount to that year's ordinary taxable income;
- the taxable account generates a dividend and interest yield proportional to its balance, which is taxed as ordinary income or at preferential qualified-dividend rates;
- Required Minimum Distributions enforce a minimum withdrawal from each tax-deferred account once the owner reaches the applicable RMD starting age;
- any spending surplus—income exceeding expenses in a given year—is deposited back into the taxable account;
- HSA contributions are pre-tax (netted out of anticipated wages) and cease at Medicare enrollment; HSA withdrawals fund qualified medical expenses entirely tax-free and are not included in taxable income;
- at the death of the first spouse, a user-specified fraction of each account is transferred to the surviving spouse's corresponding account, reflecting beneficiary designations; the surviving spouse inherits the HSA intact at full tax-advantaged status.

The balance at the start of the following year is the result of all of these flows. All accounts are assumed to be rebalanced to their target asset allocation at the end

of each year.

3.4 Asset Allocation and Glide Paths

The target allocation of each account across asset classes (equities, corporate bonds, Treasury notes, and cash) is specified by the user. Allocation can be prescribed at three levels of granularity: per individual per account, per individual across all accounts combined, or identically for the entire household.

When the user specifies starting and ending allocation values, Owl interpolates smoothly over the plan horizon using either a linear glide path or an S-curve (hyperbolic tangent) glide path. The S-curve produces a gradual, natural transition that accelerates through the midpoint and tapers at both ends, which is often a more realistic model of how investors de-risk their portfolios over time. For couples, each individual's allocation glides independently over their own planning horizon; the survivor's allocation therefore continues from whatever value it had reached at the moment of the first spouse's death.

3.5 Spending Profile and Objectives

Spending profile

Net spending in each year follows a user-specified *spending profile* ξ_n that scales a single level the optimizer chooses, the *basis* g: real net spending in year n is $g\,\xi_n$. The profile encodes the desired shape of spending over time; the optimizer determines the scale that is achievable given the household's resources and constraints.

Two profiles are available. A *flat* profile holds real spending constant, $\xi_n = 1$. A *smile* profile spends more in early and late retirement and less in the middle years, built as a cosine dip of amplitude a (15% by default) plus a linear increase b (12% by default) over the horizon, optionally delayed by a number of years:

$$\xi_n \;\propto\; 1 + a\cos\!\left(\frac{2\pi(n-n_0)}{N_n - 1 - n_0}\right) + \frac{b\,(n-n_0)}{N_n - 1}, \qquad n \geq n_0, \tag{3.2}$$

held flat at its first value for $n < n_0$. At the death of the first spouse, in year n_d, spending is multiplied by a user-specified survivorship fraction χ reflecting the lower expenses of a single person.

The smile profile is then *normalized* so that $\sum_n \xi_n = N_n - (1-\chi)(N_n - n_d)$, the sum a flat profile with the same survivorship would give. This makes the basis profile-neutral: it is the *average* of the yearly amounts over the plan, not the first year's. The

distinction matters whenever the profile is not flat, since the two numbers then differ, and the reporting surfaces do not all use the same one: the desired net spending a user enters, and the spending axes of the trade-off and spending optimization tools, are *first-year* amounts, while the library keys and the conversion-regret measure are in basis dollars. The case summary reports both. On a flat profile $\xi_0 = 1$ and the distinction disappears.

Primary objectives

Two optimization objectives are supported:

Maximize net spending.
Subject to a desired bequest—which may be zero—the optimizer maximizes the sum of inflation-adjusted annual spending over the full plan horizon. This is the most common objective for retirees who want to sustain the highest possible standard of living.

Maximize bequest.
Subject to a fixed annual spending level, the optimizer maximizes the after-tax estate value delivered to heirs at the end of the horizon. The heir's marginal income tax rate on inherited tax-deferred balances can be specified by the user.

Solver options for spending and bequest

One option modifies how the primary objectives are formulated:

- `timePreference` (%/year, default 0) — a subjective time-discount rate ρ that front-loads spending. The objective weight for year n is multiplied by $(1+\rho)^{-n}$, so near-term spending is valued more than equivalent spending in the distant future. At $\rho = 0$ (the default) all years carry equal real weight.

A `fixedSpending` option, which pinned g_0 to a committed level under `maxSpending`, has been withdrawn. With the default zero profile slack, the spending-profile rows tie every g_n to g_0, so pinning g_0 fixes the whole spending vector. The `maxSpending` objective is assembled solely from the g_n coefficients and therefore became *constant* over the feasible set. Since the bracket-fill variables $f_{t,n}$ are a relaxation of the progressive schedule, tight only because the objective pushes ordinary income into the cheapest brackets, the solver was then free to fill the top bracket while lower ones sat empty; the resulting excess tax was absorbed by the estate at no cost to a constant objective. Raising the slack does not repair this, because the upper edge of the slack band is itself proportional to the pinned g_0, so the objective merely goes flat at saturation instead.

The well-posed replacement is to sweep the `bequest` floor under `maxSpending`, tracing the spending–bequest efficient frontier. Every point of that sweep leaves g_0 free, so the objective retains a nonzero gradient and prices the tax variables, and each point is a genuine optimum rather than an arbitrary vertex of a degenerate program.

Lexicographic weight

A small secondary weight can be added to the objective to break ties among solutions with the same primary objective value. By default, this weight mildly discourages unnecessary Roth conversions (preferring fewer conversions when spending is otherwise unaffected) and mildly prefers drawing withdrawals from the first individual before the second when both choices are equivalent. The weight is small enough that it never overrides the primary objective.

3.6 The Self-Consistent Iteration Loop

Several quantities that enter the LP constraints are nonlinear functions of the income variables being optimized. The most important are:

- the *Social Security taxability fraction*—the share of SS benefits subject to ordinary income tax—which depends on provisional income, which in turn includes SS benefits themselves and all other income sources;
- *Medicare IRMAA costs*, which depend on MAGI from two years prior and interact with the current year's spending and tax calculations;
- the *Net Investment Income Tax (NIIT)*, which depends on MAGI and on investment income;
- the *LTCG bracket upper bounds*, which depend on how much of each threshold is already occupied by ordinary income;
- the *OBBBA 65+ senior bonus deduction phaseout*, which is a nonlinear function of MAGI;
- *ACA marketplace premiums*, which depend on MAGI through the premium tax credit;
- the *gain fractions* of the taxable accounts, when a cost basis is supplied, which depend on the withdrawals and deposits of earlier years.

Owl handles these nonlinearities through the SC loop. At each iteration, all nonlinear quantities are held fixed at their values from the previous iteration, making the problem an LP (or MIP). The solver produces a new income solution; the nonlinear quantities are recomputed from that solution and compared to their previous values.

The loop terminates when two conditions hold together: the objective has settled within `absTol` or `relTol`, and the fixed-point residual, the largest disagreement between a quantity the LP was built with and the value the resulting plan implies (summed over the horizon in today's dollars, per tax family), is at most `residualTol` per year. An objective that has settled is not enough on its own: an iterate whose own income implies different premiums or a different taxable share of Social Security is not a fixed point. Otherwise the updated values are substituted and the process repeats. Within each solve the fed-back quantities are constants, so the optimizer cannot trade income against them (an IRMAA tier, the Social Security phase-in, the senior-bonus phase-out): the loop can settle on a plan that is self-consistent and still not the best one. The `"optimize"` formulations below remove this for the families they cover, as they carry the thresholds inside the optimization.

The loop also stops on three other conditions. When the objectives fall into a repeating cycle, the best iterate of the cycle is kept. When branch-and-bound solves keep timing out without improvement (three time-outs in eight iterations), or when `maxIter` iterations have run, the iterate with the best objective is kept, the first iteration excepted since its initial guesses undercharge. When an iterate cannot be solved at all, the loop falls back on the *most settled* earlier iterate, the one with the smallest residual, not the richest: the fed-back costs start at nothing and catch up pass by pass, so the best objective belongs to the iterate least charged for what its income brings.

To prevent oscillation near the SS taxability thresholds—where small changes in income can flip the taxability fraction back and forth—the updated SS taxability estimate is blended with the previous estimate using a damped update, 30% new and 70% old. The loop typically converges in fewer than fifteen iterations for well-posed plans.

Six optional MIP formulations replace portions of the SC loop with binary constraints, carrying the corresponding thresholds inside the optimization:

- `withSSTaxability="optimize"` — models SS taxability as an piecewise-linear MIP using binary variables z^{σ} to select the correct taxability threshold (0%, 50%, or 85%) each year.
- `withLTCG="optimize"` — uses binary variables z^{l} to select the correct 0%, 15%, or 20% LTCG bracket each year, rather than approximating bracket thresholds from the previous SC iteration.
- `withNIIT="optimize"` — uses a binary variable z^{j} to select, each year, which term of the IRS minimum ($\min(\mathbb{G} - \mathbb{G}^{\text{NIIT}}, \mathbb{I} + Q)$) bounds the tax.
- `withACA="optimize"` — folds ACA premium bracket selection and MAGI into the MIP, replacing the per-iteration SC approximation.

- `withMedicare="optimize"` — models IRMAA bracket selection using binary variables z^m to select each year's bracket.
- `withSeniorBonus="optimize"` — carries the OBBBA senior-bonus phase-out, with one binary z_n^{sb} per bonus year marking it fully phased out (Section 4.1.5).

The loop still runs in every mode: the investment income, the gain fractions, and the senior deduction of a state that conforms to the federal one remain nonlinear, so even a branch-and-bound solve of every family is not exact. Each formulation removes the corresponding SC-loop approximation at the cost of additional binary variables. When multiple flags are active simultaneously, the resulting MILP can be large; Section 3.9 describes the two strategies that solve it.

3.7 Filing Status and the Death of a Spouse

For married couples, the year of the first spouse's death is a known input to the plan. Beginning in the year after it, Owl automatically transitions all relevant quantities from married-filing-jointly (MFJ) status to single status:

- the standard deduction drops from the MFJ amount to the single-filer amount;
- tax bracket widths narrow to the single-filer schedule;
- Social Security taxability thresholds shift to the single-filer values;
- Medicare premiums reflect only the surviving spouse;
- the survivor begins receiving the survivor benefit computed at the year of the spouse's death;
- the spending profile is multiplied by the user-specified survivorship factor.

Because the death year is fixed at plan-construction time, this transition requires no binary variables. Separate parameter vectors are constructed for the MFJ period and the single-survivor period; the LP selects the correct values automatically for each year.

3.8 Solvers

Owl's optimization model is assembled in a solver-agnostic matrix form: inequality constraints, equality constraints, variable bounds, and an objective vector. Any LP or MIP solver that accepts this standard input format can be used.

Two solvers are supported: **HiGHS**, a high-performance open-source LP and MIP solver accessed directly via the `highspy` Python package, and **MOSEK**, a commercial solver requiring a separate license, which generally provides faster solution times for large MIP problems. The default, `solver="default"`, selects MOSEK

when it is installed and licensed and HiGHS otherwise.

Two retries guard against verdicts HiGHS can get wrong. A linear program that HiGHS ends without a verdict (no status, an unknown status, a solve, presolve or postsolve error, or "unbounded or infeasible"), and without a feasible point, is solved once more with the interior-point method and presolve off, which settles it either way; whether HiGHS's default path stops there can turn on rounding in the last digits. A mixed-integer program that HiGHS reports infeasible is retried, cheapest first, with one presolve rule turned off and then with presolve off, before the verdict is believed.

3.9 MIP Strategies: Branch-and-Bound and Local Search

When the `"optimize"` flags are active simultaneously—`withSSTaxability`, `withMedicare`, `withACA`, `withLTCG` and `withNIIT`—the MILP carries on the order of 300 binary variables from the five bracket-selector families (z^{σ}, z^{m}, z^{a}, z^{l}, z^{j}), plus at most three senior-bonus binaries z^{sb} under `withSeniorBonus`; with every tax mode in loop mode the problem is a pure linear program. The option `mipStrategy` selects how those binaries are solved within each iteration of the SC loop.

Branch-and-bound (`mipStrategy="branch-and-bound"`, the default) hands the whole MILP to the solver's complete search. It stops when the relative gap between the incumbent and the best bound falls below `gap`, or at `maxTime`, so its answer is certified only to that gap. It is not exact in a wider sense either: it is solved inside the SC loop, which still iterates the quantities that remain nonlinear in the solution (investment income, gain fractions, the OBBBA phaseout unless `withSeniorBonus` is optimized, and the senior deduction of a state that conforms to the federal one). Solve times grow quickly with the binary count; on the bequest cases studied, a median of about six minutes per solve with MOSEK, and HiGHS is far slower.

Local search (`mipStrategy="local-search"`) is a fix-and-optimize heuristic. Within each SC iteration it solves a sequence of small restricted problems, each with most binaries pinned:

1. *A feasible starting plan.* The previous iteration's binaries (an LP); failing that, a repair from the self-consistent loop's own plan, with its withdrawals and conversions held in a narrow band and the binaries free. The full MILP is never solved: if every attempt fails, the loop's plan is kept.

2. *Moves.* (a) z^{σ} pinned, the other families freed one at a time; (b) the other families pinned, z^{σ} free within a local-branching ball of at most k flips from the current plan; (a′) when neither moved, the other families freed jointly; (c) every family's binaries freed in the years where the plan's own income implies a different cost than the one charged (the fixed-point residual). The senior-bonus binaries z^{sb}, two or three in all, are never pinned: every restricted problem keeps them free.
3. Repeat until no move improves the objective.

Every restricted problem contains the current plan, so each is feasible and none returns a worse plan: the search descends monotonically. Before searching, the plan is solved once in loop mode; that plan seeds the search and is also its floor, so the result is never worse than the loop's. Each restricted solve is capped by branch-and-bound nodes (`localSearchStepNodes`; 3,000 with HiGHS, 20,000 with MOSEK, which count nodes differently) rather than by time, so that the same case gives the same answer on any machine; `localSearchStepTime` (60 s) is only a backstop. The search has a budget (`localSearchTime`, 300 s), and k is `localSearchRadius` (4). The preset `breakpointMethod="local-search"` replaces a fixed Social Security taxable fraction with the IRS formula: local search keeps the problem well behaved without it. Local search carries no certificate and can stop short of the optimum, chiefly on the unrestricted problem, which has the most freedom. Claiming-age selectors (`withSSAges="optimize"`) and withdrawal-ordering gates are not searched; such cases are solved by branch-and-bound.

The preset `breakpointMethod="local-search"` (or `"branch-and-bound"`) sets every applicable family to `"optimize"` and selects the strategy in one step.

In tests of a prototype with HiGHS, on the shipped examples (with a benchmark ACA premium added where someone is under 65), local search was never worse than the loop and improved 17 of 23 cases. Over 72 historical start years of a bequest-maximizing case swept over first-year Roth conversions (1,261 cells), the cells whose regret came out negative—an artifact of the solver, since a restricted plan cannot beat an unrestricted one—fell from 124 with the loop to 47 with local search, at a median of 28 s per solve.

4. Formulation with imposed asset allocation ratios

The indices, variables, and parameters this chapter uses are defined in Appendix A, which is the notation key for everything that follows; the reader meeting a symbol for the first time will find it there.

We first present the case where the sums of assets in each savings account b_{ijn} are known and for which we assume prescribed asset allocation ratios. The amount in each asset class k for b_{ijkn} is simply obtained from $\alpha_{ijkn} b_{ijn}$ in this case. This formulation assumes that the accounts are always balanced. This is a reasonable assumption given the auto-balancing feature offered by many financial service providers and robo-advisers.

The benefit of this approach is that it has fewer variables and that only the sums of all asset classes in each savings account need to be considered. The rate of return of the account is then simply the product of the account balance with the sum of the rates of return weighted according to the desired allocation ratio. This approach allows us to eliminate k by summing over it and rewrite equations such as

$$\sum_k b_{ijk(n+1)} \;=\; \sum_k b_{ijkn}(1+\tau_{kn}) + \ldots, \tag{4.1}$$

for the annual evolution of account balances from year n to year $n+1$ as the simpler expression

$$\begin{aligned} b_{ij(n+1)} &= b_{ijn} \sum_k \alpha_{ijkn}(1+\tau_{kn}) + \ldots, \\ &= b_{ijn}(1+\mathcal{T}_{ijn}) + \ldots, \end{aligned} \tag{4.2}$$

where

$$\mathcal{T}_{ijn} \;:=\; \sum_k \alpha_{ijkn}\tau_{kn}. \tag{4.3}$$

It follows that the allocation ratios are normalized to unity, i.e.,

$$\sum_k \alpha_{ijkn} = 1. \tag{4.4}$$

In this formulation where the α_{ijkn} are prescribed, we will use $\mathcal{T}_{ijn}$ to add the market returns to the savings balances.

4.1 Constraints

4.1.1 Required minimum distributions (RMDs)

Withdrawals from the ($j = 1$) tax-deferred savings accounts must be greater than or equal to the required minimum distributions, and therefore,

$$w_{i1n} + \Xi_{in} - \rho_{in} b_{i1n} \geq 0. \tag{4.5}$$

The qualified charitable distribution Ξ_{in} appears here because a QCD satisfies the RMD dollar for dollar, so it lowers the floor on the taxable withdrawal the plan is forced to take (Section 4.1.2). As b_{ijn} are the balances at the beginning of year n, they are also the balances at December 31 of the previous year, which is the amount on which the IRS bases the RMDs. Eq. (4.5) has to hold for each year n and each individual i, and therefore, there are $N_i \times N_n$ such equations (although trivial when $\rho_{in} = 0$). These constraints avoid paying up to 25% penalty on amounts not withdrawn when RMDs are required. Note that aggregate rules need to be considered separately as this approach only considers the sum of assets in a class with similar tax treatment (e.g., IRA and 401k).

4.1.2 Qualified charitable distributions

A qualified charitable distribution (QCD) is a transfer paid directly from a tax-deferred account to a qualified charity, authorized by IRC §408(d)(8). It is a prescribed parameter $\Xi_{in} \geq 0$ rather than a decision variable: how much to give is a preference of the household, not a quantity to optimize. It is worth modeling because it is not equivalent to withdrawing the money and then donating it, and it enters the formulation in four places.

First, it is excluded from adjusted gross income. In this formulation that exclusion requires no term at all: Ξ_{in} simply never appears in the taxable ordinary income of Eq. (A.23), and it therefore propagates automatically to every quantity derived from

it — the state taxable income, the provisional income governing the taxability of Social Security, and the modified adjusted gross income driving IRMAA and the net investment income tax. This is a stronger benefit than a charitable deduction, which is unavailable to the majority of retirees who claim the standard exemption $\bar{\sigma}_n$.

Second, it counts toward the required minimum distribution, which is why Ξ_{in} appears in Eq. (4.5). A QCD large enough to cover the RMD releases the plan from taking any taxable distribution at all that year.

Third, the money leaves the tax-deferred account, so Ξ_{in} appears in the balance carryover Eq. (4.28) and in the withdrawal limit of Section 4.1.6. A consequence worth noting is that a QCD shrinks the base on which later RMDs are computed.

Fourth, it never becomes spendable cash, and so is absent from the net spending of Section 4.1.15. This is what distinguishes Ξ_{in} from a negative big-ticket item Λ^{-}_{in}, which is an after-tax outflow from the household budget.

The freed capacity does not go to waste. Because the QCD removes ordinary income that the RMD would otherwise have forced, it opens room under the tax brackets, and an optimizing plan fills that room with a Roth conversion of the same size: taxable income is left essentially unchanged and the benefit accrues to the estate.

Two eligibility rules are checked on the input before the plan is built, rather than in the model, and a violation is reported as an error rather than quietly adjusted. The owner must have reached age $70\frac{1}{2}$ on or before the distribution, which is not the age at which required distributions begin; and the exclusion is capped per person per year, at \$111,000 for 2026, indexed for inflation. Because the IRS sets that figure only a year at a time, later years carry the last published figure forward at 3% a year. That rate is deliberately a constant rather than the scenario's own inflation: the cap validates user input, and a scenario-dependent bound would accept an entry on one Monte Carlo path and reject it on the next.

Two simplifications are worth stating. A QCD must be paid from an IRA and not directly from a 401k, while this formulation aggregates both into a single tax-deferred account $j = 1$ and so cannot distinguish them. This is less restrictive than it appears: rolling the employer plan over to an IRA makes the money eligible, and such a rollover is a non-event here, since both balances already occupy the same account. Ensuring it has been done is left to the user. The anti-abuse provision, under which deductible IRA contributions made after age $70\frac{1}{2}$ cumulatively reduce the excludable amount, is not modeled.

4.1.3 Income tax brackets

Taxable ordinary income is divided into tax brackets as defined in Eq. (A.1), and therefore

$$0 \leq f_{tn} \leq \bar{\Delta}_{tn}, \tag{4.6}$$

where $\bar{\Delta}_{tn}$ is the inflation-adjusted income tax bracket width.

4.1.4 Standard exemption

The standard exemption is constrained by

$$0 \leq e_n \leq \bar{\sigma}_n. \tag{4.7}$$

Variable e_n is required for accounting for years when the taxable ordinary income is smaller than the standard exemption.

4.1.5 The OBBBA senior bonus and its phase-out

OBBBA §1002 adds $6,000 to the deduction of each individual aged 65 or older, not indexed, shrinking by $6 per $100 of MAGI above a filing-status threshold and so vanishing $100,000 above it (Section 2.13). The phase-out makes the deduction a function of the solution, which the self-consistent loop can only handle by freezing it at the previous iterate's MAGI. A solve under that constant cannot trade income against the bonus, and will accept income the bonus would have sheltered.

With `withSeniorBonus="optimize"` the phase-out is solved with the plan. Let k_n be the number of individuals receiving the bonus in year n, $\mathcal{T}_n^{\mathrm{sb}}$ the threshold of that year's filing status, Σ = $6,000 the bonus, $r = 0.06$ the phase-out rate, and $\bar{\sigma}_n^0$ the standard exemption *without* the bonus. A continuous variable p_n carries the per-person reduction and a binary z_n^{sb} marks the year as fully phased out:

$$\begin{aligned} e_n + k_n\, p_n &\leq \bar{\sigma}_n^0 + k_n \Sigma, && (4.8)\\ p_n &\geq r\,(\mathbb{G}_n - \mathcal{T}_n^{\mathrm{sb}}) - \mathcal{M}_n^{\mathrm{sb}}\, z_n^{\mathrm{sb}}, && (4.9)\\ p_n &\geq \Sigma\, z_n^{\mathrm{sb}}, && (4.10) \end{aligned}$$

with $0 \leq p_n \leq \Sigma$ and $\mathcal{M}_n^{\mathrm{sb}} = r \max(0, \bar{\mathcal{C}}_n - \mathcal{T}_n^{\mathrm{sb}}) + \Sigma$. The reduction is a cost, so p_n settles on its largest lower bound and the binary selects which bound that is: below the end of the phase-out, Eq. (4.9) with $z_n^{\mathrm{sb}} = 0$ is the cheaper branch; above it that row would demand more than Σ, which the column bound forbids, so $z_n^{\mathrm{sb}} = 1$ is the

only feasible choice and Eq. (4.10) pins $p_n = \Sigma$. A small objective weight on p_n settles ties in years whose cash is worth nothing to the objective. Because the deduction is now $\bar{\sigma}_n^0 + k_n(\Sigma - p_n)$ rather than a constant, the term $k_n p_n$ joins e_n wherever the deduction appears, including the capital-gains bracket rows of Eqs. (4.37)–(4.38). The MAGI here is the AGI basis of Eq. (4.47), shared with the NIIT.

4.1.6 Withdrawal limits

We introduce another set of constraints that might look unnecessary, but can help convergence, and prevent overdrafts during the year of passing of spouse i_d. As withdrawals and conversions are at the beginning of the year we impose that

$$w_{ijn} + \delta_{j,1}\,(x_{in} + \Xi_{in}) \leq b_{ijn}, \tag{4.11}$$

which just states that account balances need to be at least as large as withdrawals, possible Roth conversions, and any qualified charitable distribution, all of which are drawn from the same account.

4.1.7 Withdrawal ordering

By default, the optimizer is free to choose the order in which accounts are drawn down. When `withdrawalOrder="taxable_first"` is selected, the conventional ordering — taxable first, then tax-deferred, then Roth — is enforced instead, modeling the naive strategy a hand-managed plan would follow; it serves as a baseline policy when quantifying the value added by the optimization. Two household-level binary gates per year, z^o_{0n} and z^o_{1n}, indicate that the taxable and the tax-deferred balances, respectively, are exhausted at the end of year n. Withdrawals beyond what the gates allow are blocked through

$$\begin{aligned} w_{i1n} - \rho_{in}\, b_{i1n} &\leq \mathcal{M}^d_n\, z^o_{0n}, \\ w_{i2n} &\leq \mathcal{M}^r_n\, z^o_{1n}, \end{aligned} \tag{4.12}$$

so that, while the taxable gate is closed ($z^o_{0n} = 0$), the tax-deferred withdrawal is pinned to exactly the required minimum distribution by Eq. (4.5), and no Roth withdrawal can occur while the tax-deferred gate is closed. A gate may open only when the corresponding household balance is exhausted at the end of the year,

$$\begin{aligned} \sum_i b_{i0(n+1)} &\leq \mathcal{M}^{\mathrm{tx}}_{n+1}\,(1 - z^o_{0n}), \\ \sum_i b_{i1(n+1)} &\leq \mathcal{M}^{\mathrm{td}}_{n+1}\,(1 - z^o_{1n}), \end{aligned} \tag{4.13}$$

and the full ordering follows from the monotonicity condition

$$z^o_{1n} \leq z^o_{0n}. \tag{4.14}$$

Each big-$\mathcal{M}$ bounds only what its row gates: $\mathcal{M}^{\text{tx}}_n$ and $\mathcal{M}^{\text{td}}_n$ bound the household taxable and tax-deferred balances at the start of year n, and $\mathcal{M}^d_n$ and $\mathcal{M}^r_n$ bound a year-n tax-deferred and Roth withdrawal. They follow the one-way paths money can take between accounts (tax-deferred to Roth by conversion, any account to taxable as a surplus), compounding the starting balances, the contributions, and every fixed income that could reach taxable at the best return of the accounts on each path, and are then doubled so that they never bind. A bound built on the whole portfolio at the best return of any account runs to hundreds of times the real portfolio over a long plan, which HiGHS does not resolve reliably next to dollar coefficients (issue #178). HSA withdrawals (qualified medical expenses) are not gated. Because a surplus deposit is still in the taxable balance at the end of its year (it lands at the start of the year and grows with it, Eq. (4.28)), a year cannot both deposit a surplus and claim the taxable-exhausted gate, which closes the loophole of draining an account and immediately redepositing.

4.1.8 Posthumous account activities

For cases with spouses, no withdrawal, Roth conversion, or deposit should occur in the accounts of the passed individual i_d:

$$\begin{aligned} w_{i_d jn} &= 0, \\ d_{i_d n} &= 0, \\ x_{i_d n} &= 0, \\ &\qquad \forall j \in \{0, \ldots, N_j - 1\} \\ &\qquad \forall n \in \{n_d, \ldots, N_n - 1\}. \end{aligned} \tag{4.15}$$

4.1.9 Roth conversions

Roth conversions x_{in} are forced to zero in the last two years of each individual's horizon (see Section 2.7). Otherwise, Roth conversions cannot be larger than the balance at the beginning of the year in the account:

$$x_{in} \leq b_{i1n}. \tag{4.16}$$

This constraint, however, is naturally satisfied when $b_{ijn} \geq 0$ non-negativity bounds are enforced. Additional maximum Roth conversion constraints x_{max} can be imposed by the user. For a single individual, the previous equation becomes

$$x_{in} \leq \min(b_{i1n}, x_{max}). \tag{4.17}$$

For couples, the cap applies to each individual's conversions:

$$x_{in} \leq x_{max}, \quad i \in \{0, 1\}. \tag{4.18}$$

As these equations involve a variable and a parameter in the min function, they are coded as separate constraints.

4.1.10 Minimum taxable balance

To maintain liquidity throughout the plan, the user may impose a lower bound $\beta_i^{\min}$ (in today's dollars) on the taxable account balance of each individual. The inflation-adjusted floor is enforced from year 1 through the end of the individual's life horizon:

$$b_{i0n} \geq \beta_i^{\min} \gamma_n, \qquad n \in \{1, \ldots, \text{horizon}_i - 1\}. \tag{4.19}$$

This is controlled by the `minTaxableBalance` option. The constraint is omitted when `minTaxableBalance` is not specified.

4.1.11 Initial balances

The initial balances β_{ij} are one of the main inputs of the model. The initial savings account balances are imposed through the constraints

$$b_{ij0} = \beta_{ij}. \tag{4.20}$$

At this point, we assume that all accounts are balanced according to the desired allocation ratios α_{ijk0}.

4.1.12 Roth 5-year maturation and conversion ladder

Two IRS rules govern penalty-free access to Roth funds. (1) Roth *contributions* can always be withdrawn tax- and penalty-free. (2) Roth *conversions* can be withdrawn penalty-free only after a 5-year holding period, with each conversion carrying its own clock; *earnings* additionally require age $59\frac{1}{2}$. The two constraints below together

allow only penalty-free Roth withdrawals, so no 10% early-withdrawal penalty is assessed on w_{i2n}.

The maturation constraint (row group I_8 in the matrix appendix, tag `roth_maturation`) enforces this rule:

$$b_{i2n} - w_{i2n} \;\geq\; \sum_{k=1}^{4} \Gamma_k \, x_{i,n-k} + \sum_{k=1}^{5} \max(0,\, \Gamma_k - 1)\, \kappa_{i2,n-k}, \quad \Gamma_k = \prod_{\ell=1}^{k} (1 + \mathcal{T}_{i2,n-\ell}), \tag{4.21}$$

forcing conversions still in their period and recent contribution gains to remain in the account. A conversion's period is five taxable years, the year of the conversion and the four after it (IRC § 408A(d)(3)(F)): a year-m conversion enters the balance in $m+1$, is held through $m+4$, and is free in $m+5$. Contribution gains are held five years. Γ_k follows the account's real returns, losses included, since the carryover rows grow the balance at those returns and a conversion can never be worth more than that; flooring a loss at zero growth asked the account to retain more than its recent conversions were worth, which on a Roth holding only those left no conversion feasible at all. A contribution is retained for its gains only, and after a loss it has none. Historical amounts ($n - k < 0$) use a conservative 10%/yr assumption.

Before $59\frac{1}{2}$ a second constraint (tag `roth_early_basis`) keeps earnings in the account. A Roth distribution comes from contributions first, then conversions, and earnings last (IRC § 408A(d)(4)(B)), so for each individual and each year n before the $59\frac{1}{2}$ year, the first death, and the end of the horizon,

$$\sum_{m=0}^{n} w_{i2m} \;\leq\; b_{i20} + \sum_{m=0}^{n} \kappa_{i2m} + \left(1 - \frac{\text{oppCostX}}{100}\right) \sum_{m=0}^{n-5} x_{im}. \tag{4.22}$$

Only conversions past their period count: one still within it is charged the 10% when it comes out before $59\frac{1}{2}$, and by the ordering rules it is what a withdrawal beyond the matured ones takes first. Counting it would let a year convert X and withdraw X from a Roth holding only earnings, at no charge. Early earnings are forbidden rather than taxed, conservatively, like the maturation rows. The whole starting balance b_{i20} counts as contributions and conversions (Owl has no input for its basis); the rows stop at the first death, since a surviving spouse can keep the deceased's Roth as an inherited Roth, exempt from the 10%; and Roth 401(k) money is treated as a Roth IRA's.

This constraint enables the *Roth conversion ladder*, a strategy popular in the FIRE (Financial Independence, Retire Early) community. An early retiree converts tax-deferred funds in years $n, n{+}1, \ldots$ and, 5 years later, withdraws the corresponding

Roth amounts penalty-free. The optimizer discovers this strategy automatically; the post-solve restoration of the conversion/withdrawal exclusion leaves the pre-59$\frac{1}{2}$ years untouched so as not to dismantle it (see the AMO discussion below).

4.1.13 Cash-flow surplus

When both spouses are alive, surplus s_n gets deposited in the taxable accounts according to variable η as described in Eq. (A.22),

$$d_{in} = [\delta_{i,0}(1-\eta) + \delta_{i,1}\eta]s_n. \tag{4.23}$$

Otherwise, for $n \geq n_d$, variable η gets redefined as $\eta = \delta_{1,i_s}$. With `spousal_surplus_deposit_fraction="optimize"`, the split is left to the optimizer while both spouses live: Eq. (4.23) is replaced for $n < n_d$ by the single row

$$d_{0n} + d_{1n} = s_n, \tag{4.24}$$

so that each year's surplus can go to either taxable account, for instance to the one carrying fewer unrealized gains. A tie-break (Section 5.3) deposits with the first individual where the split does not matter. With beneficiary fractions below 1 this also decides how much of the first spouse's estate reaches the other heirs: the optimizer then deposits with the survivor, and a warning recommends a fixed fraction depositing all surplus with the first spouse to pass. Surplus can be caused by large influx of money coming from big-ticket items, compulsory RMDs, or the disposition of fixed assets. The `noLateSurplus` option pins surplus to zero in the final two plan years. This is useful because during market downturns, large sums can otherwise be recycled through the taxable account in those years with little tax or growth consequence, since the long-term compounding benefit of such deposits vanishes near the end of the plan.

4.1.14 Account balances

Contributions are assumed to be made at half-year to better represent periodic contributions made throughout the year. As we already mentioned, the account balance at the end of a year is the same as the balance at the beginning of the following year. Changes include contributions κ, distributions and withdrawals w, conversions x, surplus deposits d, and growth τ on the account through the year. For each spouse i, we track each savings account j separately, and tax-deferred accounts are coupled with the corresponding tax-free account through Roth conversions.

The timing of Roth conversions, withdrawals, and deposits brings additional coupling between these variables, and is worth a detailed discussion. First, the financial aspects, and then the algorithmic ones. For the former, some financial advisors would recommend making Roth conversions at the beginning of the year, while making withdrawals at the end. Obviously, financial simulators would always yield higher numbers when using this scenario, as the money needed to pay the regular bills stayed in the bank until the end of the year. More realistically, however, it would be more accurate to assume withdrawals at mid-year, to better represent evenly distributed withdrawals. So, financially, conversions at the beginning of the year, and withdrawals at mid-year make good sense. Conversions are also typically best when timed with market downturns, which are obviously not always at the beginning of the year.

Now, let's look at the optimization side of these transactions. During years of positive returns, a direct withdrawal from the tax-deferred account at mid-year will always be unfavorable when compared to a Roth conversion at the beginning of the year, followed by a tax-free withdrawal later in the same year. This is because the second scenario involves gains which are tax-free over the half-year, while the first one does not. Moving account withdrawals at the beginning of the year, and the conversions in mid-year can only solve part of this artificial bias under specific conditions.

Mutually exclusive transactions. Moving all transactions at the beginning of the year can also cause undesired transactions. To solve these spurious scenarios, it would be desirable to make the following at-most-one (AMO) exclusions between surplus s_n, withdrawals w_{ijn}, and conversions x_{in}:

$$\begin{aligned}\left(\sum_i w_{i0n} + \sum_i w_{i2n}\right) \quad &\texttt{AMO} \quad s_n, \\ \sum_i x_{in} \quad &\texttt{AMO} \quad \sum_i w_{i2n},\end{aligned}$$

for $j \neq 1$, i.e., for all withdrawals except those from tax-deferred accounts. The AMO relation can also be represented by a logical NAND operator. The first exclusion prevents surplus deposits when withdrawals from taxable or tax-free accounts occur from either spouse, while the second exclusion prevents simultaneous Roth conversions and tax-free withdrawals from either spouse. The natural way to impose these exclusions is with binary variables and big-$\mathcal{M}$ constraints, one pair per exclusion per year. Owl did exactly that, and no longer does. The reason is worth setting out, because it says something about the objective rather than about the constraints.

Why the binaries were removed Measured across every shipped example case under both solvers, the constrained optimum equals that of the relaxation: the exclusions never change what the plan is worth. They are symmetry breakers, selecting the legible member of a family of equally valued solutions. They are not free, however. Owl's objective distinguishes the three account types *only* through their tax treatment, so a household with a zero marginal rate on both ordinary income and capital gains gains nothing from where a dollar sits. The objective then goes flat across a combinatorial set of plans, and branch-and-bound loses the ability to prune: it holds the optimal value at the root node and spends the rest of its time proving no tie beats it. The effect is severe. A single retiree with about $119,000 of savings, no pension, and Social Security of $200 per month solves in 0.05 seconds without the binaries and took 577 seconds with them.

Two details of that pathology are worth recording. First, a loose $\mathcal{M}$ is not the cause: tightening it by two orders of magnitude changes the solve time by a quarter, because the bound was already exact and the failure is discrimination rather than tightness. Second, zero federal tax is necessary but not sufficient. The same household in a state with no income tax solves in 0.06 seconds; it is the state deduction and exemption variables, free to move when there is no tax to pay, that widen the flat face far enough to matter. These exclusions were also the only binary family a default plan carried, so removing them turns the default problem into a pure linear program.

Restoring the exclusions after the solve Both properties are re-established on the solved vector instead. For the Roth exclusion, a year $n \geq n_{i,59\frac{1}{2}}$ carrying both a conversion and a tax-free withdrawal is rewritten with $m = \min(x_{in},\, w_{i2n})$ as

$$x_{in} \;\to\; x_{in} - m, \qquad w_{i2n} \to\; w_{i2n} - m, \qquad w_{i1n} \;\to\; w_{i1n} + m. \tag{4.25}$$

Every affected row is invariant rather than merely slack. The two balance carryovers cancel, the second of them provided $\bar{x} = 1$; taxable income is unchanged because a conversion and a tax-deferred withdrawal are both fully taxable; net cash is unchanged once the ten-percent early-withdrawal penalty is absent, which is what confines the substitution to $n \geq n_{i,59\frac{1}{2}}$; and the tax-deferred withdrawal limit $b_{i1n} - w_{i1n} - x_{in} \geq 0$ is invariant term by term. The RMD floor, the five-year Roth retainer and the remaining withdrawal limits only loosen. The substitution is therefore exact and needs no second solve.

Confining it to $n \geq n_{i,59\frac{1}{2}}$ is not merely a technical requirement: before an individual reaches $59\frac{1}{2}$ it is legitimate to withdraw mature five-year-old conversions

while initiating new ladder rungs, so removing that overlap would dismantle the Roth conversion ladder. The pre-$59\frac{1}{2}$ years are left exactly as solved.

For the surplus exclusion, note first that the surplus is deposited into the same taxable account a withdrawal may have just drawn from, so the gross figures describe a movement that does not occur. Reporting the two net of one another is an identity for the cash flow balance, since the surplus appears on its left-hand side and the deposit within the withdrawals on its right, and cancelling them removes the same amount from both. Balances, taxes, spending and the bequest are untouched. Only a sale that realizes no gain is netted. A sale at a gain is a real transaction, whatever the federal bracket: the gain sits in Q_n, in MAGI and in the state base, and the rebuy resets the cost basis from which every later year's gain fraction is computed. Selling and rebuying at no gain moves neither, so there the gross and the net describe the same plan. Whatever overlap survives is re-examined by a churn-minimizing resolve of the same program with spending, the terminal balances and the conversion schedule pinned, which cannot change any reported objective and for which the incumbent is itself a feasible point.

What is not recovered Three situations admit no exact substitution and are reported rather than removed: a non-zero opportunity cost on conversions, for which $\bar{x} \neq 1$ breaks the balance invariance; the `taxable_first` withdrawal ordering, whose gate pins tax-deferred withdrawals to the RMD and so forbids the substitution's larger withdrawal; and a couple in which one spouse converts while the other withdraws, since the original exclusion was a household-level row while the substitution is per individual. In each case the overlap is left in the reported plan, which costs nothing, rather than paid for out of the objective. Two vanishingly small preferences in the objective—for leaving a dollar in an account that is never taxed again, and for not withdrawing money only to bank it—keep the solvers from selecting arbitrarily among the remaining ties.

Another approach could be to perform Roth conversions at mid-year, while withdrawals could be made at the beginning of the year, and surplus deposits, if needed due to RMDs or receiving large sums of money, could be made at the end of the year. Let's formulate this approach in more detail and investigate for potential problems. Timing controls which terms get multiplied by the rate of return $(1 + \mathcal{T}_{ijn})$. Therefore, our current choice would yield

$$\begin{aligned} b_{ij(n+1)} &= [b_{ijn} - w_{ijn} + 0.5\kappa_{ijn}](1 + \mathcal{T}_{ijn}) + [\delta_{j,2} - \delta_{j,1}]x_{in}(1 + \mathcal{T}_{ijn}/2) \\ &\quad + \delta_{j,0}d_{in} + 0.5\kappa_{ijn}, \end{aligned} \tag{4.26}$$

where we use discrete Kronecker δ functions for selecting the specific accounts in-

volved in Roth conversions. These conversions are made such that asset allocation ratios in the sending and receiving accounts are unchanged.

Bringing all variables to the left-hand side, this gets rewritten as

$$\begin{aligned} & b_{ij(n+1)} - (b_{ijn} - w_{ijn})(1 + \mathcal{T}_{ijn}) \\ - & [\delta_{j,2} - \delta_{j,1}] x_{in} (1 + \mathcal{T}_{ijn}/2) - \delta_{j,0} d_{in} \;\; = \;\; \kappa_{ijn}(1 + \mathcal{T}_{ijn}/2). \end{aligned} \tag{4.27}$$

When $j = 0$, this equation introduces a path to shelter negative returns by performing an over-withdrawal from the taxable account at the beginning of the year followed by a deposit in the same account at the end of the year. This can be removed by making these events exclusive. Owl handles the same round-trip after the solve instead, by reporting the withdrawal net of the deposit that returns to the account (Eq. (4.25) and the discussion around it), which avoids the binary variable and the branch-and-bound cost that comes with it.

A much simpler approach, while not so natural, is to move all transactions to be synchronous at the beginning or at the end of the year thus avoiding undesirable movements of funds. If we select the beginning of the year, except for contributions, this leads to

$$\begin{aligned} & b_{ij(n+1)} - [b_{ijn} + \delta_{j,0} d_{in} - w_{ijn} + (\delta_{j,2} - \delta_{j,1}) x_{in} - \delta_{j,1} \Xi_{in}](1 + \mathcal{T}_{ijn}) \\ & \qquad = \kappa_{ijn}(1 + \mathcal{T}_{ijn}/2). \end{aligned} \tag{4.28}$$

The term in Ξ_{in} removes any qualified charitable distribution from the tax-deferred account, on the same timing as a withdrawal (Section 4.1.2). Two further terms are left out of Eq. (4.28) for clarity: with an opportunity cost on conversions (option `oppCostX`), only the fraction $1 - \text{oppCostX}/100$ of x_{in} reaches the Roth account, and a SPIA premium is taken from the tax-deferred balance in its purchase year (Section 2.6). This is the current approach used in Owl. Simultaneous deposits and withdrawals, and simultaneous Roth conversions and tax-free withdrawals, are not excluded by constraints; they are restored after the solve (Section 4.1.14).

4.1.15 Net spending

For calculating the net spending g_n, we consider the cash flow of all withdrawals, wages, other ordinary income, Social Security and pension benefits, proceeds from liquidation of fixed assets, and big-ticket items. Then we subtract potential surplus s_n and all taxes (federal, state, and payroll), penalties, debts, Medicare premiums,

and ACA premiums paid:

$$\begin{aligned} g_n &= \sum_i [\omega_{in} + \upsilon_{in} + \nu_{in} + \bar{\zeta}_{in} + \bar{\pi}_{in} + \bar{a}_{in}] + \sum_{i,j} w_{ijn} + \sum_i \Lambda^{\pm}_{in} + \sum_{*=x,c,f} \mathcal{A}^*_n \\ &\quad -s_n - P_n - T_n - U_n - J_n - T^s_n - m_n - m^{\text{aca}}_n - \mathcal{D}_n - \mathcal{F}_n, \end{aligned} \tag{4.29}$$

where $T^s_n = \sum_t \theta^s_{tn} f^s_{tn} - c^s_n$ is the state income tax net of any personal credit (Section 4.1.19), and $\mathcal{F}_n$ the employee payroll tax. Here m^{aca}_n is the ACA net premium cost. It is a decision variable when `withACA="optimize"`, or a fixed SC-loop parameter when `withACA="loop"`. Notice how big-ticket items $\Lambda^{\pm}$ contribute directly to the cash flow. Qualified charitable distributions Ξ_{in} are deliberately absent: the money moves from the tax-deferred account to the charity without ever reaching the household, so it funds no spending (Section 4.1.2). Replacing intermediate variables and bringing all variables to the left-hand side, we get

$$\begin{aligned} & g_n - \sum_{i,j} w_{ijn} + 0.1 \sum_i (1 - \mathcal{H}(n - n_{i,59\frac{1}{2}})) w_{i1n} \\ & + m_n + m^{\text{aca}}_n + s_n + \sum_{t=0}^{N_t - 1} f_{tn} \theta^x_{tn} + U_n + \sum_t \theta^s_{tn} f^s_{tn} - c^s_n \;=\; \sum_{*=x,c,f} \mathcal{A}^*_n + \sum_i \Lambda^{\pm}_{in} \\ & \qquad + \sum_i [\omega_{in} + \upsilon_{in} + \nu_{in} + \bar{\zeta}_{in} + \bar{\pi}_{in} + \bar{a}_{in}] \\ & \qquad - \mathcal{D}_n - \mathcal{F}_n - J_n. \end{aligned} \tag{4.30}$$

When `withACA="loop"`, m^{aca}_n is not an LP variable; the ACA cost is computed in the SC loop and subtracted on the RHS (omitted from the equation above for clarity). Notice that we do not consider market losses or tax-loss harvesting, and we do not track individual stock purchases over the years. We also treat J_n as a parameter as it is computed through a self-consistent loop, unless `withNIIT="optimize"` makes it a variable (Section 4.1.17).

We want the net spending to be predictable and smooth. For that purpose, we use

$$g_n / \bar{\xi}_n = g_0 / \bar{\xi}_0, \tag{4.31}$$

where the net spending is adjusted for inflation and where we use the time series of parameter ξ_n, allowing for additional adjustments to the overall desired spending. Note that $\bar{\xi}_0 = \xi_0$ as $\gamma_0 = 1$. This profile is used to lower the desired net spending amount by a reduction factor χ after the passing of one spouse and/or to allow for

more realistic spending profiles, such as the *smile* profile described above. Eq. (4.31) can be rewritten as

$$g_n \xi_0 - g_0 \bar{\xi}_n = 0, \tag{4.32}$$

for the constraints to be enforced. Once g_0 is determined, the whole time series of net spending is determined. When using slack variable λ, the spending profile is then implemented as inequality constraints

$$\begin{aligned} g_n \bar{\xi}_0 - g_0(1-\lambda)\bar{\xi}_n &\geq 0, \\ -g_n \bar{\xi}_0 + g_0(1+\lambda)\bar{\xi}_n &\geq 0. \end{aligned} \tag{4.33}$$

In this case, the objective function will need to consider the sum over all years when optimizing net spending.

4.1.16 LTCG bracket constraints

The LTCG tax $U_n = 0.15\, q_{1n} + 0.20\, q_{2n}$ is determined by three LP bracket allocation variables q_{pn} ($p \in \{0, 1, 2\}$). The partition constraint

$$q_{0n} + q_{1n} + q_{2n} \geq Q_n \tag{4.34}$$

ensures the allocations cover the total LTCG Q_n. The bracket upper-bound constraints prevent overflow into higher-rate brackets. Because LTCG is stacked on top of ordinary taxable income $G_n = \sum_{t=0}^{N_t - 1} f_{tn}$, the room available in each bracket depends on how much of the threshold is already occupied by ordinary income. The constraints can be expressed as

$$q_{0n} \leq \max(0,\ \bar{\mathcal{B}}_n^{15} - G_n), \tag{4.35}$$

$$q_{0n} + q_{1n} \leq \max(0,\ \bar{\mathcal{B}}_n^{20} - G_n), \tag{4.36}$$

where $\bar{\mathcal{B}}_n^{15}$ and $\bar{\mathcal{B}}_n^{20}$ are the inflation-adjusted total taxable income thresholds above which the 15% and 20% LTCG rates apply. In these constraints, G_n is taken from the solution of the previous SC loop iteration (initialized to zero on the first pass). These constraints converge together with the SC loop. Since the cost is convex ($0\% < 15\% < 20\%$), no binary variables are needed: the LP naturally allocates gains to the cheapest bracket first.

When ordinary income does not use up the standard deduction, the unused part reduces taxable gains, which widens the bracket room by that amount. Writing

$u_n = \max(0,\, \bar{\sigma}_n - G_n - e_n)$ for the unused deduction of the previous iterate, Owl adds, in years where $u_n > 0$,

$$q_{0n} + e_n \;\le\; \max(0,\, \bar{\mathcal{B}}_n^{15} - G_n) + \bar{\sigma}_n, \tag{4.37}$$

$$q_{0n} + q_{1n} + e_n \;\le\; \max(0,\, \bar{\mathcal{B}}_n^{20} - G_n) + \bar{\sigma}_n, \tag{4.38}$$

alongside the plain bounds of Eqs. (4.35)–(4.36), whose right-hand sides are raised by u_n. Carrying e_n on the left rather than u_n on the right makes the extra room shrink by a dollar for every dollar of the deduction that ordinary income claims, so the widening cannot be double-counted. Where the senior bonus deduction is itself a decision variable, its term joins e_n in these rows.

When the option `withLTCG="optimize"` is selected, the bracket thresholds are no longer fixed from a previous iteration: G_n itself becomes a continuous LP variable, and two binary variables z_{0n}^l, $z_{1n}^l \in \{0, 1\}$ (per year n) indicate whether room exists below the 15% and 20% thresholds, respectively ($z^l = 1$ means room exists, G_n is below the threshold). The big-$\mathcal{M}$ link constraints are

$$\bar{\mathcal{B}}_n^{15} \;\le\; G_n + z_{0n}^l\, \mathcal{M}_n, \tag{4.39}$$

$$G_n + z_{0n}^l\, \mathcal{M}_n \;\le\; \bar{\mathcal{B}}_n^{15} + \mathcal{M}_n, \tag{4.40}$$

and analogously for z_{1n}^l at threshold $\bar{\mathcal{B}}_n^{20}$, with $\mathcal{M}_n = \max(\bar{\mathcal{B}}_n^{20},\, \bar{\mathcal{C}}_n - \bar{\mathcal{B}}_n^{15})$, which reaches from a threshold to anything ordinary income can be, in either direction. A monotonicity constraint

$$z_{0n}^l \le z_{1n}^l \tag{4.41}$$

ensures consistency (if G_n is below the 15% threshold it is also below the 20% threshold). The bracket room upper bounds then become linear:

$$q_{0n} \;\le\; z_{0n}^l\, \mathcal{M}'_n, \tag{4.42}$$

$$q_{0n} + q_{1n} \;\le\; z_{1n}^l\, \mathcal{M}'_n, \tag{4.43}$$

where $\mathcal{M}'_n = \max(\mathcal{M}_n,\, \bar{\mathcal{B}}_n^{20} + \bar{\sigma}_n)$ reaches past the 20% threshold by the deduction, whose unused part widens the brackets. The rooms themselves are bounded, as in Eqs. (4.37)–(4.38) but with G_n now a variable, by $q_{0n} + G_n + e_n \le \bar{\mathcal{B}}_n^{15} + \bar{\sigma}_n + \mathcal{M}_n(1 - z_{0n}^l)$ and $q_{0n} + q_{1n} + G_n + e_n \le \bar{\mathcal{B}}_n^{20} + \bar{\sigma}_n + \mathcal{M}_n(1 - z_{1n}^l)$. Because the cost is still convex, the optimizer fills the cheapest bracket first, so no additional binary variables are needed for bracket ordering; only the bracket room bounds require binaries.

4.1.17 Net investment income tax

In the default SC-loop mode, the NIIT J_n is derived from MAGI and investment income after each solver pass and held fixed for the next LP solve. When the option `withNIIT="optimize"` is selected, MAGI $\mathbb{G}_n$ becomes a continuous LP variable $\texttt{magi}_n$, a binary variable $z_n^j \in \{0,1\}$ determines whether MAGI exceeds the statutory NIIT threshold $\mathbb{G}^{\text{NIIT}}$, and J_n becomes a continuous LP variable.

One binary for the minimum

The IRS formula is $J_n = 0.038\,\max\big(0,\,\min(\mathbb{G}_n - \mathbb{G}^{\text{NIIT}},\ \mathbb{I}_n + Q_n)\big)$, where $\mathbb{I}_n + Q_n$ is net investment income (NII). High ordinary income (e.g. large RMDs or Roth conversions) can push $\mathbb{G}_n - \mathbb{G}^{\text{NIIT}}$ above NII, in which case the NIIT base is capped at NII; with large investment income, MAGI can instead exceed the threshold by less than NII, and the base is the excess. Since J_n is a cost the optimizer minimizes, it settles on the largest of its lower bounds, and the binary z_n^j selects which term of the minimum bounds it:

$$J_n \;\geq\; 0.038\,(\mathbb{G}_n - \mathbb{G}^{\text{NIIT}}) - z_n^j\,\mathcal{M}_n^j, \tag{4.44}$$

$$J_n \;\geq\; 0.038\,(\mathbb{I}_n + \mathbb{G}_n - G_n - e_n) - (1 - z_n^j)\,\mathcal{M}_n^j, \tag{4.45}$$

with $J_n \geq 0$ and $\mathcal{M}_n^j = 0.038\,\max(\bar{\mathcal{C}}_n,\,\mathbb{G}^{\text{NIIT}})$, where $\bar{\mathcal{C}}_n$ bounds the year's income. The optimizer picks the branch with the smaller bound, so $J_n = 0.038\,\max\big(0,\,\min(\mathbb{G}_n - \mathbb{G}^{\text{NIIT}},\,\text{NII}_n)\big)$ over every income range. Those two rows bound J_n only from below, and rely on its being minimized. Where a year's money is worth nothing to the objective the solver is free to overpay, so a fourth row caps the tax at 3.8% of investment income whichever branch applies, with the same terms as Eq. (4.45) and no switch:

$$J_n \;\leq\; 0.038\,(\mathbb{I}_n + \mathbb{G}_n - G_n - e_n). \tag{4.46}$$

The Q_n term of NII is written through MAGI because, on the AGI basis, $\mathbb{G}_n = G_n + e_n + Q_n$, so $\text{NII}_n = \mathbb{I}_n + \mathbb{G}_n - G_n - e_n$, with $\mathbb{I}_n$ the SC-loop parameter.

MAGI equality

An equality constraint links the NIIT MAGI (AGI basis) to ordinary income, dividends, and LTCG. Because $G_n + e_n$ already carries the taxable Social Security portion $R_n \sum_i \bar{\zeta}_{in}$, no additional Social Security term is added:

$$\mathbb{G}_n = G_n + Q_n + e_n. \tag{4.47}$$

In the LP, Q_n is represented directly by the LTCG bracket allocation variables $q_n^{(0)} + q_n^{(1)} + q_n^{(2)}$, which equal Q_n when the partition lower-bound constraint (4.34) is tight. The raw portfolio capital-gain LP expression must *not* be substituted for Q_n here: it evaluates to zero at the partition minimum and would remove Q_n from MAGI, causing the NIIT to be computed as $0.038\,\mathbb{I}_n$ rather than $0.038\,(\mathbb{I}_n + Q_n)$.

4.1.18 Taxable ordinary income

We connect the two definitions for G_n stated above in Eqs. (A.1) and (A.23),

$$\begin{aligned}\sum_{t=0}^{N_t-1} f_{tn} \;&=\; \mathcal{A}_n^x + \sum_i [\omega_{in} + \upsilon_{in} + \nu_{in} + R_n\bar{\zeta}_{in} + \bar{\pi}_{in} + \bar{a}_{in}] \\ &\quad + \sum_i [w_{i1n} + x_{in}] \\ &\quad + \sum_{i,k\neq 0} [(b_{i0n} - w_{i0n} + d_{in} + 0.5\kappa_{i0n})\alpha_{i0kn} \max(0, \tau_{kn})] - e_n, \end{aligned} \tag{4.48}$$

and rearrange to move variables to the LHS as follows

$$\begin{aligned} e_n + \sum_{t=0}^{N_t-1} f_{tn} - l_n^f - \sum_i [w_{i1n} + x_{in}] \\ - \sum_{i,k\neq 0} [(b_{i0n} - w_{i0n} + d_{in})\alpha_{i0kn} \max(0, \tau_{kn})] \\ = \mathcal{A}_n^x + \sum_i [\omega_{in} + \upsilon_{in} + \nu_{in} + R_n\bar{\zeta}_{in} + \bar{\pi}_{in} + \bar{a}_{in}] \\ + 0.5 \sum_{i,k\neq 0} \alpha_{i0kn} \max(0, \tau_{kn})\kappa_{i0n}. \end{aligned} \tag{4.49}$$

A negative *other inc* entry υ_{in} is an above-the-line deduction, and taxable income cannot go below zero, so the part of it the year's income cannot absorb is lost, not carried forward. Variable l_n^f, with $0 \le l_n^f \le \sum_i \max(0, -\upsilon_{in})$, carries that loss; it exists only in cases with a negative entry, and a tie-break (Section 5.3) keeps it no larger than the year's income makes necessary.

4.1.19 State income tax

When the configured state levies an income tax, state ordinary income is allocated across the N_s state brackets through variables f_{tn}^s, in direct analogy with Eq. (4.6),

$$0 \le f_{tn}^s \le \bar{\Delta}_{tn}^s, \tag{4.50}$$

where $\bar{\Delta}^s_{tn}$ are the inflation-adjusted state bracket widths. Variable e^s_n absorbs the state standard deduction through $0 \le e^s_n \le \bar{\sigma}^s_n$ and, when the state offers a retirement-income exemption, variable r^s_n, bounded by the inflation-adjusted per-person cap $\bar{c}^r_n$ as $0 \le r^s_n \le \bar{c}^r_n$, exempts eligible retirement income. State taxable income is tied to the federal quantities through the equality

$$\sum_{t=0}^{N_s-1} f^s_{tn} + e^s_n + \sum_i r^s_{in} - \sum_{t=0}^{N_t-1} f_{tn} - e_n - \sum_{p=0}^{2} q_{pn} + \mathbb{1}[\text{state excludes conversions}] \sum_i x_{in} - l^s_n = -E^\sigma_n - E^\pi_n, \tag{4.51}$$

where the conversion term removes Roth conversions from state income in the states that leave them out at any age (PA, MS), and l^s_n, bounded like l^f_n, is the part of a negative *other inc* the state base cannot absorb: a deduction can bring the state base below zero where the federal one is not, as when PA leaves out a conversion that federal income carries. The federal standard exemption e_n is *added back*: $\sum_t f_{tn}$ is federal taxable income, already net of the federal deduction, and a state that grants its own deduction e^s_n must not also inherit the federal one. Omitting this term subtracted the deduction twice and understated the state tax. The retirement-income exemption is per person: each r^s_{in} is capped by the state amount and by that individual's own eligible income, and an unused amount does not transfer to a spouse. Here $E^\sigma_n = R_n \sum_i \bar{\zeta}_{in}$ (the variable t^σ_n under `withSSTaxability="optimize"`) when the state does not tax Social Security — removing the federally taxable portion embedded in Eq. (4.49), with R_n the self-consistent taxability ratio — and $E^\sigma_n = 0$ otherwise, and where $E^\pi_n = \sum_i \min(\bar{\pi}_{in},\, \bar{c}^\pi_{in})$ applies the state pension exemption, each person's pension up to that person's own cap $\bar{c}^\pi_{in}$ (zero before the state's exemption age). The retirement-income exemption cannot exceed the eligible income that individual actually received,

$$r^s_{in} \le w_{i1n} + \mathbb{1}[\text{state exempts conversions}]\; x_{in} + \mathbb{1}[\text{no separate pension exemption}]\; \bar{\pi}_{in}. \tag{4.52}$$

Roth conversion income qualifies only where the state's rule admits it, and a pension shares the retirement exemption only in years whose state has no separate pension exemption of its own. A state that offers a personal or senior credit carries it as a variable c^s_n, which can only offset tax and so is capped by the year's state tax,

$$c^s_n \le \sum_t \theta^s_{tn} f^s_{tn}. \tag{4.53}$$

Finally, the state tax net of the credit, $T^s_n = \sum_t \theta^s_{tn} f^s_{tn} - c^s_n$, with θ^s_{tn} the state marginal rates, is added to the tax terms of the cash-flow constraint Eq. (4.30).

4.1.20 Social Security taxability

Under IRS Publication 915, the fraction of Social Security benefits subject to income tax is determined by the *provisional income* (PI), defined as

$$\Pi_n = \mathbb{G}_n^{\text{aca}} - \tfrac{1}{2}\sum_i \bar{\zeta}_{in}, \tag{4.54}$$

where $\mathbb{G}_n^{\text{aca}}$ is the MAGI that counts the full benefit, so that Π_n holds half of it. The IRS piecewise formula gives the taxable SS amount as

$$\mathcal{S}_n = \begin{cases} 0 & \Pi_n < \mathcal{P}^{\text{lo}}, \\ 0.5\,(\Pi_n - \mathcal{P}^{\text{lo}}) & \mathcal{P}^{\text{lo}} \le \Pi_n < \mathcal{P}^{\text{hi}}, \\ \min\Big(0.85\sum_i \bar{\zeta}_{in},\ 0.5\,(\mathcal{P}^{\text{hi}} - \mathcal{P}^{\text{lo}}) + 0.85\,(\Pi_n - \mathcal{P}^{\text{hi}})\Big) & \Pi_n \ge \mathcal{P}^{\text{hi}}, \end{cases} \tag{4.55}$$

so that $R_n = \mathcal{S}_n \,/\, \sum_i \bar{\zeta}_{in} \in [0,\, 0.85]$.

Introducing the excess provisional income above each threshold,

$$p_n^{\sigma,\text{lo}} = \max(0,\, \Pi_n - \mathcal{P}_n^{\text{lo}}), \qquad p_n^{\sigma,\text{hi}} = \max(0,\, \Pi_n - \mathcal{P}_n^{\text{hi}}), \tag{4.56}$$

the three cases of Eq. (4.55) collapse into the single expression

$$\mathcal{S}_n = \min\Big(0.85\sum_i \bar{\zeta}_{in},\ 0.50\,\min\Big(\sum_i \bar{\zeta}_{in},\, \min(\Delta\mathcal{P},\, p_n^{\sigma,\text{lo}})\Big) + 0.85\,p_n^{\sigma,\text{hi}}\Big), \tag{4.57}$$

where $\Delta\mathcal{P} = \mathcal{P}^{\text{hi}} - \mathcal{P}^{\text{lo}}$ is the bracket width (\$12,000 for MFJ; \$9,000 for single filers). The inner $\min(\sum_i \bar{\zeta}_{in},\, \cdot\,)$ guards against the unusual case $\sum_i \bar{\zeta}_{in} < \Delta\mathcal{P}$, in which SS benefits are too small to fill the 50%-bracket entirely. Because the clipped excess $\min(\Delta\mathcal{P},\, p_n^{\sigma,\text{lo}}) \le \Delta\mathcal{P}$, whenever $\sum_i \bar{\zeta}_{in} \ge \Delta\mathcal{P}$ the inner min is automatically satisfied and Eq. (4.57) simplifies to

$$\mathcal{S}_n = \min\Big(0.85\sum_i \bar{\zeta}_{in},\ 0.5\,p_n^{\sigma,\min} + 0.85\,p_n^{\sigma,\text{hi}}\Big), \qquad p_n^{\sigma,\min} = \min(\Delta\mathcal{P},\, p_n^{\sigma,\text{lo}}). \tag{4.58}$$

This condition holds for virtually all retirees, since combined annual SS benefits nearly always exceed \$12,000 (MFJ) or \$9,000 (single).

Because R_n enters the LP only as a constant multiplier of the fixed series $\bar{\zeta}_{in}$ on the right-hand side of Eq. (4.49), it is treated as a *parameter* rather than a decision variable. It is updated between LP solves in the self-consistent (SC) loop: after

each solve, $\mathbb{G}_n$ is computed from the current solution, Π_n is evaluated, and R_n is refreshed via Eq. (4.55). A 30% damping blend $R_n \leftarrow 0.3\, R_n^{\text{new}} + 0.7\, R_n$ prevents oscillation near the bracket boundaries. To avoid unnecessary LP rebuilds, R_n is only updated when $\max_n |\Delta R_n| > 10^{-3}$. The SC loop terminates when the change in the objective function (net spending or bequest) falls below a prescribed tolerance and the fixed-point residual is small enough (Section 3.6).

Alternatively, when the solver option `withSSTaxability="optimize"` is set, all max/min operations in Eqs. (4.56)–(4.58) are encoded exactly within the LP as a mixed-integer program (MIP) using the continuous variables $p_n^{\sigma,\text{lo}}$, $p_n^{\sigma,\text{hi}}$, $p_n^{\sigma,\min}$, and t_n^σ, together with two binary variables z_{0n}^σ and z_{1n}^σ per year n for which $\sum_i \bar{\zeta}_{in} > 0$; in years with no Social Security income all four continuous variables and both binaries are trivially fixed to zero and excluded from the MIP enumeration. The non-negativity of $p_n^{\sigma,\text{lo}}$ and $p_n^{\sigma,\text{hi}}$ combined with the lower-bound constraints $p_n^{\sigma,\text{lo}} \geq \Pi_n - \mathcal{P}_n^{\text{lo}}$ and $p_n^{\sigma,\text{hi}} \geq \Pi_n - \mathcal{P}_n^{\text{hi}}$ enforce the $\max(0,\cdot)$ operations; because maximizing spending minimizes taxes, the optimizer pushes these variables to their lower bounds at the optimum. The two $\min(\cdot,\cdot)$ operations in Eq. (4.58) are formulated via big-$\mathcal{M}$ lower-bound constraints. For $p_n^{\sigma,\min} = \min(\Delta\mathcal{P}_n,\, p_n^{\sigma,\text{lo}})$, the upper bounds

$$p_n^{\sigma,\min} \leq \bar{p}_n, \qquad p_n^{\sigma,\min} \leq p_n^{\sigma,\text{lo}}, \tag{4.59}$$

with $\bar{p}_n = \min(\Delta\mathcal{P}_n,\, \sum_i \bar{\zeta}_{in})$, are complemented by binary-controlled lower bounds that force the optimizer to the tighter constraint:

$$p_n^{\sigma,\min} \;\geq\; \bar{p}_n\, z_{0n}^\sigma, \qquad p_n^{\sigma,\min} - p_n^{\sigma,\text{lo}} + (\bar{\mathcal{C}}_n - \mathcal{P}_n^{\text{lo}})^+\, z_{0n}^\sigma \;\geq\; 0. \tag{4.60}$$

Capping $\bar{p}_n$ at the benefit matters when the benefit is smaller than the tier's width: a lower bound of $\Delta\mathcal{P}_n$ would then exceed $p_n^{\sigma,\min}$'s reach and make the problem infeasible. Analogously, for $t_n^\sigma = \min\bigl(0.85\sum_i \bar{\zeta}_{in},\; 0.5\, p_n^{\sigma,\min} + 0.85\, p_n^{\sigma,\text{hi}}\bigr)$:

$$t_n^\sigma \leq 0.85\sum_i \bar{\zeta}_{in}, \qquad t_n^\sigma \leq 0.5\, p_n^{\sigma,\min} + 0.85\, p_n^{\sigma,\text{hi}}, \tag{4.61}$$

$$t_n^\sigma \;\geq\; 0.85\sum_i \bar{\zeta}_{in}\, z_{1n}^\sigma, \qquad t_n^\sigma - 0.5\, p_n^{\sigma,\min} - 0.85\, p_n^{\sigma,\text{hi}} + \mathcal{M}_n^\sigma\, z_{1n}^\sigma \;\geq\; 0, \tag{4.62}$$

with $\mathcal{M}_n^\sigma = 0.5\,\bar{p}_n + 0.85\,(\bar{\mathcal{C}}_n - \mathcal{P}_n^{\text{hi}})^+$. Each row is relaxed by exactly what it gates; a flat constant was thousands of times larger than any of these. In this formulation t_n^σ is an LP variable rather than the fixed parameter $R_n \sum_i \bar{\zeta}_{in}$ used in SC-loop mode.

Moving t_n^σ to the left-hand side, Eq. (4.49) becomes

$$\begin{aligned} e_n + \sum_{t=0}^{N_t-1} f_{tn} - l_n^f - t_n^\sigma - \sum_i [w_{i1n} + x_{in}] \\ - \sum_{i,k\neq 0} [(b_{i0n} - w_{i0n} + d_{in})\alpha_{i0kn} \max(0, \tau_{kn})] \\ = \mathcal{A}_n^x + \sum_i [\omega_{in} + \upsilon_{in} + \nu_{in} + \bar{\pi}_{in} + \bar{a}_{in}] \\ + 0.5 \sum_{i,k\neq 0} \alpha_{i0kn} \max(0, \tau_{kn}) \kappa_{i0n}, \end{aligned} \tag{4.63}$$

and $R_n = t_n^\sigma / \sum_i \bar{\zeta}_{in}$ is recovered from the LP solution for reporting. There is no circular dependency: provisional income Π_n is constructed from non-SS ordinary income, Roth conversions, Q_n dividends, and the fixed half of SS ($0.5 \sum_i \bar{\zeta}_{in}$), so t_n^σ does not appear in Π_n.

4.1.21 Medicare brackets and costs

Annual Medicare costs m_n include the income-related monthly adjusted amount commonly known as IRMAA. As this additional adjustment is a piecewise constant function, it can be computed using binary variables and mixed-integer linear programming. In the current tax code, this adjustment depends on the modified adjusted gross income (MAGI) $\mathbb{G}$ from 2 years earlier. For the MAGI, we use the AGI basis $\mathbb{G}_{(n-2)} = G_{(n-2)} + Q_{(n-2)} + e_{(n-2)}$, i.e., gross taxable ordinary income plus dividends and capital gains plus the standard exemption. Only the *taxable* portion of Social Security enters IRMAA MAGI (it is already contained in $G_{(n-2)} + e_{(n-2)}$); the non-taxable portion is not added back. If anyone in the plan is on Medicare in either of the first two plan years, the MAGI values of the two years before the plan are needed to complete the calculations; they are provided by the user and default to zero.

Including regular premiums, there are $N_q = 6$ brackets of MAGI indexed for inflation defined with annual cumulative Medicare costs of $\bar{\mathcal{C}}_{qn} = \gamma_n^{\text{med}} \mathcal{C}_q$, adjusted for medical prices (Section 2.10). These segments are separated by $N_q - 1$ thresholds $\bar{\mathcal{L}}_{qn} = \gamma_n \mathcal{L}_q$, except the top one, which is held at its nominal value through 2027 and follows γ_n from 2028 on. Points $(\bar{\mathcal{L}}_{qn}, \bar{\mathcal{C}}_{qn})$ form a piecewise constant function that could be modeled using a special ordered set of type 1 (SOS1). Points $(\mathcal{L}_q, \mathcal{C}_q)$ are shown in Fig. 4.1. When $\mathbb{G}_{(n-2)} \leq \bar{\mathcal{L}}_{0n}$, the value $\bar{\mathcal{C}}_{0n}$ represents the (inflation-adjusted) base premium for Medicare Part B in year n, while higher q values include the IRMAA additional costs.

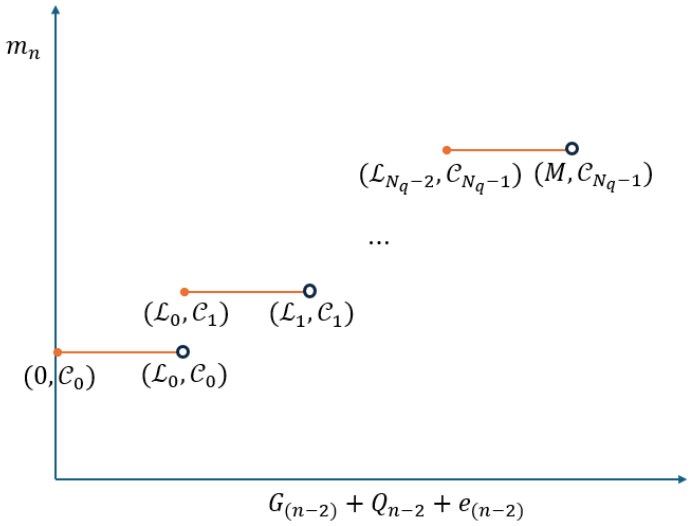


Figure 4.1: Modeling a piecewise constant monotonically increasing function. The x-axis represents the MAGI from two years ago, while m_n is the Medicare adjusted premiums. A binary variable z_q^m is associated with each segment q.

To model the IRMAA brackets we introduce binary variables $z_{qn}^m \in \{0,1\}$ (one per bracket) and continuous variables $h_{qn} \geq 0$ for the MAGI assigned to the selected bracket q. A big-$\mathcal{M}$ bound is used only for the upper bound of the top bracket (see below). We impose an SOS1 selection on binary z_{qn}^m as

$$\sum_q z_{qn}^m = 1, \tag{4.64}$$

and the disaggregation constraints

$$\begin{aligned} \sum_{q=0}^{N_q-1} h_{qn} &= \mathbb{G}_{(n-2)}, & (4.65)\\ 0 \leq h_{0n} &\leq \bar{\mathcal{L}}_{0n} z_{0n}^m, \\ \bar{\mathcal{L}}_{(q-1)n} z_{qn}^m \leq h_{qn} &\leq \bar{\mathcal{L}}_{qn} z_{qn}^m, \quad q \in \{1, \ldots, N_q - 2\}, & (4.66)\\ \bar{\mathcal{L}}_{(N_q-2)n} z_{(N_q-1)n}^m \leq h_{(N_q-1)n} &\leq \bar{\mathcal{C}}_{(n-2)} z_{(N_q-1)n}^m. \end{aligned}$$

for all $n \in \{n_m, \ldots, N_n - 1\}$, where n_m is the index year when Medicare starts for any individual i in the plan. The upper bound on the last bracket is necessary: when $z_{(N_q-1)n}^m = 0$ (the top bracket is not selected), the lower bound alone only gives

$h_{(N_q-1)n} \geq 0$, so the solver could assign positive MAGI to an inactive bracket. The constraint $h_{(N_q-1)n} \leq \bar{\mathcal{C}}_{(n-2)} z^m_{(N_q-1)n}$ forces $h_{(N_q-1)n} \leq 0$ when $z^m_{(N_q-1)n} = 0$; with $h_{qn} \geq 0$ we obtain $h_{(N_q-1)n} = 0$ for the inactive bracket. When $z^m_{(N_q-1)n} = 1$, the income ceiling $\bar{\mathcal{C}}_{(n-2)}$ of the lookback year cannot bind. For the first two years, whose MAGI $\mathbb{G}_{(-1)}$ and $\mathbb{G}_{(-2)}$ is user-provided rather than planned, the bound is raised to that MAGI when it is larger. As this applies to each individual eligible for Medicare, values $\bar{\mathcal{L}}_{qn}$ and $\bar{\mathcal{C}}_{qn}$ can include the accounting details for when both spouses are eligible and/or alive. For each year $n \geq n_m$, we can then express the Medicare costs m_n as

$$m_n = \sum_{q=0}^{N_q-1} z^m_{qn} \bar{\mathcal{C}}_{qn}, \tag{4.67}$$

as $z^m_{qn} = \delta_{q,q'}$ can only select a unique bracket q'. For $n \geq 2$, Eq. (4.65) can be expanded using Eqs. (A.23) and (A.24) as

$$\begin{aligned}
\sum_{q=0}^{N_q-1} h_{qn} = {} & \mathcal{A}^x_{(n-2)} + \mathcal{A}^c_{(n-2)} + \sum_i [\omega_{i(n-2)} + \upsilon_{i(n-2)} + \nu_{i(n-2)}] \\
& + \sum_i [R_{(n-2)} \bar{\zeta}_{i(n-2)} + \bar{\pi}_{i(n-2)} + \bar{a}_{i(n-2)}] + \sum_i [w_{i1(n-2)} + x_{i(n-2)}] \\
& + \sum_i \Big[(b_{i0(n-2)} - w_{i0(n-2)} + d_{i(n-2)}) \\
& \quad \cdot \Big(\sum_{k \neq 0} [\alpha_{i0k(n-2)} \max(0, \tau_{k(n-2)})] + \mu \alpha_{i00(n-2)} \Big) \Big] \\
& + \sum_i [w_{i0(n-2)} \alpha_{i00(n-2)} \max(0, \tau_{0\,\max(0,n-3)} - \mu)] \\
& + 0.5 \sum_i \Big[\kappa_{i0(n-2)} \Big(\sum_{k \neq 0} [\alpha_{i0k(n-2)} \max(0, \tau_{k(n-2)})] \\
& \quad + \mu \alpha_{i00(n-2)} \Big) \Big].
\end{aligned} \tag{4.68}$$

The terms on the right are income before the standard deduction, as MAGI is, so

$e_{(n-2)}$ does not appear. Collecting variables on the left-hand side yields

$$\begin{aligned}
&\sum_{q=0}^{N_q-1} h_{qn} - \sum_i [w_{i1(n-2)} + x_{i(n-2)}] \\
&\qquad - \sum_i \Big[(b_{i0(n-2)} - w_{i0(n-2)} + d_{i(n-2)}) \\
&\qquad\qquad \cdot \left(\sum_{k\neq 0} [\alpha_{i0k(n-2)} \max(0, \tau_{k(n-2)})] + \mu\alpha_{i00(n-2)} \right) \Big] \\
&\qquad - \sum_i [w_{i0(n-2)} \alpha_{i00(n-2)} \max(0, \tau_{0\,\max(0,n-3)} - \mu)] \\
&\qquad = \mathcal{A}^x_{(n-2)} + \mathcal{A}^c_{(n-2)} + \sum_i [\omega_{i(n-2)} + \upsilon_{i(n-2)} + \nu_{i(n-2)}] \\
&\qquad\quad + \sum_i [R_{(n-2)} \bar{\zeta}_{i(n-2)} + \bar{\pi}_{i(n-2)} + \bar{a}_{i(n-2)}] \\
&\qquad\quad + 0.5 \sum_i \Big[\kappa_{i0(n-2)} \Big(\sum_{k\neq 0} [\alpha_{i0k(n-2)} \max(0, \tau_{k(n-2)})] \\
&\qquad\qquad + \mu\alpha_{i00(n-2)} \Big) \Big].
\end{aligned} \tag{4.69}$$

Eq. (4.65) is expanded using Eqs. (A.23) and (A.24) to yield a linear equality in the decision variables; it is implemented directly in the constraint matrix. For indices at the beginning of the time sequence, we use $\tau_{0,\max(0,n-x)}$, when $x > n$. IRMAA MAGI is on the AGI basis, so only the *taxable* portion $R_{(n-2)}\bar{\zeta}_{i(n-2)}$ of Social Security enters; the non-taxable remainder is not added back. When Social Security taxability is itself optimized (`withSSTaxability="optimize"`), the taxable amount is the LP variable $t^{\sigma}_{(n-2)}$ rather than the parameter $R_{(n-2)}\bar{\zeta}_{i(n-2)}$.

For plans where the year index when Medicare first starts $n_m < 2$, we will request and use user-provided values for the first years' MAGIs as $\mathbb{G}_{(-1)}$ and possibly $\mathbb{G}_{(-2)}$ and set

$$\sum_{q=0}^{N_q-1} h_{qn} = \mathbb{G}_{(n-2)}, \qquad n \in \{n_m, \ldots, 1\}, \tag{4.70}$$

where $n_m \geq 0$. Using Eq. (4.66), we can also check for bounds and add constraints directly on z^m_{q0} and possibly z^m_{q1}.

4.1.22 ACA marketplace premium constraints

When `withACA="optimize"` and ACA is enabled (SLCSP > 0, $n_{\text{aca}} > 0$), ACA bracket selection and net premium cost are modeled within the optimization. ACA uses *current-year* MAGI $\mathbb{G}_n^{\text{aca}}$ (no two-year lag like Medicare). Let $N_r = 8$: seven thresholds, multiples of the poverty line of the tax household (published the year before), separate eight brackets. Bracket 0 lies below the year's first threshold, the state's Medicaid limit (138% of the poverty line in expansion states): it is covered by Medicaid at no premium, or, in a state whose Medicaid limit is below 100% of the poverty line, it is the coverage gap and pays the full benchmark premium. Brackets 1 to 6 follow the sliding scale up to 400% of the poverty line, with bracket 1 (100%–133%, the flat lowest percentage) of zero width where Medicaid reaches past 133%. Bracket 7 (400% and up) receives no credit under the 2026 rules. Each year uses its own applicable-percentage table.

SOS1 selection:

$$\sum_{r=0}^{N_r-1} z_{rn}^{\text{aca}} = 1, \qquad n \in \{0, \ldots, n_{\text{aca}} - 1\}. \tag{4.71}$$

MAGI decomposition:

$$\sum_{r=0}^{N_r-1} h_{rn}^{\text{aca}} = \mathbb{G}_n^{\text{aca}}, \tag{4.72}$$

where $\mathbb{G}_n^{\text{aca}}$ is the current-year MAGI (same structure as in Eq. (4.68) but with n instead of $n-2$, and the full Social Security benefit).

Bracket bounds (analogous to Eq. (4.66), with the income ceiling for the last bracket):

$$\begin{aligned} 0 \le h_{0n}^{\text{aca}} &\le \bar{\mathcal{L}}_{0n}^{\text{aca}} z_{0n}^{\text{aca}}, \\ \bar{\mathcal{L}}_{(r-1)n}^{\text{aca}} z_{rn}^{\text{aca}} \le h_{rn}^{\text{aca}} &\le \bar{\mathcal{L}}_{rn}^{\text{aca}} z_{rn}^{\text{aca}}, \quad r \in \{1, \ldots, N_r - 2\}, \\ \bar{\mathcal{L}}_{(N_r-2)n}^{\text{aca}} z_{(N_r-1)n}^{\text{aca}} \le h_{(N_r-1)n}^{\text{aca}} &\le \bar{\mathcal{C}}_n \, z_{(N_r-1)n}^{\text{aca}}. \end{aligned} \tag{4.73}$$

Each lower bound is raised by a small margin so that a MAGI on a threshold has one bracket. A bracket whose lower threshold lies above the MAGI at which the required contribution reaches the benchmark premium is pinned off: such incomes belong to the full-premium bracket. Where a negative "other income" (a deduction) can bring MAGI below zero, the bottom bracket reaches down by that deduction.

Cost constraint: Within brackets 1 to 6 the net premium, the applicable percentage of MAGI times MAGI, is convex in MAGI. It is represented by $K = 4$

tangent lines per bracket, at evenly spaced points, with slopes $\bar{\mathcal{C}}^{\text{aca}}_{rkn}$ and intercepts $\bar{\mathcal{I}}^{\text{aca}}_{rkn}$:

$$m^{\text{aca}}_n \geq \sum_{r=0}^{N_r-2} \left(\bar{\mathcal{C}}^{\text{aca}}_{rkn}\, h^{\text{aca}}_{rn} + \bar{\mathcal{I}}^{\text{aca}}_{rkn}\, z^{\text{aca}}_{rn}\right) + \bar{S}^{\text{aca}}_n\, z^{\text{aca}}_{(N_r-1)n}, \qquad k \in \{0,\ldots,K-1\},\ n \in \{0,\ldots,n_{\text{aca}}-1\}, \tag{4.74}$$

with $0 \leq m^{\text{aca}}_n \leq \bar{S}^{\text{aca}}_n$, where $\bar{S}^{\text{aca}}_n$ is the year's benchmark (SLCSP) premium. Only the selected bracket's terms are nonzero, so m^{aca}_n is bounded below by the largest tangent of that bracket: the sliding-scale cost from below, exact at the tangent points and short by a few dollars at most between them. Bracket 0 carries no cost under Medicaid, and the full benchmark (zero slope) in a coverage gap. m^{aca}_n carries a small objective weight (see Section 5.3) so that it does not rise above that bound where cash has no value. For $n \geq n_{\text{aca}}$, $m^{\text{aca}}_n = 0$ (Medicare-eligible; no ACA cost). The variable m^{aca}_n is added to the cash-flow constraint Eq. (4.30) alongside m_n when `withACA="optimize"` is active. A year in which one partner moves to Medicare is priced on the months-weighted benchmark of the two coverage families, keeping a single premium cap per year; this charges slightly more than the loop's pricing, and only for MAGI between the two caps. A plan starting in 2025 is priced on the 2026 rules in this formulation.

5. Objective Functions and Their Formulation

The objective function is a simple scalar defined as $c \cdot y$ that will be minimized.

5.1 Maximizing net spending

There are a few ways by which a retirement plan can be optimized. For maximizing the net spending under the constraint of a desired bequest, we introduce the following relation

$$E_n = \sum_{i,j} (1 - \nu(\delta_{j,1} + \delta_{j,3}))b_{ijn}, \tag{5.1}$$

which is the value of the estate in nominal dollars at year n, taking into consideration the heir's marginal income tax rate ν on the $(j = 1)$ tax-deferred and $(j = 3)$ HSA accounts, both of which heirs other than a spouse receive as ordinary income.

For a desired bequest ϵ_{N_n}, expressed in today's dollars, the final amount in year N_n will need to be

$$E_{N_n} = \bar{\epsilon}_{N_n} = \epsilon_{N_n}\gamma_{N_n}. \tag{5.2}$$

Requiring a bequest amounts to adding the floor

$$\sum_{i,j} b_{ijN_n}(1 - \nu(\delta_{j,1} + \delta_{j,3})) \geq \epsilon_{N_n}\gamma_{N_n} + \mathcal{D}^{\text{rem}}, \tag{5.3}$$

where $\mathcal{D}^{\text{rem}}$ is the balance of debts still outstanding at the end of the plan, which the savings must also cover; fixed assets still held at the end are added to the estate separately and do not count toward the floor. The bequest is a floor rather than

an equality: a plan may leave more when doing so costs no spending. Without a `bequest` option the floor is \$1. This adds one more row to $\ell \leq A_u y$ as

$$\begin{aligned} A_u[I(0), q_b(i,j,N_n)] &= (1 - \nu(\delta_{j,1} + \delta_{j,3})) \\ \ell[I(0)] &= \epsilon_{N_n}\gamma_{N_n} + \mathcal{D}^{\text{rem}} \qquad (5.4) \\ &\qquad \forall i \in \{0, \ldots, N_i - 1\}, \\ &\qquad \forall j \in \{0, \ldots, N_j - 1\}, \end{aligned}$$

(5.5)

where $I(0)$ is used only to provide the proper row offset C_l. See Eq. (C.1). Its row carries the tag `bequest_floor`, whose dual prices the bequest (Chapter 8).

For maximizing the net spending under the constraint of a fixed bequest, one may either maximize the first-year net spending basis g_0 (with the profile fixed by Eq. (4.32)), or maximize the sum of net spending over all years in today's dollars. In the first case, minimize the inner product $c \cdot y$ with

$$c[q_g(0)] = -1, \qquad (5.6)$$

and 0 otherwise; the whole time series g_n then follows from Eq. (4.32). In the second case, when the slack variable λ is used over net spending (as in Eq. (5.10) below), the objective is the sum $\sum_{n=0}^{N_n-1} g_n/\gamma_n$, maximized by setting

$$c[q_g(n)] = -1/\gamma_n, \qquad (5.7)$$

$\forall n \in \{0, \ldots, N_n - 1\}$, and 0 otherwise. This sum-of-net-spending formulation is the one implemented in Owl.

Variable profile with slack

When the slack variable λ is used (Eq. (4.33) or Eq. (5.10) below), the objective function is the sum of net spending over the full duration of the plan in today's dollars, $\sum_{n=0}^{N_n-1} g_n/\gamma_n$. The coefficient vector is

$$c[q_g(n)] = -1/\gamma_n, \qquad (5.8)$$

$\forall n \in \{0, \ldots, N_n - 1\}$ and 0 otherwise. In that case, constraint equality Eq. (4.32) is replaced by an inequality. Instead of obeying

$$g_n\xi_0 - g_0\bar{\xi}_n = 0, \qquad (5.9)$$

we impose the inequality constraint

$$(1-\lambda)g_0\bar{\xi}_n/\xi_0 \leq g_n \leq (1+\lambda)g_0\bar{\xi}_n/\xi_0, \tag{5.10}$$

where λ is the fractional tolerance by which annual net spending is allowed to deviate from the desired profile. It is set via the `spendingSlack` option (an integer percentage, e.g. `spendingSlack=10` gives $\lambda = 0.10$); the default is $\lambda = 0$ (strict profile). It should be noticed that when $\lambda = 0$ the two last equations are equivalent.

5.2 Maximizing bequest

If, on the other hand, one would like to maximize the bequest under the constraint of a desired net spending g_0, specified for the first year, one would add the following row to $A_e y = v$

$$\begin{aligned} A_e[I(0), q_g(0)] &= 1, \\ v[I(0)] &= g_0, \end{aligned} \tag{5.11}$$

subject to the net spending g_n obeying Eq. (4.32) over time.

The objective function would then be derived from Eq. (5.1) as minimizing the inner product $c \cdot y$, where c is

$$c[q_b(i,j,N_n)] = -(1-\nu(\delta_{j,1}+\delta_{j,3})), \tag{5.12}$$

$$\forall i \in \{0,\ldots,N_i-1\}, \quad \forall j \in \{0,\ldots,N_j-1\}, \tag{5.13}$$

and 0 otherwise.

5.3 Lexicographic tie-breaking

The objective leaves many directions flat: plans that differ only in which spouse draws a dollar, or in a round trip that costs nothing, are worth exactly the same. Small weights added to the coefficient vector c select one member of each such family without changing the primary objective in practice. Three sizes are used.

Simplex tie-breaks, ε (option `epsilon`, default 5×10^{-7}). In today's-dollar units of the objective, with $\varepsilon = 0$ turning all of them off:

- Roth conversions x_{in} are charged $\varepsilon\,(1+n)$, rising with the year, which reduces churn and front-loads the conversions that remain.
- Tax-free withdrawals w_{i2n} are charged ε, so that a household with no tax to pay does not empty the tax-free account into the taxable one for nothing.
- The surplus s_n is charged ε, against withdrawing money only to bank it again: the round trip resets cost basis, which feeds back through the self-consistent loop.
- For a couple, every withdrawal w_{1jn} of the second individual is charged ε, preferring the first individual's when otherwise equivalent; and when the surplus split is optimized, so is the second individual's deposit d_{1n} before the first death, so that the reported deposits do not move between solves.
- The state bracket amounts f^s_{tn} are charged ε, so that the full state deduction is taken even where a personal credit already cancels the year's state tax.

Larger values start to consume the economics they are meant only to break ties within: at 10^{-3} the measured Roth conversion regret falls by about a quarter.

Mixed-integer tie-breaks, 10^{-4}. Inside a branch-and-bound gap, ε is invisible, so variables that are only bounded below and would otherwise drift where they cost nothing are charged 10^{-4} per today's dollar: the ACA cost m^{aca}_n under `withACA="optimize"`, the senior-bonus reduction p_n under `withSeniorBonus="optimize"`, the lost deductions l^f_n and l^s_n, and the taxable share of Social Security t^σ_n under `withSSTaxability="optimize"`, which would otherwise be free to report half the benefit as taxable in a year whose provisional income is below $\mathcal{P}^{\text{lo}}_n$.

Tax pricing, 10^{-4}, switched on when needed. The bracket variables are tight only while the year's cash has a price. Where it has none, as in a late surplus that can only swell a bequest above its floor, any split of income across brackets is optimal, and the solver may fill the top bracket first and report tax the plan does not owe. From the first self-consistent iterate that fills a year out of order, every bracket variable (federal ordinary, capital gains, and state) is charged 10^{-4} times the tax it levies per today's dollar; an accepted plan that first goes out of order in the final iterate is re-solved the same way. It stays off otherwise, because always on it moved the fixed point the loop settles on in cases that were never out of order.

6. Rate models for return scenarios

The optimization model described in the preceding chapters requires a sequence $\{\tau_{kn}\}_{n=0}^{N_n-1}$ of annual returns for each asset class k over the plan horizon. Owl provides several models for generating this sequence, ranging from simple fixed-rate assumptions to fully stochastic simulation. This chapter describes the models available and the statistical algorithms underlying the stochastic ones.

6.1 Historical data

All built-in models draw on an annual dataset covering the period from $\mathcal{Y}_{\min} = 1928$ through the most recently completed calendar year $\mathcal{Y}_{\max}$, containing $N_y = \mathcal{Y}_{\max} - \mathcal{Y}_{\min} + 1$ annual observations [27]. Four asset-class return time series are included, corresponding to the index k:

$k = 0$ S&P 500 total return, including dividends.

$k = 1$ Moody's Baa corporate bond total return (investment-grade bonds with a moderate default premium).

$k = 2$ 10-year U.S. Treasury note total return.

$k = 3$ Consumer Price Index (CPI) inflation. This series serves a dual role: it drives the cumulative inflation factor γ_n (see Chapter 2), and, as the *cash asset*, it proxies a Treasury Inflation-Protected Security (TIPS) with a real yield of zero so that holding cash preserves but does not grow purchasing power.

All values are stored in percent (e.g. 7.0 for a 7% annual return). A user-selected subrange $[\mathcal{Y}_{\text{frm}}, \mathcal{Y}_{\text{to}}] \subseteq [\mathcal{Y}_{\min}, \mathcal{Y}_{\max}]$ is used by the `historical`, `historical_average`, `historical_gaussian`, `historical_lognormal`, `historical_bootstrap`, `historical_copula`, `vector_ar`, `garch_dcc`, `gmm`, and `hmm` models. We denote the window width

by $N_w = \mathcal{Y}_{\text{to}} - \mathcal{Y}_{\text{frm}} + 1$, and the observed return of asset class k in calendar year y by r_{ky}.

Sequence transformations: reverse and roll. Two optional transforms apply to the sequence τ'_{kn}, $n = 0, \ldots, N_n - 1$, that a model has produced for the plan, rather than to the historical window itself. The *roll* option shifts the sequence by s positions, wrapping around the plan horizon, $\tau_{kn} \leftarrow \tau'_{k,\,(n-s) \bmod N_n}$, so that a positive s delays every return by s years and brings the last s years to the start. The *reverse* option then reflects the sequence in time, $\tau_{kn} \leftarrow \tau_{k,\,N_n-1-n}$, so that the first plan year receives the return the model had placed in the last. For the `historical` method over a window at least as long as the plan, reversing alone maps calendar year $\mathcal{Y}_{\text{frm}} + N_n - 1$ to plan year 0. Both transforms preserve the set of returns in the plan while altering their order; they are useful for exploring sequence-of-returns sensitivity without changing the underlying data. They apply to every method that produces returns varying by year, including the stochastic models and `dataframe`. Constant-rate methods ignore them, with a warning.

6.2 Constant rate methods

The simplest approach assigns a constant return to every year of the plan, $\tau_{kn} = \tau_k$ for all n. Four variants are supported:

`trailing_30`
: Trailing 30-year geometric mean of annual returns, updated annually.

`conservative` **and** `optimistic`
: Preset rates based on long-term expert forecasts.

`historical_average`
: The geometric mean over the selected historical window,

$$\tau_k = \exp\left(\frac{1}{N_w} \sum_{y=\mathcal{Y}_{\text{frm}}}^{\mathcal{Y}_{\text{to}}} \ln(1 + r_{ky})\right) - 1. \tag{6.1}$$

`user`
: Freely specified by the user for each asset class.

6.3 Deterministic varying rates: `historical`

The `historical` method assigns recorded returns directly to each year of the plan:

$$\tau_{kn} = r_{k,\, y_{\text{frm}}+n}, \qquad n = 0, \ldots, N_n - 1. \tag{6.2}$$

If the plan horizon N_n exceeds the window width N_w, the sequence is repeated cyclically. The reverse and roll transforms (see § 6.1) are applied before the plan runs.

6.4 Stochastic models

Stochastic methods draw N_n annual return vectors $\boldsymbol{\tau}_n = (\tau_{0n}, \tau_{1n}, \tau_{2n}, \tau_{3n})^\top \in \mathbb{R}^{N_k}$ from a probability distribution calibrated on historical data. Because random number generation is involved, a seed can be fixed to obtain reproducible results across runs. Each fresh draw of N_n vectors constitutes one scenario; running many independent scenarios yields a Monte Carlo distribution of outcomes.

6.4.1 Inflation skewness correction

Historical US inflation rates are right-skewed: the distribution has a long right tail from high-inflation episodes (e.g. the 1970s) while low-inflation and deflation years cluster near the median. This asymmetry violates the Gaussian residual assumption shared by four of Owl's parametric stochastic models: `historical_gaussian`, `historical_lognormal`, `vector_ar`, and `garch_dcc`.

To correct for this, Owl applies a piecewise-linear (PWL) normalization transform φ to the inflation dimension ($k = 3$) before fitting each of these models, and its inverse φ^{-1} to the generated inflation samples after drawing:

$$\varphi(z;\, \kappa, s^-, s^+) = \begin{cases} (z - \kappa)\, s^- + \kappa & z \le \kappa, \\ (z - \kappa)\, s^+ + \kappa & z > \kappa, \end{cases} \tag{6.3}$$

where κ is the empirical median of the inflation values in the selected historical window and $s^- > 0$, $s^+ > 0$ are the slopes below and above the median respectively. With $s^- > 1$ and $s^+ < 1$, the transform stretches the short left tail and compresses the long right tail, producing a distribution closer to Gaussian. The inverse is

$$\varphi^{-1}(w;\, \kappa, s^-, s^+) = \begin{cases} (w - \kappa)/s^- + \kappa & w \le \kappa, \\ (w - \kappa)/s^+ + \kappa & w > \kappa. \end{cases} \tag{6.4}$$

Automatic calibration The slopes s^- and s^+ are fitted automatically from the same historical window used to calibrate the rate model, so they adapt when the user selects a different date range. They minimize the squared skewness of $\varphi(z)$ subject to a small regularization toward the identity ($s^- = s^+ = 1$) that ensures a unique, low-distortion solution:

$$(s^-,\, s^+) \;=\; \arg\min_{s^->0,\, s^+>0} \Big[\mathrm{skew}\big(\varphi(z)\big)^2 + \,0.1\big((s^- - 1)^2 + (s^+ - 1)^2\big)\Big]. \tag{6.5}$$

For US inflation over the full 1928–2025 window (sample skewness 0.31), the fit gives $\kappa \approx 2.75\%$, $s^- \approx 1.06$, and $s^+ \approx 0.94$ in return space; in log-return space, used by `historical_lognormal`, the skewness is already near zero (-0.02) and the slopes stay within half a percent of one. The slopes are searched within $[0.1, 10]$. The fitted values of κ, s^-, and s^+ are recorded in the debug log at model initialization.

Scope and domain The transform is applied to the inflation dimension only; equities, bonds, and T-notes are unchanged. User-specified models (`gaussian`, `lognormal`) and data-reuse models (`historical`, `historical_bootstrap`, `historical_average`) are unaffected. For `historical_gaussian`, `vector_ar`, and `garch_dcc`, φ is applied in *return space*: historical inflation returns $r_{3,y}$ are mapped to $\varphi(r_{3,y})$ before parameter estimation, and φ^{-1} is applied to generated inflation draws before returning them. For `historical_lognormal`, φ is applied in *log-return space*: the log-returns $\ell_{3,y} = \ln(1+r_{3,y})$ are mapped to $\varphi(\ell_{3,y})$ before Gaussian fitting, and φ^{-1} is applied to the generated log-return samples before exponentiation. Working in log-space eliminates the constraint $\varphi(r_{3,y}) > -1$ that would otherwise be required for the logarithm to remain well-defined.

6.4.2 Mean anchoring (`constrain_mean`)

An optional post-generation correction is available for `historical_gaussian`, `historical_lognormal`, `historical_copula`, `vector_ar`, `garch_dcc`, `gmm`, and `hmm`. When activated, the arithmetic mean of the generated scenario is shifted to match the historical window mean exactly.

Let $\boldsymbol{\tau}^{(1)}, \ldots, \boldsymbol{\tau}^{(N_n)} \in \mathbb{R}^{N_k}$ be the N_n annual return vectors produced by any of the stochastic models above. Define the sample mean over the scenario,

$$\hat{\boldsymbol{\mu}} = \frac{1}{N_n} \sum_{n=1}^{N_n} \boldsymbol{\tau}^{(n)}, \tag{6.6}$$

and the historical arithmetic mean over the selected window,

$$\mu_k^{\text{hist}} = \frac{1}{N_w} \sum_{y=\mathcal{Y}_{\text{frm}}}^{\mathcal{Y}_{\text{to}}} r_{ky}. \tag{6.7}$$

The correction shifts every year's return vector by the same additive offset:

$$\boldsymbol{\tau}^{(n)} \leftarrow \boldsymbol{\tau}^{(n)} + \big(\boldsymbol{\mu}^{\text{hist}} - \hat{\boldsymbol{\mu}}\big), \qquad n = 1, \ldots, N_n. \tag{6.8}$$

This preserves variances, correlations, skewness, volatility clustering, and all other distributional properties, while ensuring that the scenario's mean return matches the historical reference by construction.

The practical motivation is sampling bias in short scenarios: for small N_n, the sample mean $\hat{\boldsymbol{\mu}}$ can deviate substantially from $\boldsymbol{\mu}^{\text{hist}}$, introducing a systematic optimistic or pessimistic tilt that is purely a finite-sample artefact. Mean anchoring removes this tilt without distorting the shape of the distribution. Return floors ($\tau_{kn} \geq -1$ for $k \in \{0, 1, 2\}$; $\tau_{3n} \geq \tau_{\min}^{\text{infl}}$) are applied *after* the correction.

6.4.3 Multivariate normal: `historical_gaussian` and `gaussian`

Both methods model annual returns as draws from a multivariate normal distribution,

$$\boldsymbol{\tau}_n \overset{\text{iid}}{\sim} \mathcal{N}(\boldsymbol{\mu},\, \boldsymbol{\Sigma}), \qquad n = 0, \ldots, N_n - 1. \tag{6.9}$$

For `historical_gaussian`, the mean vector $\boldsymbol{\mu}$ and the $N_k \times N_k$ covariance matrix $\boldsymbol{\Sigma}$ are estimated from the selected historical window:

$$\mu_k = \frac{1}{N_w} \sum_{y=\mathcal{Y}_{\text{frm}}}^{\mathcal{Y}_{\text{to}}} r_{ky}, \tag{6.10}$$

$$\Sigma_{kk'} = \frac{1}{N_w - 1} \sum_{y=\mathcal{Y}_{\text{frm}}}^{\mathcal{Y}_{\text{to}}} (r_{ky} - \mu_k)(r_{k',y} - \mu_{k'}). \tag{6.11}$$

For `gaussian`, the user supplies $\boldsymbol{\mu}$ directly, together with the standard deviations $\sigma_k = \sqrt{\Sigma_{kk}}$ and the Pearson correlation coefficients $\rho_{kk'} \in [-1, 1]$, from which the covariance matrix is assembled as $\Sigma_{kk'} = \rho_{kk'}\sigma_k\sigma_{k'}$. In both cases, samples are generated as $\boldsymbol{\tau}_n = \boldsymbol{\mu} + L\boldsymbol{z}_n$, where L is the lower-triangular Cholesky factor of $\boldsymbol{\Sigma}$ and $\boldsymbol{z}_n \sim \mathcal{N}(\mathbf{0}, I)$. By construction, the multivariate normal model treats each year's returns as independent of all other years; it captures cross-sectional correlation among asset classes but no serial (year-to-year) autocorrelation.

6.4.4 Log-normal models: `lognormal` and `historical_lognormal`

Geometric Brownian Motion (GBM) [28] — the foundation of most option-pricing theory — implies that the price relative $1 + \tau_{k,n}$ is log-normally distributed. The log-normal model enforces $\tau_{k,n} > -1$ by construction (no total-loss artifacts are possible), produces a right-skewed marginal distribution (large gains are more likely than a symmetric normal would suggest), and is consistent with continuous-time finance theory.

Parameter conversion Given the arithmetic mean m_k and arithmetic standard deviation s_k of asset-class k (both in decimal), the corresponding log-space parameters are [28]

$$\sigma_{Z,k}^2 = \ln\left(1 + \left(\frac{s_k}{1+m_k}\right)^2\right), \tag{6.12}$$

$$\mu_{Z,k} = \ln(1+m_k) - \tfrac{1}{2}\sigma_{Z,k}^2. \tag{6.13}$$

One can verify that $\mathbb{E}[\exp(Z_k)] - 1 = m_k$ and $\mathrm{Var}[\exp(Z_k) - 1] = s_k^2$ when $Z_k \sim \mathcal{N}(\mu_{Z,k},\, \sigma_{Z,k}^2)$.

Sampling A log-space covariance matrix is assembled from the log-space standard deviations $\sigma_{Z,k} = \sqrt{\sigma_{Z,k}^2}$ and the Pearson correlation matrix $\mathbf{C}$ with entries $\rho_{kk'}$,

$$(\boldsymbol{\Sigma}_Z)_{kk'} = \rho_{kk'}\,\sigma_{Z,k}\,\sigma_{Z,k'}. \tag{6.14}$$

Then N_n vectors of log-returns are drawn jointly,

$$\boldsymbol{Z}_n \overset{\text{iid}}{\sim} \mathcal{N}(\boldsymbol{\mu}_Z,\, \boldsymbol{\Sigma}_Z), \qquad n = 0, \ldots, N_n - 1, \tag{6.15}$$

and annual returns are recovered by exponentiation,

$$\tau_{k,n} = \exp(Z_{k,n}) - 1. \tag{6.16}$$

Applying the correlation matrix directly in log-space is a standard approximation, exact for jointly log-normal variates and accurate for moderate return magnitudes.

`lognormal` — **user-supplied parameters** The user provides arithmetic means m_k and standard deviations s_k (in percent) together with Pearson correlations $\rho_{kk'}$. These are converted to log-space via Eqs. (6.12)–(6.13) before sampling.

`historical_lognormal` — **history-calibrated parameters** Parameters are estimated directly in log-space from the selected historical window without any intermediate conversion to arithmetic parameters. Let $\ell_{ky} = \ln(1 + r_{ky})$ be the log-return of asset class k in calendar year y. Then

$$\mu_{Z,k} = \frac{1}{N_w} \sum_{y=\mathcal{Y}_{\text{frm}}}^{\mathcal{Y}_{\text{to}}} \ell_{ky}, \tag{6.17}$$

$$(\boldsymbol{\Sigma}_Z)_{kk'} = \frac{1}{N_w - 1} \sum_{y=\mathcal{Y}_{\text{frm}}}^{\mathcal{Y}_{\text{to}}} (\ell_{ky} - \mu_{Z,k})(\ell_{k',y} - \mu_{Z,k'}). \tag{6.18}$$

Arithmetic statistics displayed in the UI are recovered from the log-space moments via the inverse of Eqs. (6.12)–(6.13):

$$m_k = \exp\big(\mu_{Z,k} + \tfrac{1}{2}(\boldsymbol{\Sigma}_Z)_{kk}\big) - 1, \tag{6.19}$$

$$s_k = (1 + m_k)\sqrt{\exp((\boldsymbol{\Sigma}_Z)_{kk}) - 1}. \tag{6.20}$$

6.4.5 Bootstrap sequence-of-returns: `historical_bootstrap`

The bootstrap sequence-of-returns model resamples N_n annual return vectors directly from the observed window $\{\boldsymbol{r}_y\}_{y=\mathcal{Y}_{\text{frm}}}^{\mathcal{Y}_{\text{to}}}$, where $\boldsymbol{r}_y = (r_{0,y}, \ldots, r_{N_k-1,y})^\top$. Resampling from actual data preserves the empirical marginal distributions exactly, including skewness, fat tails, and any departure from normality. Four resampling strategies are supported.

IID bootstrap Each year is drawn independently and with replacement from the full window [29]:

$$\boldsymbol{\tau}_n = \boldsymbol{r}_{y_n}, \qquad y_n \overset{\text{iid}}{\sim} \text{Uniform}\{\mathcal{Y}_{\text{frm}}, \ldots, \mathcal{Y}_{\text{to}}\}. \tag{6.21}$$

This is the simplest and default variant; it destroys all serial correlation present in the original data.

Block bootstrap To preserve short-run serial correlation, the block bootstrap of Künsch [30] draws $\lceil N_n/b \rceil$ starting positions s_ℓ uniformly from $\{\mathcal{Y}_{\text{frm}}, \ldots, \mathcal{Y}_{\text{to}} - b + 1\}$ and concatenates the corresponding blocks of b consecutive years, truncating to N_n years:

$$(\boldsymbol{\tau}_0, \ldots, \boldsymbol{\tau}_{N_n-1}) = (\boldsymbol{r}_{s_1}, \ldots, \boldsymbol{r}_{s_1+b-1},\ \boldsymbol{r}_{s_2}, \ldots, \boldsymbol{r}_{s_2+b-1},\ \ldots). \tag{6.22}$$

The parameter b is the block size, chosen by the user (option `block_size`, default 1).

Circular block bootstrap A variant due to Politis and Romano [31] treats the historical window as circular, wrapping from $\mathcal{Y}_{\text{to}}$ back to $\mathcal{Y}_{\text{frm}}$. Starting positions are drawn uniformly from the full window $\{\mathcal{Y}_{\text{frm}}, \ldots, \mathcal{Y}_{\text{to}}\}$, and block membership wraps around modulo N_w. This removes the boundary effect present in the block bootstrap, where observations near the endpoints of the window are underrepresented.

Stationary bootstrap In the stationary bootstrap of Politis and Romano [32], block lengths are random rather than fixed. Each block length L_ℓ is drawn from a geometric distribution,

$$P(L_\ell = \ell) = p_b\,(1 - p_b)^{\ell - 1}, \qquad \ell = 1, 2, \ldots, \tag{6.23}$$

with parameter $p_b = 1/b$, so that the expected block length equals b. This choice produces a strictly stationary bootstrap distribution, resolving the boundary discontinuities of the fixed-block method while still approximately preserving serial correlation at lags up to order b. Within a block, the window is traversed circularly, as in the circular block bootstrap. The strategy is selected by option `bootstrap_type` (`"iid"`, the default, `"block"`, `"circular"`, or `"stationary"`).

Crisis overweighting Selected years can be drawn more often than others: option `crisis_years` lists them and `crisis_weight` (default 1, no effect) multiplies their sampling probability. The weights apply to the years drawn in the IID bootstrap and to the block starting years in the other three strategies, so that a block starting in a crisis year is more likely, while the years that follow it in the block are taken as they come.

6.4.6 Vector autoregression: `vector_ar`

The VAR(1) model [33, 34] extends the multivariate normal approach by explicitly capturing year-to-year serial correlations in asset returns, such as momentum and mean-reversion. The generating model is

$$\boldsymbol{y}_t = \mathbf{c} + \mathbf{A}\boldsymbol{y}_{t-1} + \boldsymbol{\varepsilon}_t, \qquad \boldsymbol{\varepsilon}_t \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}), \tag{6.24}$$

where $\boldsymbol{y}_t \in \mathbb{R}^{N_k}$ is the vector of asset-class returns at time t, $\mathbf{c} \in \mathbb{R}^{N_k}$ is a constant intercept vector, $\mathbf{A} \in \mathbb{R}^{N_k \times N_k}$ is the transition matrix encoding one-period predictability, and $\boldsymbol{\Sigma}$ is the contemporaneous residual covariance matrix. For $\mathbf{A} = 0$, Eq. (6.24) reduces to the multivariate normal model of the `historical_gaussian` method.

Parameter estimation Given N_w historical observations $\boldsymbol{y}_1, \ldots, \boldsymbol{y}_{N_w}$, parameters $\mathbf{c}$ and $\mathbf{A}$ are estimated jointly by ordinary least squares (OLS). Stacking observations into matrices

$$Y = [\boldsymbol{y}_2 \mid \cdots \mid \boldsymbol{y}_{N_w}]^\top \in \mathbb{R}^{(N_w-1)\times N_k}, \tag{6.25}$$

$$X = [\mathbf{1} \mid \boldsymbol{y}_1 \mid \cdots \mid \boldsymbol{y}_{N_w-1}]^\top \in \mathbb{R}^{(N_w-1)\times(N_k+1)}, \tag{6.26}$$

the OLS solution $B = (X^\top X)^{-1} X^\top Y$ yields the intercept $\mathbf{c} = B[0,:]$ and transition matrix $\mathbf{A} = B[1:,:]^\top$. The residual covariance is estimated as

$$\boldsymbol{\Sigma} = \frac{(Y - XB)^\top (Y - XB)}{N_w - N_k - 2}, \tag{6.27}$$

dividing by the $N_w - 1$ regression rows less the $N_k + 1$ coefficients of each equation. At least ten historical years are required.

Stationarity and spectral shrinkage Stationarity of the VAR(1) process requires that all eigenvalues of $\mathbf{A}$ have modulus strictly less than one. Let $\rho(\mathbf{A}) = \max_i |\lambda_i(\mathbf{A})|$ denote the spectral radius. If $\rho(\mathbf{A}) \geq 0.95$ (near or above the unit circle), the transition matrix is rescaled as

$$\mathbf{A} \leftarrow \frac{0.95}{\rho(\mathbf{A})}\,\mathbf{A}, \tag{6.28}$$

ensuring $\rho(\mathbf{A}) = 0.95 < 1$ while preserving the direction of every eigenmode. This spectral shrinkage step can be disabled by the user (option `shrink`, default true, available in the case file and the API but not in the user interface), at the cost of potentially non-stationary sequences.

Sequence generation Starting from $\boldsymbol{y}_0 = \boldsymbol{\mu}$ (the historical sample mean), sequences are generated recursively from Eq. (6.24) using $\boldsymbol{\varepsilon}_t = L\boldsymbol{z}_t$, where L is the Cholesky factor of $\boldsymbol{\Sigma}$ and $\boldsymbol{z}_t \sim \mathcal{N}(\mathbf{0}, I)$. The VAR(1) model produces synthetic sequences of arbitrary length while capturing both the cross-sectional covariance structure and the serial autocorrelation of asset returns.

6.4.7 DCC-GARCH(1,1): `garch_dcc`

The DCC-GARCH(1,1) model [35, 36] extends the multivariate normal approach by allowing both the per-asset conditional variance and the cross-asset correlation matrix to vary over time. It is fitted in two steps on the selected historical window.

Step 1 — per-asset GARCH(1,1) Let $\varepsilon_{kt} = y_{k,t} - \mu_k$ be the demeaned return of asset class k at year t, with the N_w observations numbered $t = 0, \dots, N_w - 1$. The conditional variance h_{kt} evolves as

$$h_{kt} = \omega_k + \alpha_k \, \varepsilon_{k,t-1}^2 + \beta_k \, h_{k,t-1}, \tag{6.29}$$

with $h_{k,0} = \mathrm{Var}(\varepsilon_k)$, so that h_{kt} depends only on shocks before year t. The parameters $(\omega_k, \alpha_k, \beta_k)$ are estimated independently for each asset by maximizing the Gaussian log-likelihood

$$\ell_k = -\tfrac{1}{2} \sum_{t=1}^{N_w - 1} \big[\ln h_{kt} + \varepsilon_{kt}^2 / h_{kt} \big], \tag{6.30}$$

subject to $\omega_k > 0$, $\alpha_k, \beta_k > 0$, and the covariance-stationarity constraint $\alpha_k + \beta_k < 1$. Optimization uses L-BFGS-B; if convergence fails, a fallback shaped like the RiskMetrics exponentially weighted moving average [37] ($\alpha_k = 0.06$, $\beta_k = 0.94$, $\omega_k = 10^{-4} \, \mathrm{Var}(\varepsilon_k)$) is used instead, its persistence $\alpha_k + \beta_k$ scaled to 0.999 to keep it stationary. The decay factor 0.94 is the one RiskMetrics calibrated for daily returns (its monthly value is 0.97); it is applied here to annual returns without recalibration, and serves only as a stand-in when the likelihood cannot be maximized.

The standardized residuals $z_{kt} = \varepsilon_{kt} / \sqrt{h_{kt}}$, each scaled by the variance forecast made before its own shock, are passed to the second step.

Step 2 — DCC dynamics Let $\overline{\mathbf{Q}}$ be the unconditional correlation matrix of the standardized residuals $\mathbf{Z} = [z_{kt}]$, as in [35]: the second-moment matrix $\mathbf{M} = \frac{1}{N_w} \mathbf{Z}^\top \mathbf{Z}$ normalized to a unit diagonal, $\overline{\mathbf{Q}} = \mathrm{diag}(\mathbf{M})^{-1/2} \, \mathbf{M} \, \mathrm{diag}(\mathbf{M})^{-1/2}$. The quasi-correlation matrix $\mathbf{Q}_t$ evolves as

$$\mathbf{Q}_t = (1 - a - b) \, \overline{\mathbf{Q}} + a \, \boldsymbol{z}_{t-1} \boldsymbol{z}_{t-1}^\top + b \, \mathbf{Q}_{t-1}, \tag{6.31}$$

starting from $\mathbf{Q}_0 = \overline{\mathbf{Q}}$, with $a, b > 0$ and $a + b < 1$. The time-varying correlation matrix is obtained by normalizing:

$$\mathbf{R}_t = \mathrm{diag}(\mathbf{Q}_t)^{-1/2} \, \mathbf{Q}_t \, \mathrm{diag}(\mathbf{Q}_t)^{-1/2}. \tag{6.32}$$

The DCC parameters (a, b) are estimated by maximizing the concentrated log-likelihood

$$\ell_{\mathrm{DCC}} = -\tfrac{1}{2} \sum_{t=1}^{N_w - 1} \big[\ln |\mathbf{R}_t| + \boldsymbol{z}_t^\top \mathbf{R}_t^{-1} \boldsymbol{z}_t \big], \tag{6.33}$$

also with L-BFGS-B, falling back on $(a, b) = (0.05, 0.90)$ if it fails.

Sequence generation The simulation starts from the forecasts for the year after the window, $\mathbf{Q}_{N_w}$ and $\boldsymbol{h}_{N_w} = (h_{0,N_w}, \ldots, h_{N_k-1,N_w})$, obtained by a forward pass through all N_w historical residuals. Each simulated year is then drawn as

$$\boldsymbol{z}_t = L(\mathbf{R}_t)\, \boldsymbol{u}_t, \qquad \boldsymbol{y}_t = \boldsymbol{\mu} + \mathrm{diag}(\boldsymbol{h}_t)^{1/2}\, \boldsymbol{z}_t, \qquad \boldsymbol{u}_t \sim \mathcal{N}(\mathbf{0}, I), \tag{6.34}$$

where $L(\mathbf{R}_t)$ is the lower Cholesky factor of $\mathbf{R}_t$, so that $\boldsymbol{z}_t$ is the standardized residual of the simulated year. The GARCH update of Eq. (6.29) with $\varepsilon_t = \boldsymbol{y}_t - \boldsymbol{\mu}$ and the DCC update of Eq. (6.31) with $\boldsymbol{z}_t$ then give the next year's $\boldsymbol{h}$ and $\mathbf{Q}$. This produces sequences that exhibit *volatility clustering* — large shocks in one year increase conditional variance in the next — and *time-varying cross-asset correlations* that spike during simulated market-stress episodes, consistent with empirical evidence in financial return data.

6.4.8 Gaussian mixture model: `gmm`

The Gaussian mixture model (GMM) [38] generalizes the single multivariate normal of `historical_gaussian` by representing the joint return distribution as a weighted sum of K multivariate Gaussian components:

$$p(\boldsymbol{r}) = \sum_{k=1}^{K} \pi_k\, \mathcal{N}(\boldsymbol{r};\, \boldsymbol{\mu}_k,\, \boldsymbol{\Sigma}_k)\,, \qquad \pi_k \geq 0, \quad \sum_{k=1}^{K} \pi_k = 1. \tag{6.35}$$

Each component k has its own mean vector $\boldsymbol{\mu}_k \in \mathbb{R}^{N_k}$, full covariance matrix $\boldsymbol{\Sigma}_k \in \mathbb{R}^{N_k \times N_k}$, and mixture weight π_k.

Motivation Historical return data are not well-described by a single Gaussian. The empirical distribution exhibits fat tails, negative skewness for equities, and distinct clustering consistent with alternating market regimes — a prolonged bull market, a bear market, and acute crisis periods. A GMM with $K = 3$ components can capture these regimes jointly across all asset classes, so that in a "crisis" component, both equities and investment-grade bonds may fall together (correlation breakdown), while in a "bull" component the usual negative stock–bond correlation is restored. Each component's $\boldsymbol{\Sigma}_k$ encodes the cross-asset covariance *conditional on being in that regime.*

Fitting The parameters $\{\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\}_{k=1}^{K}$ are estimated from the N_w annual return vectors in the selected historical window $[\mathcal{Y}_{\mathrm{frm}}, \mathcal{Y}_{\mathrm{to}}]$ by Expectation-Maximization (EM) [39]. The EM algorithm alternates between:

- **E-step**: compute the posterior responsibility $\gamma_{nk} = \pi_k \mathcal{N}(\boldsymbol{r}_n; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \;/\; \sum_j \pi_j \mathcal{N}(\boldsymbol{r}_n; \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)$ for each observation n and component k;
- **M-step**: update $\pi_k = \frac{1}{N_w} \sum_n \gamma_{nk}$, $\boldsymbol{\mu}_k = \frac{\sum_n \gamma_{nk} \boldsymbol{r}_n}{\sum_n \gamma_{nk}}$, $\boldsymbol{\Sigma}_k = \frac{\sum_n \gamma_{nk} (\boldsymbol{r}_n - \boldsymbol{\mu}_k)(\boldsymbol{r}_n - \boldsymbol{\mu}_k)^\top}{\sum_n \gamma_{nk}}$.

A diagonal regularization of 10^{-6} is added to every $\boldsymbol{\Sigma}_k$. The means start at K distinct observations drawn at random (hence the seed also fixes the fit), the weights start uniform, and the covariances start at the pooled covariance. Convergence is declared when the log-likelihood gains less than 10^{-6}, within at most 200 iterations.

Sequence generation To draw one simulated year of returns:

1. Sample a regime index $k^* \sim \text{Categorical}(\boldsymbol{\pi})$.
2. Draw $\boldsymbol{\tau}_n \sim \mathcal{N}(\boldsymbol{\mu}_{k^*}, \boldsymbol{\Sigma}_{k^*})$.

Each of the N_n plan years is drawn independently, so the model captures cross-sectional correlation within each regime but no serial autocorrelation across years. The number of components K is a user parameter (option `n_components`, default $K = 3$, at least 2 and no more than the number of years in the window); every component has a full covariance matrix.

6.4.9 Hidden Markov model: `hmm`

The hidden Markov model (HMM) [40] extends the GMM by adding temporal structure to the regime sequence. An unobserved discrete state $s_n \in \{1, \ldots, K\}$ evolves as a first-order Markov chain with transition matrix $\boldsymbol{A} \in \mathbb{R}^{K \times K}$, $A_{kj} = P(s_{n+1} = j \mid s_n = k)$, with $\sum_j A_{kj} = 1$, and each year's returns are drawn from the multivariate Gaussian of the current state:

$$s_{n+1} \mid s_n = k \;\sim\; \text{Categorical}(\boldsymbol{A}_{k\cdot}), \qquad \boldsymbol{\tau}_n \mid s_n = k \;\sim\; \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k). \tag{6.36}$$

The emission parameters $\{\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\}$ have the same interpretation as in the GMM; the key new quantity is $\boldsymbol{A}$.

Motivation The GMM treats each simulated year as an independent regime draw. Historical return data exhibit regime *persistence*: bull and bear markets tend to continue for several years before switching. The HMM encodes this by coupling consecutive years through $\boldsymbol{A}$, so a diagonal entry A_{kk} close to 1 produces multi-year runs of the same regime. This is directly relevant to sequence-of-returns risk: a

retiree entering a bear-market regime faces an elevated probability of further down years before the regime changes, amplifying the compounding impact of early losses in ways the i.i.d. GMM cannot reproduce.

Fitting Given N_w historical return vectors $\boldsymbol{r}_0, \ldots, \boldsymbol{r}_{N_w-1}$, the parameters $\{\boldsymbol{\pi}, \boldsymbol{A}, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\}$ are estimated by the Baum-Welch algorithm [42], which applies EM [39] to the HMM likelihood. Each iteration runs a scaled forward-backward pass and a parameter update.

Forward pass:

$$\hat{\alpha}_0(k) = \pi_k \, b_k(\boldsymbol{r}_0), \qquad c_0 = \textstyle\sum_k \hat{\alpha}_0(k), \qquad \hat{\alpha}_0 \leftarrow \hat{\alpha}_0 / c_0, \tag{6.37}$$

$$\hat{\alpha}_n(j) = b_j(\boldsymbol{r}_n) \textstyle\sum_k \hat{\alpha}_{n-1}(k) \, A_{kj}, \qquad c_n = \textstyle\sum_j \hat{\alpha}_n(j), \qquad \hat{\alpha}_n \leftarrow \hat{\alpha}_n / c_n, \tag{6.38}$$

where $b_k(\boldsymbol{r}) = \mathcal{N}(\boldsymbol{r}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$. The log-likelihood is $\ell = \sum_n \log c_n$.

Backward pass:

$$\hat{\beta}_{N_w-1}(k) = 1, \qquad \hat{\beta}_n(k) = \frac{1}{c_{n+1}} \sum_j A_{kj} \, b_j(\boldsymbol{r}_{n+1}) \, \hat{\beta}_{n+1}(j). \tag{6.39}$$

Posterior quantities:

$$\gamma_n(k) = \hat{\alpha}_n(k) \, \hat{\beta}_n(k) \Big/ \textstyle\sum_j \hat{\alpha}_n(j) \hat{\beta}_n(j), \qquad \xi_n(k,j) \;\propto\; \hat{\alpha}_n(k) \, A_{kj} \, b_j(\boldsymbol{r}_{n+1}) \, \hat{\beta}_{n+1}(j). \tag{6.40}$$

M-step:

$$\pi_k = \gamma_0(k), \tag{6.41}$$

$$A_{kj} = \frac{\displaystyle\sum_{n=0}^{N_w-2} \xi_n(k,j) + \delta}{\displaystyle\sum_{n=0}^{N_w-2} \gamma_n(k) + K\delta}, \tag{6.42}$$

$$\boldsymbol{\mu}_k = \frac{\sum_n \gamma_n(k) \, \boldsymbol{r}_n}{\sum_n \gamma_n(k)}, \quad \boldsymbol{\Sigma}_k = \frac{\sum_n \gamma_n(k) \, (\boldsymbol{r}_n - \boldsymbol{\mu}_k)(\boldsymbol{r}_n - \boldsymbol{\mu}_k)^\top}{\sum_n \gamma_n(k)} + \varepsilon \, \boldsymbol{I}, \tag{6.43}$$

where $\delta > 0$ (default 10^{-3}) is a Laplace smoothing term that prevents zero-probability transitions for regimes not visited in the historical window, and $\varepsilon = 10^{-6}$ regularizes the covariances. Convergence is declared when the improvement in ℓ falls below a tolerance; the transition matrix is initialized to an 80% self-transition prior, $A_{kk} = 0.8$, to encourage persistence from the first iteration.

Sequence generation Let $\boldsymbol{\pi}_\infty$ denote the stationary distribution satisfying $\boldsymbol{\pi}_\infty = \boldsymbol{\pi}_\infty \boldsymbol{A}$, computed by power iteration. A plan of N_n years is simulated as:

1. Draw the initial state $s_0 \sim \boldsymbol{\pi}_\infty$ (or from a user-specified `init_regime`).
2. For $n = 0, 1, \ldots, N_n - 1$: draw $\boldsymbol{\tau}_n \sim \mathcal{N}(\boldsymbol{\mu}_{s_n}, \boldsymbol{\Sigma}_{s_n})$, then transition $s_{n+1} \sim \text{Categorical}(\boldsymbol{A}_{s_n,\cdot})$.

The fit depends on the seed, which picks the starting means among the observations. Fitted on 1928–2025 annual US data with $K = 3$ and seeds 1, 2, 3, 7, and 42, two regimes are persistent, with diagonal entries A_{kk} between 0.47 and 0.98 (mean runs $1/(1 - A_{kk})$ of two to fifty years), while the third is often short-lived, with A_{kk} between 0 and 0.80. The HMM is strictly more expressive than the GMM: if $\boldsymbol{A}$ is set to the outer product $\mathbf{1}\boldsymbol{\pi}^\top$ (all rows identical), the model reduces to the i.i.d. GMM, so the maximum likelihood of the HMM is at least that of the GMM. The fitted values need not compare that way: Baum-Welch, like EM for the GMM, reaches a local maximum from a random start, and the Laplace smoothing of $\boldsymbol{A}$ makes it a slightly penalized fit.

6.4.10 Gaussian copula: `historical_copula`

The Gaussian copula model [41] generates synthetic return sequences whose marginal distributions match the historical data exactly, while their joint dependence structure is captured by a parametric Gaussian copula. Unlike `historical_gaussian` and `historical_lognormal`, which impose a Gaussian (or log-normal) shape on every marginal, this method is fully non-parametric in the marginals: it preserves the exact empirical distribution of each asset class — including the left skew of equity returns, the right skew of T-Note returns, and the right skew of inflation.

Motivation Sklar's theorem [41] states that any joint distribution can be decomposed into its marginal distributions and a copula that captures all dependence. In the Gaussian copula family, the dependence is parameterized by a $N_k \times N_k$ correlation matrix $\boldsymbol{R}$, separating the fitting of individual-asset behavior from the fitting of cross-asset correlations. For retirement planning, this matters because inflation in particular is neither Gaussian nor log-normal; preserving its empirical shape avoids systematic bias in real spending projections.

Fitting Let $\boldsymbol{r}_y = (r_{1y}, \ldots, r_{N_k,y})^\top$ denote the observed return vector for calendar year y in the selected window $[\mathcal{Y}_{\text{frm}}, \mathcal{Y}_{\text{to}}]$, with N_w total observations.

Step 1 — rank-based empirical CDF. For each asset class k, map the N_w observations to the uniform grid via rank statistics:

$$U_{ky} = \frac{\text{rank}(r_{ky}) - \frac{1}{2}}{N_w}, \qquad y = \mathcal{Y}_{\text{frm}}, \ldots, \mathcal{Y}_{\text{to}}, \tag{6.44}$$

where rank is computed within the window for each k separately. The $-\frac{1}{2}$ centering keeps all values strictly inside $(0, 1)$ so that the next step is well-defined.

Step 2 — normal scores. Apply the standard normal quantile function to obtain copula variates:

$$Z_{ky} = \Phi^{-1}(U_{ky}), \tag{6.45}$$

where Φ denotes the standard normal CDF.

Step 3 — copula correlation matrix. Estimate the $N_k \times N_k$ Gaussian copula correlation matrix from the normal scores:

$$\boldsymbol{R} = \text{corrcoef}\left(\boldsymbol{Z}^\top\right), \tag{6.46}$$

where $\boldsymbol{Z} \in \mathbb{R}^{N_w \times N_k}$ collects all normal-score rows. $\boldsymbol{R}$ captures Pearson (linear) correlation in the normal-score space, which corresponds to Spearman rank correlation in the original return space.

Sequence generation To draw one plan year of synthetic returns:

Step 4 — sample from the copula. Draw a standard multivariate normal vector:

$$\boldsymbol{z}_n \sim \mathcal{N}(\boldsymbol{0},\, \boldsymbol{R}). \tag{6.47}$$

Step 5 — map back to the original scale. Convert to uniform scores and apply the empirical quantile function for each asset class:

$$u_{kn} = \Phi(z_{kn}), \qquad \tau_{kn} = \hat{Q}_k(u_{kn}), \tag{6.48}$$

where $\hat{Q}_k$ is the empirical quantile function of asset k, obtained by linear interpolation over the N_w sorted historical values $r_{k,(1)} \leq \cdots \leq r_{k,(N_w)}$ at the grid $\{(i - \frac{1}{2})/N_w\}_{i=1}^{N_w}$. Values of u_{kn} outside $[\, u_{k,1},\, u_{k,N_w}]$ are clamped to the observed $[\min, \max]$, preventing extrapolation beyond the historical record. The resolution of $\hat{Q}_k$ equals N_w, so a wider window provides finer quantile resolution.

Step 6 — inflation floor. To exclude the Great Depression artefact of deeply negative inflation, the generated inflation series is floored at -5%.

Each of the N_n plan years is drawn independently; the model captures cross-asset rank correlation but no serial autocorrelation across years.

6.5 User-supplied rates: `dataframe`

The `dataframe` method allows the user to bypass all built-in rate models and supply the return matrix $\{\tau_{kn}\}$ directly as a pandas `DataFrame` (one row per plan year, with columns named `S&P500`, `BondsBaa`, `T-Notes`, and `Inflation`). This is useful for custom scenarios, external forecasts, or deterministic stress tests not expressible through the other methods. The rows are read in order, after skipping the first `offset` rows (default 0); the DataFrame must hold at least N_n rows past the offset, and a shorter one is refused rather than repeated. Values are read in percent unless `in_percent` is false, in which case they are taken as decimal fractions. No statistical estimation or sampling is performed; the user bears full responsibility for the consistency and units of the supplied data. This method is accessed programmatically via the Python API (`setRates("dataframe",...)`); it is not available through the TOML configuration file.

6.6 Monte Carlo and stress testing

In Monte Carlo mode, Owl calls one of the stochastic models (`historical_gaussian`, `historical_lognormal`, `gaussian`, `lognormal`, `historical_bootstrap`, `historical_copula`, `vector_ar`, `garch_dcc`, `gmm`, or `hmm`) independently N_s times, each time solving the full optimization problem with a different rate scenario. The distribution of resulting optimal spending or bequest values across the N_s scenarios provides a probabilistic assessment of the retirement plan.

In stress-testing mode (Historical Range), the `historical` method is applied repeatedly: the starting year $\mathcal{Y}_{\text{frm}}$ is advanced one year at a time over a specified calendar range, producing one deterministic solve per starting year (by default). This sweep is equivalent to asking: "How would this plan have fared had the retiree begun at every historical starting year?" When the *augmented* option is enabled, the run is expanded: for each starting year, every $(reverse, roll)$ pair in $\{\text{false}, \text{true}\} \times \{0, \ldots, N_n - 1\}$ is executed, yielding $2N_n$ sequences per year instead of one. For a 30-year plan over 50 years of history, that produces $50 \times 60 = 3000$ outcomes rather than 50, providing a much denser sample for the result histogram while still using only observed historical returns. Reverse and roll (see § 6.1) alter the ordering of the same returns; augmented mode systematically explores all such orderings.

Robustness of first-year decisions Only the first year's decisions of a plan are executed; later years are re-optimized as returns realize. Owl therefore also reports, across the same scenario set, the distribution of the *first-year* decisions: per-person

Roth conversion percentiles with the fraction of scenarios recommending a conversion and an agreement metric (share of scenarios within 10% of the median), withdrawals by account, net spending, and the modal top ordinary-income bracket (`summarize_year1` in `stresstests.py`, surfaced through the `run_year1_robustness` assistant tool). A statement such as "convert \$45k–\$60k this year in 87% of scenarios" is more decision-relevant than any single-scenario schedule. Each scenario is solved with perfect foresight of its own return path, so the spread *understates* true decision uncertainty; a two-stage stochastic formulation committing the first-year decisions across scenarios would price that uncertainty and is left as future work.

7. Stochastic Spending Optimization and Longevity Risk

A natural measure of plan robustness is the *Probability of Success* (PoS), defined as the fraction of scenarios in which the chosen spending level is fully sustained throughout the plan horizon. While intuitive, PoS is a binary criterion: a plan that falls short by one percent and a plan that collapses completely are counted identically. Two plans with the same PoS may therefore carry very different risk profiles.

To address this limitation, Owl adopts a framework that treats the retirement spending decision as a commitment made *before* the sequence of returns is known. The retiree commits to a single real spending level g^*, and for each scenario s the shortfall—the amount by which g^* exceeds what the scenario can sustain—is measured explicitly. Tracing the trade-off between committed spending and expected shortfall as a two-dimensional Pareto frontier, controlled by a risk-aversion parameter, yields a richer picture of plan risk that is analogous to the mean-variance efficient frontier of Markowitz portfolio theory [43]. The conservative extreme of the frontier recovers the classical safe withdrawal rate of Bengen [44]: the spending level that succeeds in every scenario.

7.1 Scenario bases and the spending commitment

Let $s = 0, 1, \ldots, S-1$ denote the scenario index (0-based, following the C-array convention used throughout this document), where S is the total number of scenarios considered. For each scenario s, Owl solves one full optimization problem (the LP, or MILP when options call for binary variables, described in the preceding chapters) with the return sequence $\{\tau_{kn}^{(s)}\}$ of that scenario. The result is the *scenario basis*

$$g_s \;=\; \max\bigl\{g : \text{the plan is feasible under scenario } s \text{ at spending level } g\bigr\}. \quad (7.1)$$

Because g_s is computed by the full optimization, it incorporates all tax rules, RMDs, Social Security, Medicare, and Roth conversion constraints as modeled there. The basis is the spending at profile factor one; the first year's net spending of the scenario is $\xi_0 g_s$, which equals g_s on a flat profile. The commitment LP below is positively homogeneous, so its frontier is the same in either unit, up to that scale, and Owl reports it in both. Social Security claiming ages are held at the plan's values in every scenario (Chapter 9): choosing them again in each scenario would choose them knowing that scenario's returns and lifespans.

The retiree must commit to a single first-year real spending level g^* *before* observing which scenario will unfold. In scenario s the *shortfall* is

$$\varsigma_s \;=\; \max(0,\; g^* - g_s). \tag{7.2}$$

Scenario s is a *success* when $\varsigma_s = 0$. The *Probability of Success* is the fraction of scenarios without shortfall:

$$\varrho \;=\; \frac{1}{S}\,\#\big\{s : \varsigma_s = 0\big\}, \tag{7.3}$$

and the *mean shortfall* is

$$\bar{\varsigma} \;=\; \frac{1}{S}\sum_{s=0}^{S-1} \varsigma_s. \tag{7.4}$$

7.2 The commitment LP and efficient frontier

Given the scenario bases $\{g_s\}$ and a non-negative risk-aversion parameter $\Lambda \geq 0$, the optimal committed spending is determined by the *commitment LP*:

$$\begin{aligned} \text{maximize} \quad & g^* - \frac{\Lambda}{S}\sum_{s=0}^{S-1} \varsigma_s \\ \text{subject to} \quad & \varsigma_s \geq g^* - g_s, \qquad s = 0, \ldots, S-1, \\ & \varsigma_s \geq 0, \qquad\qquad\; s = 0, \ldots, S-1, \\ & 0 \leq g^* \leq \max_s\, g_s. \end{aligned} \tag{7.5}$$

The decision variables are g^* and the S shortfall variables $\{\varsigma_s\}$. The parameter Λ controls the spending–shortfall trade-off:

- $\Lambda = 0$: maximize g^* without any shortfall penalty. This is the most aggressive point on the frontier; it corresponds to committing to the best-case scenario basis, $g^* = \max_s g_s$.

- $\Lambda \to \infty$: the shortfall penalty dominates, forcing $\varsigma_s = 0$ for all s. This is the most conservative point; it recovers the safe withdrawal rate $g^* = \min_s g_s$.

The *efficient frontier* is the curve traced in the $(g^*, \bar{\varsigma})$ plane as Λ is swept over 61 values: $\Lambda = 0$ and 60 values spaced logarithmically from 10^{-1} to 10^3. Every frontier point is Pareto-optimal: no other committed spending level achieves simultaneously higher g^* and lower $\bar{\varsigma}$. Table 7.1 draws the analogy between this framework and mean-variance portfolio theory.

Concept	**Portfolio theory**	**Spending framework**
Decision variable	Portfolio weights	Committed spending g^*
Objective	Expected return	Committed spending g^*
Risk measure	Variance σ^2	Mean shortfall $\bar{\varsigma}$
Risk-aversion param	Λ	Λ
Conservative extreme	Min-variance portfolio	Safe withdrawal rate
Aggressive extreme	Max-return portfolio	Best-case scenario

Table 7.1: Analogy between Markowitz mean-variance portfolio theory and the spending/shortfall efficient frontier.

Connection to Conditional Value-at-Risk. The mean shortfall $\bar{\varsigma}$ is related to the Conditional Value-at-Risk (CVaR) [45, 46] at confidence level ϱ by

$$\mathrm{CVaR}_{\varrho} \;=\; \frac{\bar{\varsigma}}{1-\varrho}. \tag{7.6}$$

CVaR is a coherent risk measure that quantifies the expected shortfall conditional on being in the failure region. Because the commitment LP (7.5) directly minimizes mean shortfall, it is equivalent to minimizing CVaR at the corresponding success rate.

Owl reports a floor-capped version that bounds the heavy tail of Monte Carlo ensembles: given a spending floor F, each scenario's shortfall is capped at $g^* - F$,

$$\mathrm{CVaR}^{F}_{\varrho} \;=\; \frac{1}{1-\varrho}\,\frac{1}{S}\sum_{s=0}^{S-1} \max\bigl(0,\; g^* - \max(F, g_s)\bigr). \tag{7.7}$$

With $F = \min_s g_s$, the historical safe floor, this is the CVaR of Eq. (7.6). The *Retirement Efficiency Score* at each frontier point is the spending above the floor per unit of tail risk,

$$\mathrm{RES} \;=\; \frac{g^* - F}{\mathrm{CVaR}^{F}_{\varrho}}, \tag{7.8}$$

defined where $g^* > F$ and the CVaR is positive. Owl reports its maximum, the success rate ϱ^* at which it occurs, and the CVaR there and at the user's target success rate (functions `compute_cvar` and `compute_res`).

7.3 Longevity risk

All scenario analyses described so far use a fixed planning horizon N_n equal to the assumed life expectancy. In practice, however, the time of death is uncertain, and the risk of outliving one's assets—*longevity risk*—is a central concern in retirement planning. Owl models longevity risk by randomizing the planning horizon within each Monte Carlo scenario.

For each Monte Carlo scenario s and each individual $i \in \{0, \ldots, N_i - 1\}$, an age-at-death $T_i^{(s)}$ is drawn independently from the SSA period life table conditioned on having survived to the individual's current age:

$$T_i^{(s)} \;\sim\; F_i\big(\,\cdot \mid \text{current age}_i,\, \text{sex}_i\big), \qquad i = 0, \ldots, N_i - 1, \tag{7.9}$$

where F_i is the discrete conditional survival distribution. For a couple, the plan must remain funded until the last survivor, so the effective plan horizon for scenario s is

$$T^{(s)} \;=\; \max_i \, T_i^{(s)}. \tag{7.10}$$

Each scenario is solved with a fresh plan clone in which each individual's life expectancy is set to that individual's drawn age $T_i^{(s)}$, so the optimization horizon ends at the last-survivor horizon $T^{(s)}$ and the first death, with its change of filing status and survivor benefits, falls where it was drawn. Scenarios with $T^{(s)} \leq 1$ year are pre-assigned $g_s = 0$ to handle the degenerate case of an imminent end of horizon.

7.4 Scenario generation

The scenario bases $\{g_s\}$ are generated by one of two methods. The rate models underlying each method are described in Chapter 6.

Historical Each calendar starting year $\mathcal{Y}_{\text{frm}}, \ldots, \mathcal{Y}_{\text{to}} - N_n + 1$ yields one fixed real-return sequence and therefore one scenario. S equals the number of valid starting years. Augmented mode expands this set by applying all $(reverse, roll)$ combinations, yielding up to $2N_n$ sequences per starting year.

Monte Carlo S independent return paths are drawn from the plan's stochastic rate model (see Chapter 6); a deterministic method is refused. The resulting bases span the full tail of the return distribution, including sequences more extreme than any observed historically. Longevity risk sampling is available in this mode only (see below).

7.5 Implementation in Owl

The stress-test computation is implemented in `stresstests.py` and exposed through the `runStochasticSpending` method of the `Plan` class, which calls `run_stochastic_spending` in that module. Each of the S scenarios is solved independently and in parallel using a `ThreadPoolExecutor` with up to `cpu_count` workers; each worker receives an independent clone of the base plan and calls `plan.solve` with the `"maxSpending"` objective. Scenarios for which Owl finds no feasible solution are assigned $g_s = 0$, contributing a full shortfall $\varsigma_s = g^*$ to the commitment LP—the most conservative treatment consistent with the framework. Once all S bases are collected, the commitment LP (7.5) is solved for each of the 61 values of Λ; because the LP has only $S + 1$ variables, this final sweep is fast relative to the scenario solves.

The Streamlit interface (`ui/Spending_Optimization.py`) caches the scenario data; adjusting the target success rate ϱ via the slider merely looks up the corresponding frontier point without re-solving any scenario. An integer seed may be supplied for reproducible Monte Carlo paths and longevity draws. The `run_stochastic_spending` call returns a dictionary whose keys include `bases`, `frontier_g`, `frontier_prob`, `frontier_shortfall`, `lambdas`, and `drawn_lifespans`, the first-year versions `bases_year1`, `frontier_g_year1`, `frontier_prob_year1`, and `frontier_shortfall_year1`, the profile factor `xi_0`, the per-scenario `partial_bequests`, and the first-year decisions `year1_decisions`.

Limitations and assumptions

- Longevity risk sampling is supported only with Monte Carlo scenarios. Historical scenarios have a bounded return series; a drawn lifespan could exceed the length of available data, making the combination infeasible.
- Scenario bases are computed independently: the return sequences of different scenarios are assumed to be statistically independent. This is exact for Monte Carlo but approximate for historical scenarios, which share overlapping return sub-sequences when the plan horizon exceeds one year.

- The historical scenario set is limited by the length of the available dataset. For a 30-year plan, only starting years up to $\mathcal{Y}_{\max} - N_n + 1$ are valid, reducing S as the horizon grows.
- The commitment LP assumes that the scenario bases $\{g_s\}$ are fixed parameters independent of g^*. This is valid because each scenario's solve maximizes spending given the return sequence, so g_s does not depend on the committed level.
- Longevity draws use static life tables, conditioned on current age. Eight tables are selectable, here ordered by life expectancy at age 65 (average of men and women): the SOA VBT 2015 Smoker table (82.3 years), the SSA 2025 period table (83.3, the general US population, and the default), the SOA Pub-2010 Public Safety (84.9) and General Employees (85.6) tables, the SOA RP-2014 Healthy Annuitant table (85.9), the VBT 2015 Non-Smoker table (86.7), the SOA IAM 2012 Individual Annuity table (86.7), and the SOA Pub-2010 Teachers table (87.1). The spread matters: an annuitant or a non-smoker table carries a materially longer horizon than the population table, and occupation and smoking status are the two inputs that select among them. Cohort tables, which account for expected future mortality improvements, are not used.

8. The Spending–Bequest Trade-off

The two objectives of Chapter 5 are the two ends of one question: how much spending does each dollar left to heirs cost? Maximizing spending under a bequest floor gives one point of the answer, and maximizing the bequest under a spending level gives another. Owl traces the whole curve between them by sweeping the bequest floor ϵ_{N_n} of Eq. (5.3) under the `maxSpending` objective. The user-facing surfaces call the result the spending–bequest *trade-off*, to keep it apart from the spending–shortfall frontier of Chapter 7: both are Pareto frontiers, but only that one is a risk–return curve in the sense of Markowitz.

8.1 Why the bequest is swept and spending is not

The same curve could in principle be traced the other way, by pinning the spending level and maximizing what is left. Under `maxSpending`, pinning g_0 makes the objective constant over the feasible set, and the bracket-fill variables f_{tn}, which are tight only because the objective pushes income into the cheapest brackets, are then no longer priced: the solver may fill a top bracket while lower ones sit empty (Section 3.5; issue #140). A bequest floor constrains the estate instead, so g_0 stays free, the objective keeps a nonzero gradient, and every point of the sweep is a genuine optimum of an ordinary solve. No new optimization structure is involved.

8.2 Deterministic trade-off

Given a grid of bequest levels $B_0 < B_1 < \cdots < B_{K-1}$ in today's dollars, sorted and without duplicates, Owl solves the plan on its own rates at each level,

$$S(B_k) \;=\; \max\big\{\, g \;:\; \text{the plan is feasible with } \epsilon_{N_n} = B_k \text{ and spending basis } g \,\big\}, \tag{8.1}$$

where g is the spending basis of Section 3.5, the average of the yearly amounts; the first year's spending is $\xi_0 S(B_k)$. A level that does not solve is recorded as unreachable.

Exchange rate. The cost of the bequest is measured on the curve itself, one grid segment at a time, by the secant

$$r_k \;=\; \frac{S(B_k) - S(B_{k-1})}{B_k - B_{k-1}}, \qquad k = 1, \ldots, K-1, \tag{8.2}$$

the change in spending basis per today's dollar of bequest (negative on a sloping curve), and $\xi_0 r_k$ in first-year dollars. One entry is reported for every segment, empty when either end did not solve, so that the k-th entry always describes the segment ending at B_k.

Shadow-price cross-check. On request, each level is also solved with duals, and the dual π of the `bequest_floor` row converts to the lifetime spending, in today's dollars, that one more today's dollar of bequest costs, $\Pi_B = -\pi\, c_{\text{obj}}\, \gamma_{N_n}$, where c_{obj} converts the solver's objective into today's dollars of the reported objective. Spread over the profile, it implies the basis slope

$$\hat{r}_k \;=\; -\Pi_B(B_k) \Big/ \sum_n \xi_n, \tag{8.3}$$

which is compared with the secant: the segment is flagged reliable when $|\hat{r}_k - r_k| \le 0.2\,|r_k|$. The dual belongs to the final LP with the self-consistent loop's parameters frozen, so it is the partial derivative of a fixed point and runs shallow; a larger disagreement means the segment is driven by the loop (taxable Social Security, IRMAA, gain fractions) rather than by the LP, and the secant, not the dual, is the slope to read. The dual is not computed in the stochastic modes below, where a single scenario's price says nothing about the ensemble.

Reach. A sweep can only bracket the largest estate a plan can leave: it lies at or above the highest level that solved, and below the lowest level above it that did not. Failures need not form a suffix of the grid, since a solve can time out at one level and succeed above it, so the upper end is the lowest failure *above* the highest success. When every level solves, the bracket has no upper end and the highest level merely echoes the grid.

What is not reported. A "free bequest", the largest floor that costs no spending, and a "knee" of the curve both read as properties of the plan but are set by the grid: the first is zero for any curve that slopes at all, and the second lands on the second grid point whenever the first two segments differ. Resolving either needs a much finer sweep than a handful of levels, so neither is reported.

8.3 Stochastic trade-off

In the historical and Monte Carlo modes, every bequest level is solved across the whole scenario ensemble of Chapter 7, giving the scenario bases $g_s(B_k)$, and the commitment LP of Eq. (7.5) is solved over them for every Λ. This turns the curve into a surface $S(B, p)$: spending as a function of the bequest target B and of the probability p of not falling short. For each success rate p requested (by default 50%, 75%, and 90%), the spending reported at level B_k is the committed spending g^* at the least conservative Λ whose shortfall probability is at most $1 - p$. Reading the surface down a column gives spending against bequest at fixed confidence, the fan across p being the sequence-of-returns risk; reading it across a row gives the spending–success curve of Chapter 7 at that bequest. The exchange rate of Eq. (8.2) is then measured along the column of a chosen success rate (90% by default).

Three details keep the surface well behaved. In Monte Carlo mode the rate generator is seeded once (seed 0 unless one is given), so every bequest level meets the *same* ensemble: without common random numbers, the surface wanders non-monotonically in B on sampling noise alone. A scenario that cannot reach a level enters the commitment LP with basis zero, a full shortfall, as in Chapter 7; only a level that fails outright is unreachable, since individual scenarios failing at a high floor is exactly what the success rates express. And Social Security claiming ages are held at the plan's values (Chapter 9), as in every scenario analysis.

8.4 What reaches heirs

The bequest B is the after-tax value of the savings accounts at the end of the plan, net of remaining debt (Eq. (5.3)). Two further amounts reach heirs and are reported beside it. Fixed assets still held at the end of the plan are set by the asset table rather than by the floor, so their value is the same at every level and is added on top of every bequest figure; Owl also notes whether that today's-dollar value depends on the inflation path, which it does unless every asset held to the end grows at a real rate quoted at the plan's start. The partial bequest passing to heirs other than the

spouse at the first death, net of their tax, is solved rather than set, so it moves with the floor; it is reported per level, as the median and range across scenarios in the stochastic modes. The total estate at each level is the sum of the three.

8.5 Implementation in Owl

The sweep is `run_spending_bequest_frontier` in `stresstests.py`, exposed as `Plan.runSpendingBequestFrontier`, and summarized by `summarize_spending_bequest_frontier`, which computes the exchange rates, the reach bracket, and the estate rows. It is offered through the *Spending vs Bequest* page of the user interface (default grid \$0, \$500k, \$1M, \$2M, and \$4M), the `owlclifrontier` command, and the `run_spending_bequest_frontier` tool of the assistant. Each solve is limited to 120 seconds unless the options set `maxTime`, since a sweep multiplies any pathological scenario by the number of levels. Every spending figure is reported twice, as the basis the solver optimizes and as the first year's amount, which is the quantity the `netSpending` option of `maxBequest` pins and therefore the one to compare a `maxBequest` run against; the two differ by ξ_0 and coincide on a flat profile.

9. Optimization of Social Security Claiming Ages

The age at which an individual claims Social Security retirement benefits is one of the most consequential financial decisions in retirement planning. Claiming early—as young as age 62—provides income sooner but at a permanently reduced rate; deferring to age 70 maximizes the monthly benefit through delayed retirement credits but foregoes years of payments. The optimal age depends on individual longevity expectations, household income, tax situation, and the presence of a spouse whose spousal and survivor benefits are linked to both partners' claiming decisions.

In Owl's base operating mode, the user supplies a fixed claiming age for each individual and the Social Security benefit stream $\bar{\zeta}_{in}$ is computed as a deterministic parameter. When the option `withSSAges` selects one or more individuals, Owl chooses their claiming ages by *solving the plan at each claiming month still open* and keeping the ages that gave the best objective. Each candidate is an ordinary fixed-age solve, in the case's own tax modes, so the benefits of every candidate age are taxed exactly as that age would be taxed. The claiming age is therefore not a variable of the optimization problem: it is an outer search whose inner problem is the plan of Chapter 4, unchanged. Section 9.4 records why the claiming age is not chosen inside the MILP, which is how Owl once did it.

9.1 The claiming-age grid

Social Security claiming ages are discretized to a monthly grid. The ages open to individual i run from eligibility, or from the current month if that is later, through age 70 in whole months:

$$\mathcal{A}_i \;=\; \left\{ \frac{m}{12} \;:\; m = \max\left(m_i^{\text{elig}},\, m_i^{\text{today}}\right), \ldots, 840 \right\}, \tag{9.1}$$

where m counts whole months of age, $840 = 70 \times 12$, and m_i^{today} is the age in whole months that individual i reaches in the current calendar month. A claiming month already behind us is excluded because SSA does not pay retroactively for it, so the grid shrinks as a plan ages: it holds at most 97 points, for someone not yet 62, and fewer for anyone older. Eligibility is $m_i^{\text{elig}} = 744$ (62 years, 0 months) for an individual born on the first or second day of a month, and 745 (62 years, 1 month) otherwise, following the SSA rule that a benefit cannot begin in the month of attainment unless age 62 is attained on its first day.

An individual whose entered claiming age is *already past* cannot be searched, and Owl refuses the case rather than silently holding that age. A past age reads as benefits already being collected, and a claim cannot be redone: every month still open would pay more than the reduced benefit actually received. An individual with no PIA, or one not named by `withSSAges`, keeps the entered age and contributes a fixed benefit stream as in the base mode.

9.2 Claiming-age adjustment factors

The benefit multiplier $\Phi(\alpha, \text{FRA})$ encodes the SSA rules for early reduction and delayed credits. It is applied not to the conventional claiming age α but to the *SSA age*

$$\tilde{\alpha} \;=\; \min\!\big(\alpha + \tfrac{1}{12}\mathbb{1}[\text{born on the 1st}],\; 70\big)\,, \tag{9.2}$$

because an individual born on the first of a month attains each age on the last day of the prior month and has therefore been that age one month longer (POMS RS 00615.015); being born on the second confers no such shift. The cap at 70 is the end of delayed credits, which stop with the month before age 70 is attained (42 U.S.C. 402(w)(2)(A); POMS RS 00615.690), so an individual born on the first earns no credit for a claim at a conventional age of 70. With that substitution,

$$\Phi(\alpha, \text{FRA}) = \begin{cases} 1 + 0.08\,(\tilde{\alpha} - \text{FRA}), & \tilde{\alpha} \geq \text{FRA}, \\ 1 - \frac{1}{15}\,(\text{FRA} - \tilde{\alpha}), & \text{FRA} - 3 < \tilde{\alpha} < \text{FRA}, \\ 0.80 - \frac{1}{20}\,(\text{FRA} - 3 - \tilde{\alpha}), & \tilde{\alpha} \leq \text{FRA} - 3. \end{cases} \tag{9.3}$$

Delayed credits of 8% per year apply above FRA, capping at $f = 1.24$ at age 70 when FRA is 67. For early claiming the reduction is $\frac{1}{15}$ per year (i.e., 5/9 of 1% per month) for the first 3 years before FRA, and $\frac{1}{20}$ per year (i.e., 5/12 of 1% per month) for any additional years, yielding a minimum of approximately 70% of PIA

at age 62. The Full Retirement Age FRA_i is determined from the individual's birth year as described in Chapter 2.

A separate factor applies to the spousal benefit, which receives no delayed credit beyond the spousal FRA and whose reduction before FRA is 25/36 of 1% per month for the first 36 months and 5/12 of 1% for each further month, so that a spouse claiming at 62 with an FRA of 67 receives 32.5% of the worker's PIA. Where benefits are withheld by the earnings test, the months withheld are credited back at FRA through the adjusted reduction factor, which recomputes Φ at an effective age advanced by the number of crediting months.

9.3 The search over fixed-age solves

Let $\mathcal{F}$ be the set of individuals whose ages are searched and $V(\boldsymbol{\alpha})$ the objective value of the plan solved with claiming ages $\boldsymbol{\alpha}$, in the case's own tax modes and with `withSSAges` set to `"fixed"`. A candidate that does not solve contributes no value and is skipped.

For a single searched individual the search is exhaustive: every age in $\mathcal{A}_i$ is evaluated and the best is kept. For two, the ages interact through the spousal and survivor benefits, and Owl runs coordinate descent: each searched individual's age is swept over $\mathcal{A}_i$ in turn while the other is held at its incumbent value, and the sweeps repeat until a full round moves neither age, to a maximum of five rounds,

$$\boldsymbol{\alpha}_i^{(k+1)} \;=\; \arg\max_{a \in \mathcal{A}_i} \; V\left(a, \boldsymbol{\alpha}_{-i}^{(k)}\right), \qquad i \in \mathcal{F}, \tag{9.4}$$

starting from the ages entered in the case. An age replaces the incumbent only on a strict improvement, with a relative margin of 10^{-9}, so ties keep the incumbent and the sweep terminates.

Because every candidate is a complete solve of the unmodified plan, the ages returned are never worse than any ages the user could have entered by hand, and the reported plan is exactly the plan of the winning candidate: Owl sets the chosen ages and solves once more to produce it. The cost is one solve per candidate. A single individual costs $|\mathcal{A}_i| + 1$ solves, at most 98; a couple costs up to five rounds of $|\mathcal{A}_0|+|\mathcal{A}_1|$, less the candidates already evaluated. Candidates are therefore memoized by their month-pair key and run on independent clones of the plan in a thread pool sized to the number of cores, which is where the parallelism of a claiming-age search lives—not inside any one solve.

The earnings test is evaluated for each candidate within its own solve, so the search sees the benefits actually payable at each age. The warning that a candidate's

benefits would be wholly withheld is emitted once, before the candidates, computed at the earliest ages still open, so that it covers every year the test could reach. The survivor claiming age is held at its entered value and is not searched.

9.4 Why the claiming age is not a MILP variable

Owl once chose the claiming age inside the MILP, with a binary per claiming month selecting one age from the grid, an at-most-one row, and a precomputed benefit table linking the selection to a continuous benefit variable. Spousal and survivor benefits, which depend on both partners' ages in a way that is not linear, were carried as a parameter refreshed by the self-consistent loop. That formulation has been removed, for two reasons that are worth recording because they apply to any similar reformulation.

First, it priced candidates on the wrong tax. Every row that charges tax or premiums on benefits—taxable income, the IRMAA and ACA MAGI rows, the state Social Security exclusion—used the previous iterate's benefits, so the MILP compared claiming ages on benefits that were very nearly pre-tax, and on the case of issue #195 it selected the earliest open month.

Second, taxing them exactly made the problem intractable. Once the benefit is a variable, the taxable-benefit formula and the bracket rows need their own binaries per year, and neighbouring claiming ages then differ by less than the relative gap the solver is asked to close: the MILP cannot distinguish the candidates it is meant to rank. A month of claiming age is a small change in a large plan, which is precisely the regime in which an outer search over exact solves beats a single reformulated model.

9.5 Implementation in Owl

The search is implemented by `_ssAgeSearch` in `plan.py`, which intercepts the solve before the model is built whenever `withSSAges` names an individual. Claiming ages, FRAs, adjustment factors, and the spousal and survivor streams are computed in `socialsecurity.py`; each candidate sets them through `setSocialSecurity` on its own clone, preserving the case's benefit trim and survivor claiming age. `withSSAges` accepts `"fixed"` or `"none"` (no search, the default), `"optimize"` (every individual with a PIA), or an individual's name or a list of names, which searches those individuals only.

Scenario analyses hold the claiming ages rather than searching them: a historical range, Monte Carlo run, stochastic spending frontier, spending–bequest frontier, or conversion-regret sweep would otherwise run the whole search once per scenario. Those tools pin `withSSAges` to `"fixed"` at the plan's current ages and report that they have done so.

Limitations and assumptions

- The search is a coordinate descent for couples, not an exhaustive joint search over both grids. It is exhaustive for a single individual, and for a couple it returns a point from which no single-age change improves the objective; a joint move of both ages that improves on it is not ruled out.
- The objective compared across candidates is the plan's own objective. A claiming age chosen under `maxSpending` need not be the age chosen under `maxBequest`, and neither accounts for longevity risk unless the case is run through the stochastic tools—which hold the ages fixed.
- The claiming age grid is monthly. Finer sub-monthly grids are not supported, and the grid excludes months already past.
- The survivor benefit is assumed to be claimed at the entered survivor claiming age; deferring the survivor benefit is not searched.
- The SSA family maximum benefit cap is not modeled; benefits may be overstated when multiple beneficiaries claim on one worker's record.
- An individual already collecting benefits cannot be searched, and a case that asks for one is refused rather than solved with that age held.

A. Indices, variables, and parameters

In the next sections, the indices, variables, and parameters are described in detail. Then the model constraints are introduced. For implementation in a linear programming solver, index mapping functions are introduced to map all variables into a single one-dimensional array that is optimized subject to inequality and equality constraints expressed in matrix form. Finally, the constraint matrices are built and so are some useful objective functions.

A.1 Indices

For all indices, we will follow the C array style (starting at 0), rather than the traditional mathematical standard starting at 1. This will facilitate the final sequential mapping of all the variables into a single one-dimensional array, and serve as a direct reference for better understanding the code implementation.

The indices used and their range are defined here, while we also introduce the characteristics and dimensions of the problem. Upper bounds on indices are indicated by the letter N, with the index name as a subscript, e.g., N_i for index i. In other instances, the subscript will indicate that the array depends on subscripts, for example, b_{ikn} is a multidimensional array depending on subscripts i, k, and n.

i Individual. i runs from 0 to $N_i - 1$ where $N_i = 2$ for couples, or $N_i = 1$ for single individuals. The first individual to pass is denoted by i_d while the survivor is i_s.

j Type of savings account. j goes from 0 to $N_j - 1$, for taxable ($j = 0$), tax-deferred ($j = 1$), tax-free Roth ($j = 2$), and Health Savings Account ($j = 3$) respectively. Therefore $N_j = 4$.

k Type of asset class. k goes from 0 to $N_k - 1$, for S&P 500, Baa corporate bonds, Treasury notes, and cash, respectively, and therefore $N_k = 4$. More asset classes could be considered at the cost of increasing the complexity of the problem while not generating many more insights.

n Index of the year being modeled. The period being modeled runs from the beginning of year 0 to the first day of the year following year $N_n - 1$, and therefore $N_n + 1$ years are actually considered. Year N_n is the first year following the passing of all individuals in the plan. The time period for all decision variables is annual. For spouses, the end of year $n_d - 1$ is when the first individual is assumed to pass while the survivor is assumed to die at the end of year $N_n - 1$ of the plan. $n_{m,i}$ denotes the first year individual i is on Medicare (turns 65), so that $n_m = \min_i n_{m,i}$ is the first year any individual is on Medicare and no Medicare cost is incurred for $n < n_m$. $n_{\text{aca}} = \max_i \min(n_{m,i},\, N_n^{(i)})$, where $N_n^{(i)}$ is the planning horizon of individual i, is the first year no individual is ACA-eligible; ACA applies for $n < n_{\text{aca}}$. For a single individual $n_m = n_{\text{aca}}$; for a couple $n_m \leq n_{\text{aca}}$, with both Medicare and ACA costs potentially applying in the transition years $n_m \leq n < n_{\text{aca}}$.

p Index for long-term capital gains (LTCG) tax rate brackets. $p \in \{0, 1, 2\}$ corresponds to the 0%, 15%, and 20% LTCG rates, respectively, giving $N_p = 3$ brackets.

q Index for income brackets related to Medicare premium adjustments. q goes from 0 to $N_q - 1$, from low to high. At the time of writing, there are 5 Medicare premium step adjustments in the federal tax code separating $N_q = 6$ income brackets.

r ACA MAGI bracket index. r goes from 0 to $N_r - 1$ with $N_r = 8$ brackets (7 FPL-based intervals up to 400% FPL, plus one above).

t Federal income tax bracket. t goes from 0 to $N_t - 1$, from low to high. At the time of writing, there are $N_t = 7$ federal income tax brackets in the current federal tax code.

A.2 Variables

We will use lowercase roman letters to represent variables. This is done to separate variables from parameters, which will be represented using Greek letters or

caligraphic fonts. All variables are assumed to take only non-negative values (≥ 0 inequality). Variables are resolved by the optimizer while parameters are prescribed by the user, historical data, or tax laws.

b_{ijn} Balance for individual i in savings account j at the beginning of year n. When we consider each asset class k, the variable b_{ijkn} is used instead.

d_{in} Deposit of year-n net spending surplus in taxable account of individual i. These deposits are coming from the surplus s_n, distributed to spousal taxable accounts depending on parameter η.

e_n Adjusted standard exemption for year n. This is a variable as the taxable income can sometimes be less than the standard exemption $\bar{\sigma}_n$, leading to a negative taxable income if the inflation-adjusted standard exemption is simply subtracted from the gross taxable income G_n in years of low income.

f_{tn} Amount of taxable ordinary income falling into tax bracket t. The ordinary taxable income is

$$G_n = \sum_{t=0}^{N_t-1} f_{tn}, \tag{A.1}$$

with f_{tn} bound between 0 and the bracket width $\bar{\Delta}_{tn} = \bar{\Gamma}_{tn} - \bar{\Gamma}_{(t-1)n}$. For interpretation, one can introduce the fractional fill f'_{tn} with $f_{tn} := f'_{tn}\bar{\Delta}_{tn}$ and

$$G_n = \sum_{t=0}^{N_t-1} f'_{tn}\bar{\Delta}_{tn}, \tag{A.2}$$

$$0 \leq f'_{tn} \leq 1. \tag{A.3}$$

When considering taxes, the product $\bar{\Delta}_{tn}\theta^x_{tn}$ need not be ordered monotonically, so we cannot rely on the cost structure to fill lower brackets first when minimizing. It is therefore more appropriate to optimize directly in f_{tn}, as the tax rates are progressive. Given that the rates θ^x_{tn} on tax brackets t are increasing monotonically with income, the lower brackets will be filled in first when optimizing. Definitions of $\theta^x_{tn}, \Gamma_{tn}$ and Δ_{tn} are in the section describing the parameters below.

g_n Net spending in year n.

h_{qn} Segment of Medicare bracket q in which the modified adjusted gross income (MAGI) in year $n-2$ is contained. This variable is used to model the income-related monthly adjustment amounts (IRMAA) brackets using special ordered sets of type 1 (SOS1) for the selection on the bracket indicators (as opposed to using a big-$\mathcal{M}$ method). One of the brackets, say q', will contain the MAGI from two years before $\mathbb{G}_{(n-2)}$, and therefore

$$h_{qn} = \delta_{q,q'}\mathbb{G}_{(n-2)}, \tag{A.4}$$

where $\delta_{q,q'}$ is the Kronecker delta (1 if $q = q'$, 0 otherwise), such that

$$\mathbb{G}_{(n-2)} = \sum_{q} h_{qn}. \tag{A.5}$$

The role of h will become clearer as the formulation of the MAGI calculation is exposed below.

m_n Medicare costs in year n, including Part B and Part D premiums and IRMAA income-related adjustments for both parts. Part D can be disabled via the `includeMedicarePartD=false` option. In `withMedicare="optimize"` mode, m_n is an LP variable determined within the MIP. In `withMedicare="loop"` mode (the default), m_n is computed via the SC loop and treated as a parameter.

h^{aca}_{rn} Segment of ACA bracket r in which the current-year modified adjusted gross income (MAGI) $\mathbb{G}^{\text{aca}}_n$ is contained for ACA-eligible year n. Used only when `withACA="optimize"`; index n runs over the first n_{aca} plan years when at least one individual is under age 65 and within their horizon. One bracket r' contains MAGI, so $\sum_r h^{\text{aca}}_{rn} = \mathbb{G}^{\text{aca}}_n$. ACA uses *current-year* MAGI (no two-year lag like Medicare IRMAA).

m^{aca}_n ACA net premium cost in year n (after Premium Tax Credit). Used only when `withACA="optimize"`; zero for $n \geq n_{\text{aca}}$ (Medicare-eligible years). In `withACA="loop"` mode, ACA cost is computed via the SC loop and treated as a parameter.

s_n Surplus of funds during year n, most likely caused by required minimum distributions (RMDs) or influx of money from big-ticket items (inheritance, gifts, etc.), or from the disposition of fixed assets (sale of residence, restricted stocks, etc.).

w_{ijn} Withdrawal from account j belonging to individual i at the beginning of year n. For the ($j = 1$) tax-deferred savings account, w_{i1n} is referred to as a distribution for tax purposes as it is a taxable withdrawal, and will always satisfy required minimum distributions. For the ($j = 3$) HSA, w_{i3n} represents qualified medical withdrawals, which are entirely tax-free and excluded from taxable income (simplified model; see Section 2.8).

x_{in} Roth conversion performed by individual i during year n. These events are taxable as ordinary income. Roth conversions are forced to zero in the last two years of each individual's horizon (see Limitations in Section 2.7).

$p_n^{\sigma,\mathrm{lo}}$, $p_n^{\sigma,\mathrm{hi}}$ Excess provisional income above the lower and upper Social Security taxability thresholds, respectively:

$$p_n^{\sigma,\mathrm{lo}} = \max(0,\, \Pi_n - \mathcal{P}_n^{\mathrm{lo}}), \tag{A.6}$$
$$p_n^{\sigma,\mathrm{hi}} = \max(0,\, \Pi_n - \mathcal{P}_n^{\mathrm{hi}}). \tag{A.7}$$

These continuous non-negative variables are present only when the associated solver option `withSSTaxability="optimize"`. See Eq. (4.56) for the formal constraint formulation.

$p_n^{\sigma,\min}$ 50%-zone Social Security taxability allocation, bounded in $[0,\, \Delta\mathcal{P}]$, where $\Delta\mathcal{P} := \mathcal{P}_n^{\mathrm{hi}} - \mathcal{P}_n^{\mathrm{lo}}$ is the 50%-zone width (see Parameters):

$$p_n^{\sigma,\min} = \min(\Delta\mathcal{P},\, p_n^{\sigma,\mathrm{lo}}). \tag{A.8}$$

Only present when `withSSTaxability="optimize"` is selected. Together with $p_n^{\sigma,\mathrm{lo}}$ and $p_n^{\sigma,\mathrm{hi}}$, all SS taxability LP variables use the letter p for consistency.

q_{pn} Portion of LTCG and qualified dividends allocated to bracket p in year n, where $p = 0,\, 1,\, 2$ correspond to the 0%, 15%, and 20% LTCG rate brackets, respectively (the leading letter q here is *not* the Medicare IRMAA bracket index q from the Indices list; the LTCG bracket index is p). These $N_p = 3$ continuous non-negative LP variables are always present and together partition the total LTCG:

$$q_{0n} + q_{1n} + q_{2n} \geq Q_n, \tag{A.9}$$
$$q_{0n} \leq \max\big(0,\, \bar{\mathcal{B}}_n^{15} - G_n\big)\,, \tag{A.10}$$
$$q_{0n} + q_{1n} \leq \max\big(0,\, \bar{\mathcal{B}}_n^{20} - G_n\big)\,, \tag{A.11}$$

where $\bar{\mathcal{B}}_n^{15}$ and $\bar{\mathcal{B}}_n^{20}$ are the inflation-adjusted total taxable income thresholds above which the 15% and 20% rates apply (see Parameters), and $G_n = \sum_{t=0}^{N_t-1} f_{tn}$ is the ordinary taxable income (formally defined in the Intermediate Variables section). Since the cost function is strictly convex ($0 < 0.15 < 0.20$), no binary variables are required: the LP naturally fills the lowest-cost bracket first. See Eqs. (4.35)–(4.36) for the SC-loop formulation. When `withLTCG="optimize"` is selected, G_n becomes a continuous LP variable and binary variables z^l_{0n}, z^l_{1n} replace the SC-loop approximation of bracket thresholds with exact big-$\mathcal{M}$ constraints (see Eqs. (4.39)–(4.43)).

t_n^σ Taxable Social Security amount in year n, bounded in $[0,\ 0.85\sum_i \bar{\zeta}_{in}]$. In the default SC-loop mode this quantity equals $R_n \sum_i \bar{\zeta}_{in}$ and is treated as a fixed parameter; with the solver option `withSSTaxability="optimize"` it becomes an LP decision variable computed exactly from the IRS formula without iteration.

z_*^* Binary variables are all designated by the letter z; a superscript distinguishes the seven families:

- z^m_{qn} — Medicare IRMAA bracket selectors; present when `withMedicare="optimize"`.
- z^{aca}_{rn} — ACA premium bracket selectors; present when `withACA="optimize"`.
- z^σ_{0n}, z^σ_{1n} — encode the two $\min(\cdot,\cdot)$ operations in the SS taxability MIP; present when `withSSTaxability="optimize"`.
- z^l_{0n}, z^l_{1n} — LTCG bracket-room indicators ($z^l = 1$ iff ordinary income G_n is below the 15% or 20% threshold); present when `withLTCG="optimize"`.
- z^j_n — NIIT branch selector ($z^j = 1$ when net investment income, rather than the excess of MAGI over $\mathbb{G}^{\text{NIIT}}$, bounds the tax); present when `withNIIT="optimize"`.
- z^o_{0n}, z^o_{1n} — withdrawal-ordering gates marking the taxable and the tax-deferred balances exhausted at the end of year n (Section 4.1.7); present when `withdrawalOrder="taxable_first"`.
- z^{sb}_n — marks a bonus year in which the OBBBA senior bonus is fully phased out (Section 4.1.5); present when `withSeniorBonus="optimize"`.

See the corresponding constraint subsections for the big-$\mathcal{M}$ formulations.

A.3 Parameters

For more easily distinguishing parameters from variables, all parameters are expressed either in Greek letters or uppercase roman letters using caligraphic fonts. Parameter values are either set by the user, historical data, or by the tax code.

β_{ij} Initial balances in savings accounts. These amounts are used to initialize b_{ij0}.

τ_{kn} Annual rate of return for asset class k in year n. A time series of annual return rates for each asset class. Here, inflation and the rate of return of cash ($k = 3$) are assumed to be the same. In other words, investing in cash yields constant dollars as the return perfectly matches inflation, equivalent to a Treasury inflation-protected security (TIPS) having a real yield of 0%.

γ_n Cumulative inflation at the beginning of year n computed as the product

$$\gamma_n = \prod_{n'=0}^{n-1}(1 + \tau_{3n'}), \tag{A.12}$$

with $\gamma_0 := 1$, and where n' is a dummy index. As the time span of interest goes from the beginning of the first year to the beginning of the year following the last year, variable γ_n will have $N_n + 1$ elements. Parameters indexed for inflation will be indicated by a bar on top as in $\bar{\sigma}_n$. See the next entry for a specific example.

ψ_{in} Effective capital-gain coefficient for withdrawals from person i's taxable account in year n. It is computed by the self-consistent loop and enters the LP as a fixed parameter each iteration. When a cost basis is provided via `setCostBasis()`, $\psi_{in} = (1 - K_{in}/b_{i0n})/\alpha_{i00n}$, capped at 1 (see Eq. (A.26)). Otherwise, $\psi_{in} = \max(0, \tau_{0(n-1)} - \mu)$, i.e., only the most recent year's price appreciation is treated as a gain (Eq. (A.24)).

σ_n Standard deduction. It can be adjusted for inflation as follows

$$\bar{\sigma}_n = \sigma_n \gamma_n, \tag{A.13}$$

and can be modified for additional exemptions after age 65, for example. It is a simple time series which can include any foreseeable changes in the tax code, or change in filing status due to the passing of one spouse for $n \geq n_d$. The value of $\bar{\sigma}_n$ is an upper bound for variable e_n.

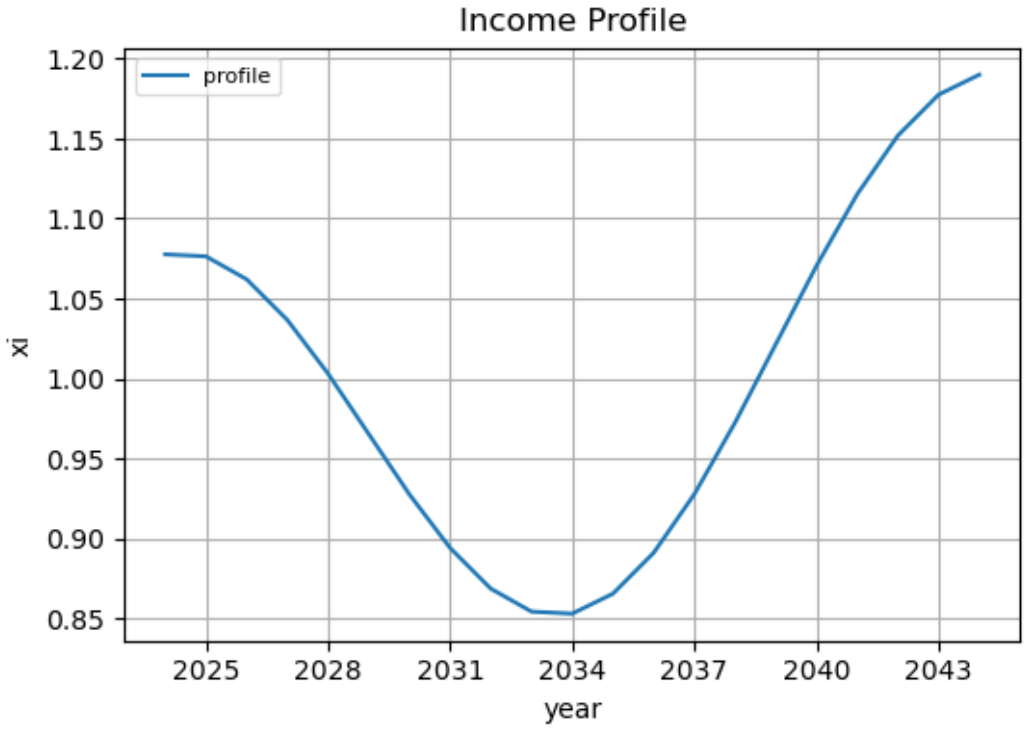


Figure A.1: Example of a spending profile with a 15% cosine factor and a 12% linear component.

ξ_n Spending profile. This is a time series that multiplies a basis for the desired net spending amount. It is $\xi_n = 1, \forall n$ for a flat profile, or can be a *smile* profile allowing for more money at the start of retirement and modulating it over retirement. Parameter ξ_n can also contain spending adjustments typically made at the passing of one spouse. The *smile* is a cosine superimposed over a gentle linear increase, optionally delayed by n_0 years, as given by Eq. (3.2). Values of $a = 15\%$ and $b = 12\%$ provide curves that are similar to realistic spending profiles reported in the literature. See Fig. A.1 for an example. At the passing of one spouse, both profiles are reduced by a factor χ for $n \geq n_d$, and the smile is normalized so that $\sum_n \xi_n = N_n - (1-\chi)(N_n - n_d)$, the sum a flat profile with the same survivorship would give. The inflation-adjusted profile is $\bar{\xi}_n = \gamma_n \xi_n$.

χ Factor to reduce spending profile after the passing of one spouse. It is typically assumed to be 0.6. That is, we are assuming that the surviving

spouse can live with 60% of the net spending amount that was available to the couple.

n_d Year index at which the first spouse (i_d) is assumed to pass. For single individuals, $n_d = N_n$. For couples, n_d is derived from the life-expectancy inputs so that the end of year $n_d - 1$ coincides with the assumed date of death of i_d. Filing status, tax brackets, Social Security thresholds, and LTCG thresholds all change at $n = n_d$ (from married filing jointly to single).

$n_{m,i}$, n_m $n_{m,i}$ is the year index at which individual i turns 65 and joins Medicare. $n_m = \min_i n_{m,i}$ is the first year any individual is on Medicare; no Medicare cost m_n is incurred for $n < n_m$. Used in the Medicare constraints; see the Medicare paragraph in the Constraints section.

$\mathcal{N}_i^{\text{hsa}}$ Year index at which individual i enrolls in Medicare and HSA contributions must cease. By default $\mathcal{N}_i^{\text{hsa}} = n_{m,i}$ (age 65), but `setHSA()` accepts a custom enrollment age so the two can differ. For $n \geq \mathcal{N}_i^{\text{hsa}}$, the contribution κ_{i3n} is zero; values in the *HSA ctrb* column for those years are ignored.

ρ_{in} Required minimum distribution for individual i in year n. Expressed in fractions determined from IRS tables. The IRS Uniform Lifetime Table III (Pub. 590-B) is used in standard cases. When the spouse is the sole designated beneficiary and more than ten years younger than the account owner, the Joint and Last Survivor Table II is used instead.

Γ_{tn} Bounds for federal income tax brackets. We define $\Gamma_{(-1)n} := 0$, so that Γ_{0n} is the upper bound for the 10% tax bracket in year n. As the filing status can change for couples, and so can the tax code, Γ_{tn} will be changing over n. Inflation-adjusted brackets are $\bar{\Gamma}_{tn} = \gamma_n \Gamma_{tn}$.

Δ_{tn} Difference between upper bound Γ_t and lower bound Γ_{t-1} of a federal income tax bracket,

$$\Delta_{tn} = \Gamma_{tn} - \Gamma_{(t-1)n}. \tag{A.14}$$

Inflation-adjusted bracket widths are $\bar{\Delta}_{tn} = \gamma_n \Delta_{tn}$. Once adjusted for inflation, the taxable income can be expressed as in Eq. (A.1). These data are 7 time series. The filing status changes after the passing of one spouse ($n \geq n_d$) and income tax brackets and differences are adjusted accordingly.

θ^x_{tn} Tax rate for ordinary income tax bracket t in year n. Using N_t time series allows one to adjust income tax rates in the foreseeable future. For example, as of 2026 the rates (in decimal) are .10, .12, .22, .24, .32, .35, and .37. While these rates were extended indefinitely by Congress in 2025, they could still revert in the future to 2017 or similar higher rates (.10, .15, .25, .28, .33, .35, and .396). See Eq. (A.34) for its use.

$\mathcal{B}^{15}$, $\mathcal{B}^{20}$ Base (2026 nominal) total taxable income thresholds above which the 15% and 20% long-term capital gains (LTCG) tax rates apply, respectively. LTCG is taxed at 0% when total taxable income (ordinary + LTCG) is below $\bar{\mathcal{B}}^{15}_n$, at 15% between $\bar{\mathcal{B}}^{15}_n$ and $\bar{\mathcal{B}}^{20}_n$, and at 20% above $\bar{\mathcal{B}}^{20}_n$, where $\bar{\mathcal{B}}^{15}_n = \gamma_n \mathcal{B}^{15}$ and $\bar{\mathcal{B}}^{20}_n = \gamma_n \mathcal{B}^{20}$ are the inflation-adjusted thresholds. In 2026, $(\mathcal{B}^{15}, \mathcal{B}^{20})$ = (\$49,450, \$545,500) for single filers and (\$98,900, \$613,700) for married filing jointly. Filing status changes at $n = n_d$ for couples. Used in the LTCG bracket constraints; see Eqs. (4.35)–(4.36).

α_{ijkn} Desired asset allocation for savings account j of individual i in asset class k during year n. Allocation ratios come in many flavors as they could be specified globally between individuals and accounts as α_{kn}, for example. When specified by the user, allocation ratios are given two values, one at the beginning of the plan α_{ijk0} and the other at the end $\alpha_{ijkN_{n-1}}$, or α_{ijkn_d} for a spouse passing before the other. Then, intermediate values are interpolated either using a linear relation,

$$\alpha_{ijkn} = a + \frac{n}{N-1}(b-a), \tag{A.15}$$

where N is either n_d or N_n, or using an s-curve as in

$$\alpha_{ijkn} = a + \frac{(b-a)}{2}(\tanh((n-n_1)/n_2) + 1), \tag{A.16}$$

where n_1 is the number of years ahead when the inflection point will occur, and n_2 is the width (in years) of the transition. Constants n_1 and n_2 can be adjusted by the user. Default values are $n_1 = 15$, and $n_2 = 5$, meaning that the transition center will occur in 15 years, taking place from $15 - 5$ years to $15 + 5$ years from now. Using $a = \alpha_{ijk0}$ and $b = \alpha_{ijk(N-1)}$ is an approximation as values of ± 1 are only reached at $\pm\infty$ for a hyperbolic tangent. More precise bounds a' and b' for matching the desired start and end values can be determined by solving a 2×2 system of equations leading

to

$$\begin{aligned} a' &= (a - k_{12}b')/k_{11} \\ b' &= (b - (k_{21}/k_{11})a)/(k_{22} - (k_{21}/k_{11})k_{12}), \end{aligned} \tag{A.17}$$

where

$$\begin{aligned} k_{11} &= \frac{1}{2}(1 + \tanh(n_1/n_2)) \\ k_{12} &= \frac{1}{2}(1 - \tanh(n_1/n_2)) \\ k_{21} &= \frac{1}{2}(1 - \tanh((N-1-n_1)/n_2)) \\ k_{22} &= \frac{1}{2}(1 + \tanh((N-1-n_1)/n_2)). \end{aligned} \tag{A.18}$$

These interpolation functions allow the allocation ratios to gradually change or *glide* during retirement. Fig. A.2 provides an example of an *s-curve* gliding allocation ratios.

It is also possible to have a coarser granularity on the portfolio by having an asset allocation scheme defined on a sum of accounts. For example, allocation can be coordinated between accounts leading to α_{ikn}, or even between spouses as α_{kn}. For any of these cases, it is assumed that weights are always properly scaled so that

$$\begin{aligned} \sum_k \alpha_{ijkn} &= 1, \\ \text{or} \quad \sum_k \alpha_{ikn} &= 1, \\ \text{or} \quad \sum_k \alpha_{kn} &= 1, \end{aligned} \tag{A.19}$$

depending on the scheme selected.

$\mathcal{T}_{ijn}$ When the allocation ratios α_{ijkn} are prescribed, it is sometimes more convenient to express the return rates as

$$\mathcal{T}_{ijn} = \sum_k \alpha_{ijkn}\tau_{kn}. \tag{A.20}$$

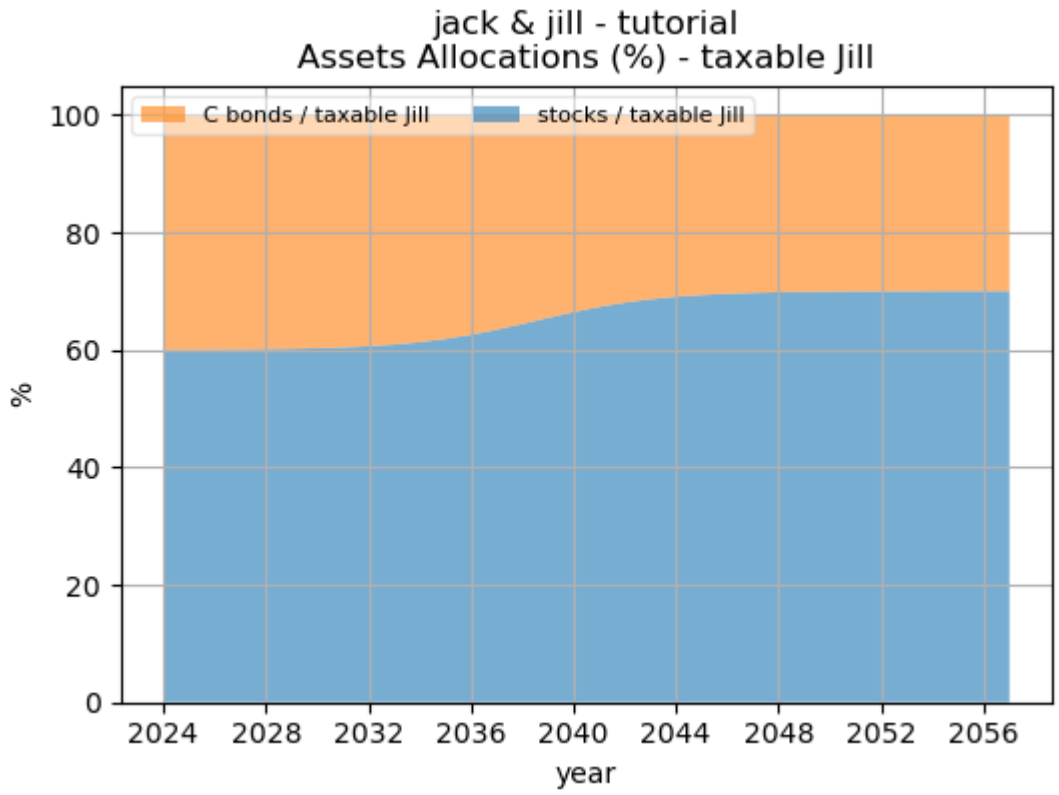


Figure A.2: Example of an allocation portfolio with 60/40% stocks/bonds transitioning to 70/30% using an s-curve.

$\Lambda_{in}^{\pm}$ Big-ticket item requested by individual i in year n. These are large expenses or influx of money that can be planned. Therefore, $\Lambda^{\pm}$ can be positive (e.g., gift received, inheritance) or negative (e.g., buy a house, large gifts).

Ξ_{in} Qualified charitable distribution (QCD) made by individual i in year n: a transfer paid directly from the tax-deferred account to a charity. Unlike $\Lambda_{in}^{\pm}$, it is a parameter of the tax-deferred account rather than of the cash flow. It is excluded from adjusted gross income, counts toward the required minimum distribution, and never becomes spendable cash (Section 4.1.2). $\Xi_{in} \geq 0$.

λ Allowed deviation from the desired net spending profile during one year. Parameter λ can be better understood as a percentage. If $\lambda = 0.10$, then the net spending amount is allowed to vary by up to 10% from the prescribed

profile. This parameter is mainly provided for educational purposes.

π_{in} Sum of pension benefits for individual i in year n. These amounts are typically specified along with the ages at which these benefits begin. Pensions can optionally be indexed for inflation and then represented as $\bar{\pi}_{in}$.

$\bar{a}_{in}$ SPIA income of individual i in year n, in nominal dollars (Section 2.6). It is ordinary income like $\bar{\pi}_{in}$, but kept apart from it because a state pension exemption does not reach it.

ζ_{in} Social Security benefits for individual i in year n. Starting age and the passing of one individual for spouses will determine the time series. $\bar{\zeta}_{in}$ is the same series adjusted for inflation.

$\mathcal{P}_n^{\text{lo}}$, $\mathcal{P}_n^{\text{hi}}$, $\Delta\mathcal{P}$ Lower and upper provisional income thresholds for Social Security taxability, following IRS Publication 915. Below $\mathcal{P}_n^{\text{lo}}$ no Social Security is taxable; between $\mathcal{P}_n^{\text{lo}}$ and $\mathcal{P}_n^{\text{hi}}$ up to 50% of benefits enter taxable income; above $\mathcal{P}_n^{\text{hi}}$ up to 85% of benefits are taxable. We define $\Delta\mathcal{P} := \mathcal{P}_n^{\text{hi}} - \mathcal{P}_n^{\text{lo}}$ (the 50%-zone width; \$9,000 for single, \$12,000 for MFJ). For single filers $(\mathcal{P}^{\text{lo}}, \mathcal{P}^{\text{hi}}) =$ (\$25,000, \$34,000); for married filing jointly $(\mathcal{P}^{\text{lo}}, \mathcal{P}^{\text{hi}}) =$ (\$32,000, \$44,000). Unlike income-tax brackets, these thresholds are *not* indexed for inflation; the time subscript n reflects only the change in filing status at $n = n_d$.

$\mathbb{G}^{\text{NIIT}}$ Statutory modified adjusted gross income (MAGI) threshold above which the net investment income tax (NIIT) of 3.8% applies. $\mathbb{G}^{\text{NIIT}}$ is not indexed for inflation. In 2026, $\mathbb{G}^{\text{NIIT}} =$ \$200,000 for single filers and \$250,000 for married filing jointly, with filing status changing at $n = n_d$. See the definition of J_n in the Intermediate Variables section.

ϵ_{N_n} Desired amount to leave as a bequest at the end of the final year of the plan, $N_n - 1$, which is the beginning of year N_n. This amount is the after-tax value of the estate for the heirs expressed in today's dollars. The inflation-adjusted target is $\bar{\epsilon}_{N_n} = \gamma_{N_n}\epsilon_{N_n}$. See parameter ν for the heirs tax rate.

κ_{ijn} Sum of contributions to savings account j made by individual i during year n. We assume that contributions are made at half-year to better represent periodic contributions made throughout the year. In practice, a contribution amount κ_{ijn} is specified, in which case the contribution to each asset class is

$$\kappa_{ijkn} = \alpha_{ijkn}\kappa_{ijn}. \tag{A.21}$$

as savings account balances are assumed to be rebalanced periodically. For the ($j = 3$) HSA, contributions κ_{i3n} are pre-tax: as with the other pre-tax contributions, wages ω_{in} are entered net of these amounts, which excludes them from taxable income, provisional income for Social Security taxability, and MAGI for Medicare IRMAA purposes in year n. HSA contributions stop when individual i enrolls in Medicare (typically at age 65): the year $\mathcal{N}_i^{\text{hsa}}$ of enrollment keeps the months before it, and later years are zero.

ω_{in} Sum of wages earned by individual i during year n, as they enter AGI: net of the pre-tax account contributions (see Section 2.3). Do not confuse wages ω with withdrawals w.

υ_{in} Other ordinary income for individual i in year n, beyond wages, pension, and Social Security. This is a user-supplied time series (e.g., rental income, alimony) loaded from the household financial plan spreadsheet.

ν_{in} Net investment income from rent or trust distributions for individual i in year n. User-supplied time series loaded from the `netinv` column of the household financial plan spreadsheet. It enters cash-flow and taxable-income constraints as ordinary income and is also included in the net investment income total for NIIT computation. Defaults to zero when the column is absent.

$\mathcal{A}_n^*$ Amounts resulting from the proceeds of liquidation of fixed assets. The liquidation of these fixed assets can generate a taxable income, marked with a superscript x as $\mathcal{A}_n^x$, long-term capital gains, marked with superscript c as $\mathcal{A}_n^c$, or tax-free proceeds marked with a superscript f as $\mathcal{A}_n^f$. These are used to capture the sale of a residence or real estate, cashing restricted stocks, or receiving a lump-sum annuity.

$\mathcal{C}_q$ Combined Medicare Part B and Part D cost for IRMAA bracket q of modified adjusted gross income (MAGI) $\mathbb{G}$, including base premiums and income-related monthly adjusted amounts (IRMAA) for both parts. When adjusted for medical prices, this becomes $\bar{\mathcal{C}}_{qn} = \gamma_n^{\text{med}}\mathcal{C}_q$. There are $N_q = 6$ brackets for IRMAA and therefore 5 step adjustments forming a piecewise constant function.

$\mathcal{D}_n$ Debt payment in year n to be considered in the cash flow. This allows the cash flow to account for a mortgage or a car loan, for example.

$\mathcal{F}_n$ Employee payroll tax in year n on the household's earnings: 6.2% of each person's earnings up to the social security wage base ($184,500 in 2026 and $192,300 in 2027, held at the 2027 level in today's dollars), 1.45% of all earnings, and 0.9% of the combined earnings above $250,000 through the year of the first death and $200,000 after (not indexed). Earnings are those the earnings test reads, the anticipated wages plus the year's tax-deferred contributions (Section 2.4). The tax is not deductible, so it is a fixed outflow of the cash flow and enters no income-tax computation. Self-employment tax is not modeled.

$\mathcal{L}_q$ Upper bounds for brackets used to determine Medicare adjustments based on the modified adjusted gross income (MAGI) $\mathbb{G}$. When adjusted for inflation, this becomes $\bar{\mathcal{L}}_{qn} = \gamma_n \mathcal{L}_q$, except for the top threshold, frozen in nominal dollars through 2027. While there are 6 brackets for IRMAA adjustments forming a piecewise constant function, $\mathcal{L}_q$ has $N_q - 1$ distinct thresholds as the last element is an arbitrarily large number bounding the last bracket. See Fig. 4.1 for a visual representation. Note that $\bar{\mathcal{L}}_{qn}$ needs to be adjusted for inflation and marital status, including adjustments due to the passing of one spouse.

$\bar{\mathcal{L}}^{\text{aca}}_{rn}$, $\bar{\mathcal{C}}^{\text{aca}}_{rkn}$, $\bar{\mathcal{I}}^{\text{aca}}_{rkn}$, $\bar{S}^{\text{aca}}_n$ ACA parameters (only when `withACA="optimize"` and $n < n_{\text{aca}}$). $\bar{\mathcal{L}}^{\text{aca}}_{rn}$: bracket thresholds for year n, multiples of the poverty line of the tax household; $N_r = 8$ brackets (7 FPL-based intervals up to 400% FPL, plus one above). $\bar{\mathcal{C}}^{\text{aca}}_{rkn}$, $\bar{\mathcal{I}}^{\text{aca}}_{rkn}$: slope and intercept of the k-th tangent line to the net premium of bracket r ($r = 1, \ldots, 6$), the year's applicable percentage of MAGI times MAGI (Section 4.1.22). $\bar{S}^{\text{aca}}_n$: inflation-adjusted annual benchmark Silver plan (SLCSP) premium; for bracket 7 (above 400% FPL), net cost equals $\bar{S}^{\text{aca}}_n$ (no subsidy).

μ Dividend return rate for equities in taxable accounts. Average is about 1.7% for S&P 500.

ν Heirs' income tax rate to be applied on the tax-deferred portion of the estate. This is not an estate tax but rather the federal income marginal tax rate that heirs would have to pay on inherited tax-deferred accounts.

ϕ_j Fraction of savings account j that is left to surviving spouse i_s as a beneficiary at the death of individual i_d, the first spouse to pass.

η Spousal ratio for surplus deposits, which goes from 0 to 1, as the fraction that goes to the $i = 1$ spouse's account. Therefore, a surplus s_n in year n

will result in a deposit d in the taxable account of individual i as

$$\begin{aligned} d_{0n} &= (1-\eta)s_n \\ d_{1n} &= \eta s_n. \end{aligned} \tag{A.22}$$

This choice is such that we can set a value depending on the surviving individual $\eta = i_s$ for $n \geq n_d$, after the passing of i_d. Default value is $(N_i - 1)/2$, i.e., 0.5 for couples and 0 for single individuals. When the beneficiary of the savings accounts is not the other spouse, i.e., when $\phi_j \neq 1, \forall j$, it is recommended that η be set to i_d so that all surplus gets deposited to i_d's accounts, thus avoiding loopholes when optimizing for the final bequest.

$\mathcal{M}$ Generic name for the constants of the so-called big-$\mathcal{M}$ method used to implement binary constraints. No single constant is used: each row is relaxed by a bound derived for what it gates, written with a subscript or superscript where it is defined. Too large a bound slows branch-and-bound and invites false optima within the solver's integer tolerance; too small a bound silently removes feasible plans. The options `bigMamo`, `bigMaca`, `bigMss`, `bigMltcg`, and `bigMniit` that once set these constants are retired.

$\bar{\mathcal{C}}_n$ Upper bound on any income of year n: a ceiling on the whole portfolio, doubled so that it never binds, plus the year's known flows (wages, other income, net investment income, pensions, annuities, Social Security, windfalls, and fixed-asset income). It sizes the big-$\mathcal{M}$ bounds of the bracket selectors.

A.4 Intermediate variables

We use intermediate variables for conciseness or clarity, but they are ultimately replaced in the final formulation. Intermediate variables are represented in roman uppercase letters, or in double stroke uppercase letters.

G_n Taxable ordinary income in year n. Proceeds from the liquidation of fixed assets taxable as ordinary income, the sum of wages, other ordinary income, pensions, taxable Social Security benefits, all withdrawals from tax-deferred accounts, including Roth conversions, and gains from securities (i.e., all gains except those from the $k = 0$ equities, which are taxed as capital

gains) in the ($j = 0$) taxable account, including contributions κ, minus the standard deduction e_n,

$$\begin{aligned} G_n &= \mathcal{A}^x_n + \sum_i [\omega_{in} + \upsilon_{in} + \nu_{in} + R_n \bar{\zeta}_{in} + \bar{\pi}_{in} + \bar{a}_{in}] - e_n \\ &\quad + \sum_i [w_{i1n} + x_{in}] \\ &\quad + \sum_{i,k\neq 0} [(b_{i0n} - w_{i0n} + d_{in} + 0.5\kappa_{i0n})\alpha_{i0kn} \max(0, \tau_{kn})] \end{aligned} \tag{A.23}$$

Social Security is indexed for inflation. The taxable portion $R_n \sum_i \bar{\zeta}_{in}$ is determined by the self-consistent loop; see the Social Security taxability paragraph in the Constraints section. Pensions can optionally be indexed for inflation. We exclude $k = 0$ to capture only non-equity gains in taxable accounts. Only positive returns are taxable: a loss on bonds or cash does not reduce income. These gains are all taxed as ordinary income. Here, we assume that withdrawals and deposits in the taxable account are taking place at the beginning of the year, while contributions, if any, are taking place in mid-year.

G_n was also already defined in Eq. (A.1) as

$$G_n = \sum_{t=0}^{N_t - 1} f_{tn},$$

and equating both equations links these variables together.

Q_n Qualified dividends and long-term capital gains obtained in year n. They only involve dividends occurring in the taxable savings accounts ($j = 0$) that were obtained from equities ($k = 0$), or sales of stocks due to withdrawals from taxable savings accounts. For simplicity, we assume that all equity sales only generate long-term capital gains and that all dividends are qualified, resulting in

$$Q_n = \sum_i \alpha_{i00n} \left[(b_{i0n} - w_{i0n} + d_{in} + 0.5\kappa_{i0n})\mu + w_{i0n} \max(0, \tau_{0(n-1)} - \mu)\right] + \mathcal{A}^c_n. \tag{A.24}$$

The last term, the long-term capital gains realized on the sale of fixed assets, also enters the LTCG bracket partition. When a cost basis is given

for the taxable account, the gain fraction of a sale, $\max(0, \tau_{0(n-1)} - \mu)$ above, is replaced by the fraction of the account that is unrealized gain, tracked from that basis and updated by the self-consistent loop (Section 3.6). A formulation where only a fraction of dividends are qualified can easily be implemented with the addition of another parameter. Notice that we are using return rates from the previous year. The first terms on the right-hand side represent dividends generated by equities ($k = 0$) in the ($j = 0$) taxable savings account plus half the yearly contributions. The second term accounts for withdrawals w of equities assumed to have been purchased a year ago. Capital gains are calculated as price appreciation only (total return minus dividend rate) to avoid double taxation of dividends, which are already included in the first term. It does not account for losses, but a market drop would most likely result in stock purchase rather than sale. For withdrawals, we make the assumption of selling the most recent stocks which would not be accurate in situations where the taxable savings account is being depleted slowly. An implementation keeping track of stock purchases and sales is beyond the scope of providing a guide for retirement decisions.

When an initial cost basis K_{i0} is provided for individual i via `setCostBasis()`, the capital-gain coefficient on withdrawals is generalized to a tracked gain fraction ψ_{in}, giving

$$Q_n = \sum_i \alpha_{i00n} \left[(b_{i0n} - w_{i0n} + d_{in} + \tfrac{1}{2}\kappa_{i0n})\mu + \psi_{in}\, w_{i0n} \right], \tag{A.25}$$

where

$$\psi_{in} = \min\left(1,\ \frac{1}{\alpha_{i00n}} \max\left(0,\ 1 - \frac{K_{in}}{b_{i0n}}\right)\right) \tag{A.26}$$

is the fraction of the equity holdings that represents unrealized gain. Bond and cash returns are taxed every year, so those holdings sit at their basis and the account's whole unrealized gain $b_{i0n} - K_{in}$ is carried by the equity share $\alpha_{i00n} b_{i0n}$; the realized gain $\alpha_{i00n}\psi_{in} w_{i0n}$ is then $(1 - K_{in}/b_{i0n})\, w_{i0n}$. Equation (A.24) is recovered when $\psi_{in} = \max(0, \tau_{0(n-1)} - \mu)$, the fallback used in the absence of a known basis. The basis evolves between self-consistent iterations via the pro-rata (average-cost) rule

$$K_{i,n+1} = K_{in}\left(1 - \frac{w_{i0n}}{b_{i0n}}\right) + \kappa_{i0n} + d_{in} + \Big(\alpha_{i00n}\mu + \sum_{k\geq 1} \alpha_{i0kn} \max(0, \tau_{kn})\Big)\big(b_{i0n} - w_{i0n} + d_{in} + \tfrac{1}{2}\kappa_{i0n}\big),$$

(A.27)

initialized at K_{i0} from `setCostBasis()`. Each withdrawal reduces the total basis in proportion to the account fraction liquidated; fixed contributions κ_{i0n} and surplus deposits d_{in} are new purchases added at full cost (zero embedded gain). The last term is the year's dividends and interest: they are taxed as they are earned and stay in the account, so they are reinvested at full cost too; leaving them out would tax them a second time on sale.

$\mathbb{G}_n$ Modified adjusted gross income (MAGI) for year n, on the adjusted-gross-income (AGI) basis. This is the canonical tax MAGI (AGI plus tax-exempt interest) used for IRMAA, the Net Investment Income Tax, and the OBBBA 65+ senior-deduction phaseout. Ignoring non-taxable municipal interest,

$$\mathbb{G}_n = G_n + Q_n + e_n, \tag{A.28}$$

where $G_n + e_n$ already contains *only* the taxable Social Security term $R_n \sum_i \bar{\zeta}_{in}$ (the non-taxable remainder is not part of AGI).

$\mathbb{G}_n^{\text{aca}}$ Full-Social-Security MAGI variant. The Affordable Care Act premium credit (IRC §36B) adds the non-taxable portion of Social Security back to AGI, and the Social Security provisional-income formula likewise uses the full benefit:

$$\mathbb{G}_n^{\text{aca}} = \mathbb{G}_n + (1 - R_n) \sum_i \bar{\zeta}_{in}. \tag{A.29}$$

This approach ignores additional IRS rules around tax-free interests which are insignificant in most cases.

Π_n Provisional income for Social Security taxability in year n (IRS Publication 915): $\Pi_n = \mathbb{G}_n^{\text{aca}} - \frac{1}{2} \sum_i \bar{\zeta}_{in}$. Used to determine the taxable portion of Social Security; see Eq. (4.54) and the Social Security taxability paragraph in the Constraints section.

R_n Effective fraction of Social Security benefits subject to income tax in year n, bounded in $[0, 0.85]$. It is computed by the self-consistent loop from the IRS provisional income formula (IRS Publication 915) and enters the LP as a fixed parameter each iteration. $R_n = \mathcal{S}_n / \sum_i \bar{\zeta}_{in}$; see the Social Security taxability paragraph in the Constraints section.

$\mathcal{S}_n$ Amount of Social Security benefits subject to federal income tax in year n. $\mathcal{S}_n$ is computed from the IRS formula in Eqs. (4.55)–(4.58), and $R_n = \mathcal{S}_n / \sum_i \bar{\zeta}_{in}$.

$\mathbb{I}_n$ Interest and dividend income from the taxable account, plus rent and trust income from the `netinv` column. Used as the base for NIIT computation (together with Q_n).

$$\mathbb{I}_n = \sum_{i,k\neq 0} [(b_{i0n} - w_{i0n} + d_{in} + 0.5\kappa_{i0n})\alpha_{i0kn} \max(0, \tau_{kn})] + \sum_i \nu_{in}. \quad \text{(A.30)}$$

Only positive returns are taxable; each asset's return is clamped to zero separately, as in Eq. (A.23).

P_n Amount of 10% early withdrawal penalty in year n. The penalty applies only to tax-deferred ($j = 1$) withdrawals before the IRS threshold of age $59\frac{1}{2}$:

$$P_n = 0.10 \sum_i (1 - \mathcal{H}(n - n_{i,59\frac{1}{2}}))w_{i1n}. \quad \text{(A.31)}$$

Here, $\mathcal{H}(n - n_{i,59\frac{1}{2}})$ is a Heaviside step function which is 0 or 1, depending on the sign of its argument:

$$\mathcal{H}(x) := \begin{cases} 0 & x < 0 \\ 1 & x \geq 0. \end{cases} \quad \text{(A.32)}$$

The parameter $n_{i,59\frac{1}{2}}$ is the year index when individual i turns $59\frac{1}{2}$, or 0 if the individual is already past $59\frac{1}{2}$ at the beginning of the plan. Because the model operates at yearly granularity:

$$n_{i,59\frac{1}{2}} = \max\big(0,\ 59 - Y_0 + y_{\text{obs},i} + \mathbf{1}[m_{\text{obs},i} > 6]\big), \quad \text{(A.33)}$$

where Y_0 is the current calendar year, $y_{\text{obs},i}$ is the birth year of individual i, and $m_{\text{obs},i} \in \{1, \ldots, 12\}$ is the birth month. The indicator $\mathbf{1}[m > 6]$ adds one year for individuals born in July–December, whose $59\frac{1}{2}$ birthday falls in January–June of the following year. Tax-free ($j = 2$) Roth withdrawals are not subject to this penalty because the maturation and early-basis constraints already restrict withdrawals to penalty-free amounts (Section 4.1.12).

T_n Amount of income tax paid on taxable ordinary income G_n in year n. This is the taxes paid on ordinary income expressed as the sum of the amounts paid in each tax bracket as

$$T_n = \sum_{t=0}^{N_t-1} f_{tn}\theta^x_{tn}. \tag{A.34}$$

Notice how f_{tn} also defines G_n in Eq. (A.1), and that optimal values of f_{tn} have to minimize T_n regardless of whether the bequest or the desired net spending is being maximized.

U_n Amount of income tax paid on long-term capital gains (LTCG) and qualified dividends in year n. The LTCG tax is computed exactly by the LP bracket variables q_{pn}:

$$U_n = 0.15\, q_{1n} + 0.20\, q_{2n}. \tag{A.35}$$

We assume that qualified dividends and long-term capital gains are taxed at the same preferential rate, which is the case for most situations.

J_n Net investment income tax (NIIT) paid in year n. The NIIT is 3.8% and applies when modified adjusted gross income $\mathbb{G}_n$ exceeds the statutory threshold $\mathbb{G}^{\text{NIIT}}$ (formally defined in the Parameters section; not indexed for inflation). We compute

$$J_n = 0.038\ \max\left(0,\ \min\left(\mathbb{G}_n - \mathbb{G}^{\text{NIIT}},\ \mathbb{I}_n + Q_n\right)\right), \tag{A.36}$$

where $\mathbb{G}^{\text{NIIT}}$ is \$200k for single filers and \$250k for married filing jointly, with filing status changing after the passing of one spouse ($n \geq n_d$). In the default SC-loop mode J_n is a derived quantity, recomputed after each LP solve from the current $\mathbb{G}_n$ and held fixed during the solve. When `withNIIT="optimize"` is selected, J_n and $\mathbb{G}_n$ become LP decision variables, and a binary variable z^j_n selects which term of the IRS minimum bounds the tax (see the Net Investment Income Tax subsection of the Constraints chapter).

K_{in} Average-cost basis of the taxable account for individual i at the start of year n. Initialized from the user-supplied value K_{i0} via `setCostBasis()` and updated each self-consistent iteration by Eq. (A.27). Not used when no cost basis is provided.

B. Mapping of decision variables

At this point, one can use one of the many algebraic modeling languages such as AMPL, GAMS, MOSEK, AIMMS, and Gurobi, and code the equations above using that language, but most of these applications are proprietary and require a license and additional software installation. These languages allow the problem to be stated at a high level and steps to cast the problem in a form suitable for solution are performed automatically. There are also object-oriented language extensions, such as Python's Pyomo, that can ease the process of solving these problems. For completeness, however, we present here a simple index mapping approach that allows solving this problem using a generic linear programming solver.

Using a simple interface for mapping sparse objects to dense ones, the approach described here has been successfully tested with both the HiGHS open-source solver and the MOSEK proprietary solver. To cast the problem in a form suitable for a linear programming solver, we will use a single block vector represented by the array $y[q()]$ with index-mapping functions $q()$. While this process can be achieved using slicing and reshaping in some programming languages, we will present a generic approach suitable for most programming languages. The detailed approach presented here also allows us to determine the size of the problem to solve. We proceed alphabetically for all variables, and continue to use the convention of having index 0 for representing the first element.

To bring all variables in a single block vector, we will simply use two generic index mapping functions defined as

$$q_*(C, \ell_1, \ell_2, \ell_3; N_1, N_2, N_3) := C + \ell_1 N_2 N_3 + \ell_2 N_3 + \ell_3, \tag{B.1}$$

and

$$q_C(C, N_1, N_2, N_3) := C + N_1 N_2 N_3, \tag{B.2}$$

with the constraint that $0 \leq \ell_i < N_i$.

Account balances (b) For storing the savings account balances appropriately, variable b_{ijn} needs to have one more entry $(N_n + 1)$ to store the end-of-life estate value. Therefore, we use

$$y[q_b(i, j, n)] = b_{ijn}, \tag{B.3}$$

where

$$q_b(i, j, n) = q_*(C_b, i, j, n; N_i, N_j, N_n + 1), \tag{B.4}$$

and where n exceptionally runs from 0 to N_n inclusively, and therefore q_b runs from $C_b = 0$ to $C_d - 1$, where

$$C_d = q_C(C_b, N_i, N_j, N_n + 1) = N_i N_j (N_n + 1).$$

Surplus deposits (d) For surplus deposits in the taxable savings accounts d_{in} we will use

$$y[q_d(i, n)] = d_{in}, \tag{B.5}$$

where

$$q_d(i, n) = q_*(C_d, i, n, 0; N_i, N_n, 1), \tag{B.6}$$

with q_d running from C_d to $C_e - 1$, where

$$C_e = q_C(C_d, N_i, N_n, 1) = N_i(N_j(N_n + 1) + N_n).$$

Standard exemption (e) For the standard exemption e_n we will use

$$y[q_e(n)] = e_n, \tag{B.7}$$

where

$$q_e(n) = q_*(C_e, n, 0, 0; N_n, 1, 1) = C_e + n, \tag{B.8}$$

with q_e running from C_e to $C_f - 1$, where

$$C_f = q_C(C_e, N_n, 1, 1) = N_i(N_j(N_n + 1) + N_n) + N_n.$$

Tax bracket amounts (f) For tax bracket amounts f_{tn} we will use

$$y[q_f(t,n)] = f_{tn}, \tag{B.9}$$

where

$$q_f(t,n) = q_*(C_f, t, n, 0; N_t, N_n, 1) \tag{B.10}$$

with q_f running from C_f to $C_g - 1$, where

$$C_g = q_C(C_f, N_t, N_n, 1) = N_i(N_j(N_n + 1) + N_n) + (N_t + 1)N_n.$$

Net spending (g) For net spending g_n we will use

$$y[q_g(n)] = g_n, \tag{B.11}$$

where

$$q_g(n) = q_*(C_g, n, 0, 0; N_n, 1, 1) = C_g + n, \tag{B.12}$$

with q_g running from C_g to $C_h - 1$, where

$$C_h = q_C(C_g, N_n, 1, 1) = N_i(N_j(N_n + 1) + N_n) + (N_t + 2)N_n.$$

MAGI bracket portions (h) This block is present only when `withMedicare="optimize"`; otherwise $N_q(N_n - n_m)$ is to be read as zero in all the offsets that follow. For h_{qn} we will use

$$y[q_h(q,n)] = h_{qn}, \tag{B.13}$$

where

$$q_h(q,n) = q_*(C_h, q, n, 0; N_q, N_n - n_m, 1), \tag{B.14}$$

with q_h running from C_h to $C_m - 1$, where

$$C_m = q_C(C_h, N_q, N_n - n_m, 1) = N_i(N_j(N_n + 1) + N_n) + (N_t + 2)N_n + N_q(N_n - n_m).$$

When `withACA="optimize"`, the ACA bracket portions h^{aca}_{rn} ($n_{\text{aca}}N_r$ entries) and costs m^{aca}_n (N_n entries) follow h and shift all later offsets by $n_{\text{aca}}N_r + N_n$, which we omit below.

Medicare costs ($\boldsymbol{m}$) For Medicare costs m_n we will use

$$y[q_m(n)] = m_n, \tag{B.15}$$

where

$$q_m(n) = q_*(C_m, n, 0, 0; N_n, 1, 1) = C_m + n, \tag{B.16}$$

with q_m running from C_m to $C_{s_0} - 1$, where

$$C_{s_0} = q_C(C_m, N_n, 1, 1) = N_i(N_j(N_n + 1) + N_n) + (N_t + 3)N_n + N_q(N_n - n_m).$$

LTCG bracket allocations ($\boldsymbol{q}$) For LTCG bracket allocation variables q_{pn} we will use

$$y[q_q(p, n)] = q_{pn}, \tag{B.17}$$

where

$$q_q(p, n) = q_*(C_q, p, n, 0; N_p, N_n, 1), \tag{B.18}$$

with q_q running from C_q to $C_s - 1$, where $C_q = C_{s_0}$ (the cumulative index after Medicare) and

$$C_s = q_C(C_q, N_p, N_n, 1) = N_i(N_j(N_n+1)+N_n)+(N_t+3+N_p)N_n+N_q(N_n-n_m).$$

Surplus ($\boldsymbol{s}$) Surplus can be generated if big-ticket items are received (inheritance, sale of a house, etc.) or due to RMDs. Surplus s is then deposited to taxable savings accounts according to variable η. We will use

$$y[q_s(n)] = s_n, \tag{B.19}$$

where

$$q_s(n) = q_*(C_s, n, 0, 0; N_n, 1, 1) = C_s + n, \tag{B.20}$$

with q_s running from C_s to $C_w - 1$, where

$$C_w = q_C(C_s, N_n, 1, 1) = N_i(N_j(N_n + 1) + N_n) + (N_t + 4 + N_p)N_n + N_q(N_n - n_m).$$

Withdrawals ($\boldsymbol{w}$) For withdrawals w_{ijn} we will use

$$y[q_w(i,j,n)] = w_{ijn}, \tag{B.21}$$

where

$$q_w(i,j,n) = q_*(C_w, i, j, n; N_i, N_j, N_n), \tag{B.22}$$

with q_w running from C_w to $C_x - 1$, where

$$C_x = q_C(C_w, N_i, N_j, N_n) = N_i\big(N_j(2N_n+1)+N_n\big)+(N_t+4+N_p)N_n+N_q(N_n-n_m).$$

Roth conversions ($\boldsymbol{x}$) Finally, for Roth conversions x_{in} we will use

$$y[q_x(i,n)] = x_{in}, \tag{B.23}$$

where

$$q_x(i,n) = q_*(C_x, i, n, 0; N_i, N_n, 1), \tag{B.24}$$

with q_x running from C_x to $C_* - 1$, where

$$C_* = q_C(C_x, N_i, N_n, 1) = N_i\big(N_j(2N_n+1)+2N_n\big)+(N_t+4+N_p)N_n+N_q(N_n-n_m). \tag{B.25}$$

With $N_i = 2$, $N_j = 4$, $N_t = 7$, $N_p = 3$, Eq. (B.25) yields $C_* = 34N_n+8+N_q(N_n-n_m)$ entries in y, of which $N_iN_j(N_n+1) = 8N_n+8$ are balances and $N_pN_n = 3N_n$ are the LTCG bracket allocations q_{pn}. With the default options, h_{qn} is absent and the problem is a pure LP of $34N_n + 8$ continuous variables, e.g., 1130 for a couple over 33 years. If the time resolution is increased to months, the number of variables scales accordingly.

The options switching a constraint into the optimization (Section 3.9) and some case features append further continuous blocks after x, all before any binary variable:

- $p_n^{\sigma,\mathrm{lo}}$, $p_n^{\sigma,\mathrm{hi}}$, $p_n^{\sigma,\mathrm{min}}$, and t_n^σ, $4N_n$ in all (`withSSTaxability="optimize"`);
- G_n, N_n (`withLTCG="optimize"`);
- $\mathbb{G}_n$, N_n (`withNIIT` or `withSeniorBonus="optimize"`);
- J_n, N_n (`withNIIT="optimize"`);

- the senior-bonus reductions p_n, one per year in which somebody receives the bonus, N_{sb} in all (`withSeniorBonus="optimize"`);
- the state bracket allocations f^s_{tn} and standard-deduction headroom e^s_n, $(N_s + 1)N_n$ (states with an income tax), the retirement-income exclusions r^s_{in}, $N_i N_n$ (states with one), and the personal credit used c^s_n, N_n (states with one);
- the federal and state parts of a deduction that income cannot absorb, l^f_n and l^s_n, $2N_n$ (cases with a negative other income).

The binary variables come last, so that y splits into a continuous and an integer part: $N_q(N_n - n_m)$ for z^m_{qn} (`withMedicare="optimize"`), $2N_n$ for $z^\sigma_{0n}, z^\sigma_{1n}$ (`withSSTaxability="optimize"`), $n_{\mathrm{aca}}N_r$ for z^{aca}_{rn} (`withACA="optimize"`), $2N_n$ for z^l_{0n}, z^l_{1n} (`withLTCG="optimize"`), N_n for z^j_n (`withNIIT="optimize"`), $2N_n$ for z^o_{0n}, z^o_{1n} (`withdrawalOrder="taxable_first"`), and N_{sb} for z^{sb}_n (`withSeniorBonus="optimize"`). Without any of these options, there are no binary variables.

B.1 Reverse mapping of indices

Values are recovered from the solution by slicing each block, $y[C : C + N_1N_2N_3]$, and reshaping it in row-major order to its index shape, as `VarBlock.extract` does in `varmap.py`, so no explicit inverse of the mapping functions is needed.

C. Building constraint matrices

Let's first define generic index-mapping functions I and J as

$$\begin{aligned} I_l(n) &= C_l + n, \\ I_l(i, n; N_n) &= C_l + iN_n + n, \\ I_l(i, j, n; N_j, N_n) &= C_l + iN_jN_n + jN_n + n, \\ \dots &= \dots \end{aligned} \tag{C.1}$$

and so on, which would cumulatively increase row count C_l at each new instance l, similar to how we proceeded in the previous section. This allows us to build rectangular matrices by iteratively adding rows. These constraint matrices have C_* (defined in Eq. (B.25)) columns but can have fewer rows, forming an underdetermined system to be optimized using linear programming. Function J is defined similarly for equality constraints, while I is used for building the rows of the matrix containing the inequality constraints. Additional indices found in columns and not in rows imply the existence of multiple elements on that row.

C.1 Inequality constraints

Inequality constraints can be upper or lower bounds expressed in matrix form as $\ell \leq A_u y \leq u$. When ℓ is not specified it is assumed 0, and an unspecified u implies ∞. Most if not all inequalities will be expressed as upper bounds.

Required minimum distributions (RMDs) We rewrite the inequality constraint on required minimum distributions Eq. (4.5) using matrix $A_u y \leq u$ starting

with the following $N_i N_n$ rows,

$$\begin{aligned} A_u[I_0(i,n), q_w(i,1,n)] &= -1 \\ A_u[I_0(i,n), q_b(i,1,n)] &= \rho_{in}, \\ u[I_0(i,n)] &= \Xi_{in}, \qquad \forall i \in \{0,\ldots,N_i-1\}, \\ &\qquad\qquad \forall n \in \{0,\ldots,N_n-1\}, \end{aligned} \tag{C.2}$$

and all other elements in the same rows of A_u being 0. Notice that while b has N_n+1 elements, the constraints for b go from 0 to N_n-1 as there is no RMD required in the last year of the plan N_n, when all individuals have passed. Years before RMDs where $\rho_{in}=0$ just add a trivial constraint. An individual without a tax-deferred balance at the start gets no RMD rows. See Eq. (B.4).

Ordinary income tax brackets Similarly, we add $N_t N_n$ more rows to matrix $A_u y \le u$ to express the inequality constraint in Eq. (4.6) setting an upper limit on amounts f_{tn},

$$\begin{aligned} A_u[I_1(t,n), q_f(t,n)] &= 1, \\ u[I_1(t,n)] &= \bar{\Delta}_{tn}, \qquad \forall t \in \{0,\ldots,N_t-1\}, \\ &\qquad\qquad \forall n \in \{0,\ldots,N_n-1\}, \end{aligned} \tag{C.3}$$

and all other elements in the same rows of A_u being 0.

Standard exemption For $e_n \le \bar{\sigma}_n$ we add

$$\begin{aligned} A_u[I_2(n), q_e(n)] &= 1, \\ u[I_2(n)] &= \bar{\sigma}_n, \qquad \forall n \in \{0,\ldots,N_n-1\}. \end{aligned} \tag{C.4}$$

Non-negativity of e_n provides the lower bound.

Withdrawal limits For $w_{ijn} + \delta_{j,1}(x_{in} + \Xi_{in}) \leq b_{ijn}$, where a QCD consumes the capacity of the account it is drawn from, we add:

$$\begin{aligned} A_u[I_3(i,j,n), q_w(i,j,n)] &= 1, \\ A_u[I_3(i,j,n), q_x(i,n)] &= \delta_{j,1}, \\ A_u[I_3(i,j,n), q_b(i,j,n)] &= -1, \\ u[I_3(i,j,n)] &= -\delta_{j,1}\Xi_{in}, \\ &\qquad \forall i \in \{0,\dots,N_i-1\}, \\ &\qquad \forall j \in \{0,\dots,N_j-1\}, \\ &\qquad \forall n \in \{0,\dots,N_n-1\} \ (n < n_d \text{ for } i = i_d). \end{aligned}$$

For a couple, the upper index for the first-to-pass individual i_d is $n_d - 1$ rather than $N_n - 1$, since posthumous constraints zero out all account activities.

Posthumous account activities For $n \geq n_d$ and the first-to-pass individual i_d, we set

$$\begin{aligned} A_u[I_4(j,n), q_w(i_d,j,n)] &= 1, \\ u[I_4(j,n)] &= 0, \\ A_u[I_5(n), q_d(i_d,n)] &= 1, \\ u[I_5(n)] &= 0, \\ A_u[I_6(n), q_x(i_d,n)] &= 1, \\ u[I_6(n)] &= 0, \\ &\qquad \forall j \in \{0,\dots,N_j-1\}, \\ &\qquad \forall n \in \{n_d,\dots,N_n-1\}. \end{aligned}$$

Roth conversion limit If a maximum conversion $x_{max} \geq 0$ is specified, the bound $0 \leq x_{in} \leq x_{max}$ is imposed directly on each variable rather than through a row of A_u,

$$\begin{aligned} 0 \leq y[q_x(i,n)] \leq x_{max}, \qquad & \forall i \in \{0,\dots,N_i-1\}, \\ & \forall n \in \{0,\dots,N_n-1\}. \end{aligned}$$

This maximum is applied to each individual's conversions in each year, not to their sum. The individual named by `noRothConversions` has x_{in} pinned to zero instead, as are the years before `startRothConversions` and from `stopRothConversions` on.

Medicare brackets Medicare's MAGI brackets use binary variables z^m_{qn} and the disaggregated MAGI variables h_{qn}. We set upper bounds for each bracket using Eq. (4.66) as

$$\begin{aligned} A_u[I_{15}(q,n), q_h(q,n)] &= 1, \\ A_u[I_{15}(q,n), q_{z^m}(q,n)] &= -\bar{\mathcal{L}}_{qn}, \\ u[I_{15}(q,n)] &= 0, \\ &\qquad \forall q \in \{0,\ldots,N_q-2\}, \\ &\qquad \forall n \in \{n_m,\ldots,N_n-1\}, \end{aligned} \tag{C.5}$$

the upper bound for the last bracket ($q = N_q - 1$) using the lookback year's income ceiling as

$$\begin{aligned} A_u[I_{15}(N_q-1,n), q_h(N_q-1,n)] &= 1, \\ A_u[I_{15}(N_q-1,n), q_{z^m}(N_q-1,n)] &= -\bar{\mathcal{C}}_{(n-2)}, \\ u[I_{15}(N_q-1,n)] &= 0, \\ &\qquad \forall n \in \{n_m,\ldots,N_n-1\}, \end{aligned} \tag{C.6}$$

and the lower bounds as

$$\begin{aligned} A_u[I_{16}(q,n), q_h(q,n)] &= -1, \\ A_u[I_{16}(q,n), q_{z^m}(q,n)] &= \bar{\mathcal{L}}_{(q-1)n}, \\ u[I_{16}(q,n)] &= 0, \\ &\qquad \forall q \in \{1,\ldots,N_q-1\}, \\ &\qquad \forall n \in \{n_m,\ldots,N_n-1\}. \end{aligned} \tag{C.7}$$

C.2 Equality constraints

Account balances For the equality constraints on account balances expressed in Eq. (4.28), we define an equality constraint matrix $A_e y = v$ starting with $N_i N_j N_n$

rows as

$$\begin{aligned}
A_e[J_0(i,j,n), q_b(i,j,n+1)] &= 1, \\
A_e[J_0(i,j,n), q_b(i,j,n)] &= -(1+\mathcal{T}_{ijn}), \\
A_e[J_0(i,j,n), q_x(i,n)] &= -(x_{\text{net}}\delta_{j,2} - \delta_{j,1})(1+\mathcal{T}_{ijn}), \\
A_e[J_0(i,j,n), q_w(i,j,n)] &= (1+\mathcal{T}_{ijn}), \\
A_e[J_0(i,j,n), q_d(i,n)] &= -\delta_{j,0}(1+\mathcal{T}_{ijn}), \\
&\qquad \forall i \in \{0,\ldots,N_i-1\}, \\
&\qquad \forall j \in \{0,\ldots,N_j-1\}, \\
&\qquad \forall n \in \{0,\ldots,N_n-1\},
\end{aligned} \tag{C.8}$$

where v is

$$v[J_0(i,j,n)] = \kappa_{ijn}(1+\mathcal{T}_{ijn}/2) - \delta_{j,1}\big[\Xi_{in}(1+\mathcal{T}_{ijn}) + \Pi^a_{in}\big]. \tag{C.9}$$

Here $x_{\text{net}} = 1 - \text{oppCostX}/100$ is the fraction of a converted dollar that arrives in the Roth account (one by default), and Π^a_{in} is a SPIA premium paid from the tax-deferred account in its purchase year (Section 2.6). The initial account balances expressed in Eq. (4.20) are imposed through

$$\begin{aligned}
A_e[J_1(i,j), q_b(i,j,0)] &= 1, \\
v[J_1(i,j)] &= \beta_{ij}, \qquad\qquad (\text{C.10}) \\
&\qquad \forall i \in \{0,\ldots,N_i-1\}, \\
&\qquad \forall j \in \{0,\ldots,N_j-1\},
\end{aligned} \tag{C.11}$$

leading to N_iN_j additional rows to A_e.

Medicare bracket selection The SOS1 selection of Eq. (4.64) is encoded as

$$\begin{aligned}
A_e[J_2(n), q_{z^m}(q,n)] &= 1, \\
v[J_2(n)] &= 1, \\
&\qquad \forall q \in \{0,\ldots,N_q-1\}, \\
&\qquad \forall n \in \{n_m,\ldots,N_n-1\}.
\end{aligned} \tag{C.12}$$

MAGI disaggregation Using Eqs. (A.23) and (A.24), the equality $\sum_q h_{qn} = \mathbb{G}_{(n-2)}$ is expanded and implemented as

$$
\begin{aligned}
A_e[J_3(n), q_h(q,n)] &= 1, \\
A_e[J_3(n), q_w(i,1,n-2)] &= -1, \\
A_e[J_3(n), q_x(i,n-2)] &= -1, \\
A_e[J_3(n), q_b(i,0,n-2)] &= -\mu\alpha_{i00(n-2)} \\
&\quad - \sum_{k\neq 0} \alpha_{i0k(n-2)} \max(0, \tau_{k(n-2)}), \\
A_e[J_3(n), q_d(i,n-2)] &= -\mu\alpha_{i00(n-2)} \\
&\quad - \sum_{k\neq 0} \alpha_{i0k(n-2)} \max(0, \tau_{k(n-2)}), \\
A_e[J_3(n), q_w(i,0,n-2)] &= \mu\alpha_{i00(n-2)} + \sum_{k\neq 0} \alpha_{i0k(n-2)} \max(0, \tau_{k(n-2)}) \\
&\quad - \alpha_{i00(n-2)} \max(0, \tau_{0(n-3)} - \mu), \\
v[J_3(n)] &= \mathcal{A}^x_{n-2} + \mathcal{A}^c_{n-2} + \sum_i \left[\omega_{i(n-2)} + \upsilon_{i(n-2)} + \nu_{i(n-2)}\right. \\
&\quad \left. + R_{n-2}\bar{\zeta}_{i(n-2)} + \bar{\pi}_{i(n-2)} + \bar{a}_{i(n-2)}\right] \\
&\quad + 0.5 \sum_i \left[\kappa_{i0(n-2)} \left(\mu\alpha_{i00(n-2)} + \sum_{k\neq 0} \alpha_{i0k(n-2)} \max(0, \tau_{k(n-2)})\right)\right], \\
&\quad \forall i \in \{0, \ldots, N_i - 1\}, \\
&\quad \forall k \in \{0, \ldots, N_k - 1\}, \\
&\quad \forall q \in \{0, \ldots, N_q - 1\}, \\
&\quad \forall n \in \{\max(n_m, 2), \ldots, N_n - 1\}.
\end{aligned}
\tag{C.13}
$$

The terms on the left are income before the standard deduction, as MAGI is, so e_{n-2} does not appear. When a cost basis is tracked, the tracked gain fraction replaces $\max(0, \tau_{0(n-3)} - \mu)$ in the coefficient of $w_{i0(n-2)}$ (see Eq. (A.24)). With `withSSTaxability="optimize"`, $R_{n-2}\sum_i \bar{\zeta}_{i(n-2)}$ leaves v and t^{σ}_{n-2} joins the left-hand side with coefficient -1. When $n_m < 2$, we instead pin the bracket of years $n < 2$ to that of the user-provided MAGI values $\mathbb{G}_{(-1)}$ and $\mathbb{G}_{(-2)}$ for the years before the plan started, by fixing z^m_{qn} directly.

Medicare costs Binary variables z^m_{qn} express the active IRMAA bracket. The costs for Medicare are obtained using Eq. (4.67):

$$\begin{aligned} A_e[J_4(n), q_m(n)] &= 1, \\ A_e[J_4(n), q_{z^m}(q,n)] &= -\bar{\mathcal{C}}_{qn}, \\ v[J_4(n)] &= 0, \\ &\qquad \forall q \in \{0, \ldots, N_q - 1\}, \\ &\qquad \forall n \in \{n_m, \ldots, N_n - 1\}. \end{aligned}$$

Net spending For the equality constraint on net spending expressed in Eq. (4.30), we add N_n more rows to $A_e y = v$ as

$$\begin{aligned} A_e[J_5(n), q_g(n)] &= 1, \\ A_e[J_5(n), q_m(n)] &= 1, \\ A_e[J_5(n), q_s(n)] &= 1, \\ A_e[J_5(n), q_f(t,n)] &= \theta^x_{tn}, \\ A_e[J_5(n), q_w(i,j,n)] &= -1 + 0.1\,\delta_{j,1}\,(1 - \mathcal{H}(n - n_{i,59\frac{1}{2}})), \\ A_e[J_5(n), q_q(1,n)] &= 0.15, \\ A_e[J_5(n), q_q(2,n)] &= 0.20, \\ A_e[J_5(n), q_{f^s}(t,n)] &= \theta^s_{tn}, \\ A_e[J_5(n), q_{c^s}(n)] &= -1, \\ &\qquad \forall t \in \{0, \ldots, N_t - 1\}, \\ &\qquad \forall i \in \{0, \ldots, N_i - 1\}, \\ &\qquad \forall j \in \{0, \ldots, N_j - 1\}, \\ &\qquad \forall n \in \{0, \ldots, N_n - 1\}, \end{aligned}$$

where $q_q(p,n)$ is the flat index of q_{pn} in the solution vector, and v is

$$v[J_5(n)] = \sum_{*=x,c,f} \mathcal{A}^*_n + \sum_i \Lambda^{\pm}_{in} + \sum_i [\omega_{in} + \upsilon_{in} + \nu_{in} + \bar{\zeta}_{in} + \bar{\pi}_{in} + \bar{a}_{in}] - \mathcal{D}_n - \mathcal{F}_n - J_n - \mathcal{M}^{\text{loop}}_n - \mathcal{M}^{\text{aca,loop}}_n. \tag{C.14}$$

The state brackets f^s_{tn} (running over the N_s state brackets) and the credit c^s_n appear only for a state with an income tax (and a credit). The last three terms of v are the NIIT, Medicare, and ACA costs computed by the self-consistent loop; each is zero

when its option is `"optimize"`, where J_n, m_n, and m_n^{aca} (coefficient 1) are variables on the left-hand side instead.

The condition of having a predictable net spending expressed as an equality in Eq. (4.32) adds $N_n - 1$ more rows to $A_e y = v$ as

$$\begin{aligned} A_e[J_6(n), q_g(0)] &= -\bar{\xi}_n, \\ A_e[J_6(n), q_g(n)] &= \xi_0, \\ v[J_6(n)] &= 0, \qquad \forall n \in \{1, \ldots, N_n - 1\}. \end{aligned} \tag{C.15}$$

Taxable ordinary income For the equality constraint in Eq. (4.49) establishing taxable ordinary income, we add N_n rows to $A_e y = v$ as follows. The taxable Social Security term $R_n \sum_i \bar{\zeta}_{in}$ appears on the right-hand side as a parameter updated each SC-loop iteration.

$$\begin{aligned} A_e[J_7(n), q_e(n)] &= 1, \\ A_e[J_7(n), q_f(t,n)] &= 1, \\ A_e[J_7(n), q_w(i,1,n)] &= -1, \\ A_e[J_7(n), q_x(i,n)] &= -1, \\ A_e[J_7(n), q_b(i,0,n)] &= -\sum_{k\neq 0} \alpha_{i0kn} \max(0, \tau_{kn}), \\ A_e[J_7(n), q_w(i,0,n)] &= \sum_{k\neq 0} \alpha_{i0kn} \max(0, \tau_{kn}), \\ A_e[J_7(n), q_d(i,n)] &= -\sum_{k\neq 0} \alpha_{i0kn} \max(0, \tau_{kn}), \\ A_e[J_7(n), q_{lf}(n)] &= -1, \\ &\qquad \forall t \in \{0, \ldots, N_t - 1\}, \\ &\qquad \forall i \in \{0, \ldots, N_i - 1\}, \\ &\qquad \forall k \in \{0, \ldots, N_k - 1\}, \\ &\qquad \forall n \in \{0, \ldots, N_n - 1\}, \end{aligned} \tag{C.16}$$

with

$$v[J_7(n)] = \mathcal{A}_n^x + R_n \sum_i \bar{\zeta}_{in} + \sum_i [\omega_{in} + \upsilon_{in} + \nu_{in} + \bar{\pi}_{in} + \bar{a}_{in}] + 0.5 \sum_i \kappa_{i0n} \sum_{k\neq 0} \alpha_{i0kn} \max(0, \tau_{kn}) \tag{C.17}$$

The column l_n^f exists only in cases with a negative other income (Eq. (4.49)). With `withSSTaxability="optimize"`, $R_n \sum_i \bar{\zeta}_{in}$ leaves v and t_n^σ joins the left-hand side with coefficient -1 (Eq. (4.63)).

C.3 Other considerations

Beneficiaries Tax-free and tax-deferred accounts have special tax rules that allow giving part or the entire value of tax-free accounts to a spouse who can then consider it as his/her own. These accounts typically use percentages to designate beneficiaries. Let ϕ_j be the fraction of the account j that a spouse i_d wishes to leave to his/her surviving spouse i_s in year n_d, the year following the passing. To account for that event in year n_d, Eq. (4.28) needs to be rewritten as

$$\begin{aligned} b_{ij(n+1)} \;=\; & [1-\delta_{n,n_d-1}\delta_{i,i_d}] \\ & \times\Big\{[b_{ijn}+\delta_{j,0}d_{in}-w_{ijn}+(\delta_{j,2}-\delta_{j,1})x_{in}-\delta_{j,1}\Xi_{in}]\,(1+\mathcal{T}_{ijn})+\kappa_{ijn}(1+\mathcal{T}_{ijn}/2)\Big\} \\ & +[\phi_j\delta_{n,n_d-1}\delta_{i,i_s}] \\ & \times\Big\{[b_{i_djn}+\delta_{j,0}d_{i_dn}-w_{i_djn}+(\delta_{j,2}-\delta_{j,1})x_{i_dn}-\delta_{j,1}\Xi_{i_dn}]\,(1+\mathcal{T}_{i_djn}) \\ & +\kappa_{i_djn}(1+\mathcal{T}_{i_djn}/2)\Big\}. \end{aligned} \tag{C.18}$$

The first multiplier [...] on the right-hand side will always be one except for i_d in year n_d-1 when it will be zero. This will result in emptying all accounts for i_d for years n_d and beyond. The second special multiplier [...] before the second set of curly braces {} will always be zero except for the surviving spouse i_s in year n_d-1, who will then inherit a fraction ϕ_j of account j that was scheduled to go into i_d's j account at the beginning of year n_d. Note that Ξ_{i_dn} has to appear in the second set of curly braces as well: the first multiplier has already zeroed i_d's own row in year n_d-1, so a qualified charitable distribution made in that final year is only ever accounted for in the balance the surviving spouse inherits.

Rewriting the last equation as a constraint results in

$$\begin{aligned} & b_{ij(n+1)} \\ & -[1-\delta_{n,n_d-1}\delta_{i,i_d}] \\ & \times\Big\{[b_{ijn}+\delta_{j,0}d_{in}-w_{ijn}+(\delta_{j,2}-\delta_{j,1})x_{in}]\,(1+\mathcal{T}_{ijn})\Big\} \\ & -[\phi_j\delta_{n,n_d-1}\delta_{i,i_s}] \\ & \times\Big\{[b_{i_djn}+\delta_{j,0}d_{i_dn}-w_{i_djn}+(\delta_{j,2}-\delta_{j,1})x_{i_dn}]\,(1+\mathcal{T}_{i_djn})\Big\} \\ = \;& [1-\delta_{n,n_d-1}\delta_{i,i_d}]\kappa_{ijn}(1+\mathcal{T}_{ijn}/2) \\ & +[\phi_j\delta_{n,n_d-1}\delta_{i,i_s}]\kappa_{i_djn}(1+\mathcal{T}_{i_djn}/2). \end{aligned} \tag{C.19}$$

We are now ready to replace Eq. (C.8) for $A_e y = v$ by

$$\begin{aligned}
A_e[J_0(i,j,n), q_b(i,j,n+1)] &= 1, \\
A_e[J_0(i,j,n), q_b(i,j,n)] &= -[1-\delta_{n,n_d-1}\delta_{i,i_d}](1+\mathcal{T}_{ijn}), \\
A_e[J_0(i,j,n), q_d(i,n)] &= -[1-\delta_{n,n_d-1}\delta_{i,i_d}]\delta_{j,0}(1+\mathcal{T}_{ijn}), \\
A_e[J_0(i,j,n), q_w(i,j,n)] &= [1-\delta_{n,n_d-1}\delta_{i,i_d}](1+\mathcal{T}_{ijn}), \\
A_e[J_0(i,j,n), q_x(i,n)] &= -[1-\delta_{n,n_d-1}\delta_{i,i_d}](\delta_{j,2}-\delta_{j,1})(1+\mathcal{T}_{ijn}), \\
\text{when } N_i = 2 \text{ and } i = i_s, & \\
A_e[J_0(i,j,n), q_b(i_d,j,n)] &= -[\phi_j\delta_{n,n_d-1}\delta_{i,i_s}](1+\mathcal{T}_{i_djn}), \\
A_e[J_0(i,j,n), q_d(i_d,n)] &= -[\phi_j\delta_{n,n_d-1}\delta_{i,i_s}]\delta_{j,0}(1+\mathcal{T}_{i_djn}), \\
A_e[J_0(i,j,n), q_w(i_d,j,n)] &= [\phi_j\delta_{n,n_d-1}\delta_{i,i_s}](1+\mathcal{T}_{i_djn}), \\
A_e[J_0(i,j,n), q_x(i_d,n)] &= -[\phi_j\delta_{n,n_d-1}\delta_{i,i_s}](\delta_{j,2}-\delta_{j,1})(1+\mathcal{T}_{i_djn}),
\end{aligned}$$

$$\begin{aligned}
&\forall i \in \{0,\ldots,N_i-1\}, \\
&\forall j \in \{0,\ldots,N_j-1\}, \\
&\forall n \in \{0,\ldots,N_n-1\}.
\end{aligned} \tag{C.20}$$

where v is

$$\begin{aligned}
v[J_0(i,j,n)] &= [1-\delta_{n,n_d-1}\delta_{i,i_d}]\kappa_{ijn}(1+\mathcal{T}_{ijn}/2) \\
&\quad +[\phi_j\delta_{n,n_d-1}\delta_{i,i_s}]\kappa_{i_djn}(1+\mathcal{T}_{i_djn}/2).
\end{aligned} \tag{C.21}$$

While the last two equations may look cumbersome, their net effect is only to include a few more terms when $n = n_d - 1$. The QCD, opportunity-cost, and SPIA-premium terms of Eq. (C.8), omitted here, carry over with the same multipliers.

Asset allocation ratios When asset allocation ratios α are imposed, they should also be applied to how contribution amounts κ_{ijn} are invested, such that

$$\kappa_{ijkn} = \alpha_{ijkn}\kappa_{ijn}. \tag{C.22}$$

For other allocation schemes, just substitute $\alpha_{ijkn} = \alpha_{ikn}$ or α_{kn} depending on the scheme selected.

Asset allocation has been handled easily by assuming that the accounts are always rebalanced and only using a single multiplier $\mathcal{T}$, defined as

$$\mathcal{T}_{ijn} = \sum_k \alpha_{ijkn}\tau_{kn}, \tag{C.23}$$

to compute the return on the total balance of each savings account.

Spousal deposits and withdrawals While keeping the problem linear, a simple constraint can be imposed on surplus deposits in taxable savings accounts: the spousal fraction η splits each surplus as in Eq. (A.22), which for the two deposits reads

$$\eta\, d_{0n} = (1-\eta)\, d_{1n}. \tag{C.24}$$

With `spousal_surplus_deposit_fraction="optimize"`, the split is instead left to the optimizer through $d_{0n} + d_{1n} = s_n$ while both spouses live (Eq. (4.23)). A similar spousal ratio could be imposed on withdrawals from tax-deferred accounts,

$$\eta\, w_{01n} = (1-\eta)\, w_{11n}, \tag{C.25}$$

but this can cause drawing from an empty account while the other spousal account is not. Only the deposit scheme has been implemented in Owl.

C.4 Constraint tags and the explanation engine

Every row added to the constraint matrices carries a machine-readable *tag*: a tuple whose first element names the constraint *family* and whose remaining elements are the row's indices, e.g. `("rmd",i,n)` for the RMD floor of individual i in year n. Tags are inert during optimization; they exist so that the solution can be *explained*. When a plan is solved with `withDuals=True`, Owl re-solves the final linear program with all binary variables fixed at their solved values and extracts the row duals (shadow prices), row activities, and column reduced costs. The tags map each dual back to the constraint it prices, letting the explanation engine (`owlplanner.assistant.explain`, surfaced through the `explain_results` tool) answer questions such as "what does the bequest requirement cost in lifetime spending?" or "which years do RMDs force withdrawals?" directly from the mathematics of the optimization.

Each family is classified in one of three classes:

Policy User goals and legal rules (Table C.1). A binding row of this class is reported to the user together with the marginal value of relaxing it, expressed in today's dollars of the reported objective.

Structural Accounting identities and definitions (Table C.2). These rows are binding by construction; their duals are valuation information — for instance, the duals of the cash-flow balances form the plan's endogenous discount curve — and are used selectively by the explanation engine.

Artifact Big-$\mathcal{M}$ linkings, at-most-one selectors, and convexification rows (Table C.3). These encode formulation technique rather than financial meaning and are excluded from user-facing output. Threshold (cliff) effects that the underlying binaries model — IRMAA tiers, ACA brackets, Social Security taxability — are instead reported as primal *headroom* to the next threshold, since marginal prices cannot express discrete jumps.

The authoritative family-to-class table is `CONSTRAINT_FAMILIES` in `owlplanner/assistant/explain.py`; it must be kept in synchrony with the tags emitted in `plan.py` and with the tables below. An unregistered family is *not* dropped: the lookup defaults to the policy class, deliberately, so that a newly tagged constraint surfaces rather than vanishes. The cost of that default is that an artifact row left unregistered is reported as though it priced a user goal; a test (`test_every_emitted_tag_is_registered`) therefore fails when `plan.py` emits a tag the registry lacks. Year indices of the Medicare and ACA families are offsets from the first Medicare- or ACA-eligible year, respectively.

Constraint rows are only half of the sensitivity picture: policy also enters the model as *bounds on decision variables*, priced by their reduced costs rather than by row duals. A companion registry (`COLUMN_FAMILIES`, same module) prices variables sitting at a policy-meaningful bound: the `maxRothConversion` cap on x_{in}, conversions pinned to zero by rule (`noRothConversions`, `startRothConversions`, the last-two-years rule), the `minTaxableBalance` safety-net floor on b_{i0n}, and the loop-mode LTCG bracket room bounds on q_{0n} and q_{1n} of Eqs. (4.35)–(4.36). At a binding upper bound, the value of one more dollar of allowance is the reduced cost converted to reported-objective units; at a binding lower bound, the same conversion prices each dollar of relaxation of the floor. Structural bounds — bracket widths $\bar{\Delta}_{tn}$, standard deductions, big-$\mathcal{M}$ caps — are deliberately not registered; bracket fill is reported through the primal bracket analysis instead. The resulting explanation payload is validated against a versioned Pydantic schema (`PlanExplanation` in `owlplanner/assistant/explain_schema.py`), which constitutes the machine-readable contract consumed by AI assistants.

Tag family	Indices	Formulation
`rmd`	(i, n)	Eq. (4.5)
`bequest_floor`	—	Eq. (5.1)
`profile_lo`, `profile_hi`	(n)	Eqs. (4.31)–(4.33)
`roth_maturation`	(i, n)	Sec. (4.1.12)
`roth_early_basis`	(i, n)	Eq. (4.22)
`hsa_medical_cap`	(n)	Sec. (2.8)
`state_ret_exempt_cap`	(i, n)	Eq. (4.52)
`ltcg_room20`	(n)	Eq. (4.36)

Table C.1: Policy-class constraint tags: user goals and legal rules.

Tag family	Indices	Formulation
`cash_flow`	(n)	Eq. (4.30)
`taxable_income`	(n)	Eq. (4.49)
`state_taxable_income`	(n)	Eq. (4.51)
`account_carryover`	(i, j, n)	Eqs. (4.26)–(4.28)
`withdrawal_limit`	(i, j, n)	Sec. (4.1.6)
`surplus_deposit`	(i, n)	Eq. (4.23)
`ltcg_gn_def`	(n)	Sec. (4.1.16)
`ltcg_partition_lo`	(n)	Eq. (4.34)
`ltcg_room15_deduction`	(n)	Eq. (4.37)
`ltcg_room20_deduction`	(n)	Eq. (4.38)
`state_credit_cap`	(n)	Eq. (4.53)
`senior_bonus_deduction`	(n)	Eq. (4.8)
`niit_magi_def`	(n)	Eq. (4.47)
`irmaa_magi_def`	(n)	Eqs. (4.69), (4.70)
`irmaa_cost_def`	(n)	Eq. (4.67)
`aca_magi_def`	(n)	Sec. (4.1.22)
`aca_cost_def`	(n)	Sec. (4.1.22)

Table C.2: Structural-class constraint tags: accounting identities and definitions.

Tag family	Indices	Formulation
wdorder_txdef_gate	(i, n)	Eq. (4.12)
wdorder_roth_gate	(i, n)	Eq. (4.12)
wdorder_taxable_exhausted	(n)	Eq. (4.13)
wdorder_txdef_exhausted	(n)	Eq. (4.13)
wdorder_gate_monotone	(n)	Eq. (4.14)
ss_tax_plo, ss_tax_phi	(n)	Eq. (4.56)
ss_tax_pmin_ub, ss_tax_pmin_lb_cap	(n)	Eq. (4.60)
ss_tax_pmin_lb_plo	(n)	Eq. (4.60)
ss_tax_tss_ub, ss_tax_tss_lb_cap	(n)	Eq. (4.62)
ss_tax_tss_lb_formula	(n)	Eq. (4.62)
ltcg_zl15_link, ltcg_zl20_link	(n)	Eqs. (4.39)–(4.40)
ltcg_room15_mip, ltcg_q0_zero	(n)	Eq. (4.42)
ltcg_room20_mip, ltcg_q01_zero	(n)	Eq. (4.43)
ltcg_zl_monotone	(n)	Sec. (4.1.16)
ltcg_partition_hi	(n)	Eq. (4.34)
niit_excess	(n)	Eq. (4.44)
niit_nii	(n)	Eq. (4.45)
niit_nii_cap	(n)	Eq. (4.46)
senior_bonus_phaseout	(n)	Eq. (4.9)
senior_bonus_cap	(n)	Eq. (4.10)
irmaa_amo	(n)	Eq. (4.64)
irmaa_bracket_lb, irmaa_bracket_ub	(n, q)	Eq. (4.66)
aca_amo	(n)	Sec. (4.1.22)
aca_bracket_lb, aca_bracket_ub	(n, r)	Sec. (4.1.22)

Table C.3: Artifact-class constraint tags: formulation machinery, excluded from user-facing explanations. The withdrawal-ordering gates implement the `withdrawalOrder="taxable_first"` baseline policy of Sec. (4.1.7).

D. Solver Options

Every option below is a key of the dictionary passed to `plan.solve()` as its second argument, or equivalently an entry of the `[solver_options]` table of a case file. An unrecognized key is reported and ignored rather than refused, so that a case file written against a later version still loads. Defaults are those of the engine; the user interface may present different initial values.

D.1 Objective and goals

`netSpending`
: Required under `maxBequest`: the net annual spending to sustain, in the units of `units`. Ignored under `maxSpending`, where spending is what is maximized.

`bequest`
: Under `maxSpending`, the floor on the final bequest (default \$1). Sweeping it traces the spending–bequest frontier of Section 5.2. Ignored under `maxBequest`.

`partialBequestWeight`
: The value of a dollar left to non-spouse heirs at the *first* death, as a fraction of a dollar of the objective, spending or final bequest (default $\max(0.01, 2\,\text{gap})$). Without it, that money is worth nothing to either objective, and the solve is free to waste it.

`timePreference`
: A subjective time-discount rate ρ, in %/year (default 0), that front-loads spending by weighting year n with $(1+\rho)^{-n}$.

`spendingSlack`
: Percent of slack allowed around the spending profile (default 0), which lets

yearly spending depart from $g\xi_n$ within a band.

`minTaxableBalance`
: A per-person floor on the taxable balance, in `units`, from year 1 through each horizon, inflation-adjusted (Eq. (4.19)). Absent, no floor row is generated.

`oppCostX`
: An opportunity cost charged on Roth conversions, in percent (default 0): the fraction $1 - \text{oppCostX}/100$ of a converted dollar is what arrives.

D.2 Roth conversion controls

`maxRothConversion`
: The largest conversion allowed to each individual in any one year, in `units`. Absent, or negative, there is no cap. Zero forbids conversions.

`noRothConversions`
: The name of an individual whose conversions are pinned to zero.

`startRothConversions, stopRothConversions`
: Calendar years before and after which conversions are pinned to zero.

`swapRothConverters`
: For a couple, a signed calendar year before which only one spouse may convert: positive lets the first individual convert first and the second from that year on, negative the reverse. It takes precedence over `noRothConversions`, which is dropped, with a warning, when both are given.

D.3 Tax-breakpoint modes

Each of the following selects how one nonlinear tax feature is handled: `"loop"` prices it from the previous self-consistent iterate, while `"optimize"` puts it in the mixed-integer program, as described in Chapter 4. A value that is none of the documented modes is refused rather than silently solved as another.

`withMedicare`
: IRMAA bracket selection (default `"loop"`); `"none"` or `false` drops Medicare entirely.

`withACA`
: ACA marketplace premiums, default `"loop"` (Section 4.1.22).

`withLTCG`
: Long-term capital-gains brackets, default `"loop"` (Section 4.1.16).

`withNIIT`
: Net investment income tax, default `"loop"` (Section 4.1.17).

`withSSTaxability`
: Taxable share of Social Security, default `"loop"`. A number in $[0, 1]$ pins the share instead of computing it.

`withSeniorBonus`
: The OBBBA 65+ bonus phase-out, default `"loop"` (Section 4.1.5).

`breakpointMethod`
: A preset, not a family: `"loop"` (default) changes nothing, while `"branch-and-bound"` or `"local-search"` sets *every* applicable family above to `"optimize"` and `mipStrategy` to the same value. Medicare is included unless it is off, and the ACA only when a benchmark premium is set. It overrides a pinned Social Security share, with a warning.

`withSSAges`
: Social Security claiming ages: `"fixed"` (default) or `"none"` (no search), `"optimize"` (every individual with a PIA), or a name or list of names (Chapter 9).

`withdrawalOrder`
: `"optimal"` (default) lets the optimizer choose the order, while `"taxable_first"` imposes the conventional taxable–tax-deferred–tax-free waterfall through gating binaries (Section 4.1.7).

D.4 Solver selection and effort

`solver`
: `"default"` (the default), `"HiGHS"`, or `"MOSEK"`. The default selects MOSEK when it is installed and licensed, and HiGHS otherwise.

`gap` Relative mixed-integer optimality gap (default 10^{-4}). When `withMedicare="optimize"` and no gap is given, the default is loosened to 3×10^{-3}, and ten times further when `maxRothConversion` is at most $15k, because those models are the hardest to close.

`maxTime`
: Time limit per solve, in seconds (default 900). Applied *per self-consistent iteration*, so the budget for one case is `maxIter` × `maxTime`.

`mipMaxNodes`
: Branch-and-bound cap: nodes under HiGHS (default 1,000,000), branchings under MOSEK (no cap by default). The two engines count different things, which is worth remembering when comparing them.

`numThreads`
: Threads per MOSEK solve, 0 meaning all cores (default 0). Setting it allows matched parallelism, where several scenarios run at a few threads each instead of one scenario at all of them. It is a no-op under HiGHS, which solves serially here.

`mipScaleOrder`
: The money unit inside a mixed-integer solve, 10^{order} dollars (default 2, i.e. hundreds of dollars; 0 is dollars). Scaling the money keeps the big-$\mathcal{M}$ coefficients and the balances within a range the solver can condition.

`withDuals`
: When true (default false), the final linear program is re-solved with every binary fixed at its solved value, to extract the shadow prices the explanation engine reads (Section C.4).

D.5 Convergence of the self-consistent loop

The loop stops when the objective has settled *and* the quantities fed back into the next iteration have stopped moving. The first is judged by `absTol` and `relTol` on the objective, the second by `residualTol` on the fixed-point residual.

`maxIter`
: Maximum number of self-consistent iterations (default 29).

`absTol`
: Absolute objective tolerance, in dollars (default 100).

`relTol`
: Relative objective tolerance (default $\max(5 \times 10^{-5}, \text{gap}/300)$).

`residualTol`
: The largest disagreement still allowed at convergence between a quantity the linear program was built with and the value the resulting plan implies, in today's dollars *per year* (default 50). Per year, not per plan: the underlying measure sums over the horizon, so a flat figure would be three times stricter on a 33-year plan than on an 11-year one.

`epsilon`
: Lexicographic tie-breaking weight (default 5×10^{-7}), described in Section 5.3.

D.6 Local search

These apply only when `mipStrategy="local-search"` (Section 3.9).

`mipStrategy`
: `"branch-and-bound"` (default) or `"local-search"`.

`localSearchTime`
: Total time budget per solve, in seconds (default 300).

`localSearchStepTime`
: Time cap per restricted solve, in seconds (default 60).

`localSearchStepNodes`
: Node limit per restricted solve. Unset, it takes the engine default, which is 3,000 nodes under HiGHS and 20,000 branchings under MOSEK — a difference large enough to confound a comparison of the two engines unless it is set explicitly.

`localSearchRadius`
: The number of flips allowed on the Social-Security-taxability binaries inside the local-branching ball (default 4).

D.7 Inputs, reporting, and units

`previousMAGIs`
: The two MAGIs preceding the plan, which the IRMAA two-year lookback needs for the first two plan years.

`includeMedicarePartD`
: False drops Part D and its IRMAA surcharge from the Medicare costs (default true).

`medicarePartDBasePremium`
: The Part D base premium, $/month per person (default 0). The Part D IRMAA surcharges apply regardless.

`noLateSurplus`
: Pins the cash-flow surplus to zero in the final two plan years (default false).

`units` The monetary units of the option values: `"k"` (the default), `"M"`, or `"1"`.

`verbose`
: Enables progress logging for one solve (default false).

D.8 Retired and removed options

An option that no longer does anything is still *accepted* and reported as deprecated, because a case file saved earlier still carries it; one whose behaviour was unsound is refused outright. Both registries live in the source, `RETIRED_OPTIONS` in `plan.py` and `REMOVED_OPTIONS` in `config/schema.py`, each entry paired with the reason a reader would need.

`amoConstraints, amoRoth, amoSurplus`
: Retired. The mutually exclusive transactions these selected are now restored after the solve (Section 4.1.14), so nothing needs constraining, and the default plan is a pure linear program.

`bigMamo, bigMaca, bigMss, bigMltcg, bigMniit`
: Retired. Each big-$\mathcal{M}$ is now derived per year from what its own row gates, so there is nothing left to tune; the flat constants these keys set were hundreds to thousands of times too large.

`withDecomposition`
: Retired. Its only mode rounded all five bracket families at once from a single relaxation, which is jointly infeasible even where each family is feasible alone, so every solve fell back to the monolithic program it was meant to avoid.

`withSCLoop`
: Retired. No mode can solve without the self-consistent loop—the OBBBA phase-out of the standard exemption and the cost-basis gain fractions are both nonlinear in the solution—and turning it off left Medicare out of the budget, overstating spending. Use `maxIter` to shorten the loop instead.

`fixedSpending`
: Removed, and refused. Pinning g_0 under `maxSpending` made the objective constant over the feasible set, after which the solver could fill the top tax bracket while lower ones sat empty at no cost to the objective (Section 3.5). Sweep the `bequest` floor instead.

E. Source File Reference

The Owl Python package is located under `src/owlplanner/`. The table below summarizes the primary source files and their roles.

`plan.py` The core optimization engine. Constructs and solves the LP/MIP using HiGHS (via `highspy`) or MOSEK directly, MOSEK being selected by default when it is licensed. Implements the self-consistent iteration loop, all constraint-building methods, the objective functions, and post-solution analysis. Delegates stress-testing and stochastic spending methods to `stresstests.py` via mixin. This is the largest and most complex file.

`varmap.py` Variable block indexing and extraction for the LP solver. `VarBlock` records the start offset and shape of one decision-variable family (e.g., b, d, e, f, g, h, m, s, w, x, z^m, ...); `VarMap` accumulates them in declaration order and exposes a cursor tracking the running offset. Used by `plan.py` to build and query all LP/MIP variable indices without hard-coded offset arithmetic.

`abcapi.py` A solver-neutral constraint-building API used by `plan.py`. Provides an abstract interface for adding rows to the constraint matrix, keeping the model description independent of any specific LP solver.

`stresstests.py` Stress-testing and stochastic spending optimization methods. Provides `run_historical_range`, `run_mc`, and `run_stochastic_spending`, which take a `Plan` instance and run the optimizer across many return scenarios using a thread pool. Also implements the efficient-frontier LP, CVaR computation, and the Retirement Efficiency Score (RES). `Plan` exposes these as methods by delegation (mixin pattern).

`tax_federal.py`
: Federal tax calculations: ordinary income brackets and standard deductions, LTCG tax thresholds, SS taxability formula, NIIT, ACA Premium Tax Credit (with `acaCosts` and `acaVals`), and Medicare IRMAA premiums and bracket thresholds. Also computes RMD fractions from the IRS Uniform Lifetime Table. Implements the OBBBA (2025) tax schedule, the 65+ bonus deduction phaseout, and the future-law reversion scenario (`yOBBBA` parameter).

`tax_state.py`
: State income tax parameters for all 50 states and the District of Columbia. Loads per-state, per-filing-status bracket structures, standard deductions, Social Security treatment, and retirement-income and pension exemption caps from `data/taxes_state.toml`, and converts them into the inflation-indexed LP parameter arrays used by the state-tax constraints (Sec. (4.1.19)).

`socialsecurity.py`
: Social Security benefit modeling: FRA schedules, own-benefit and spousal adjustment factors, spousal and survivor benefit computation, and optional benefit trim for policy scenarios. Also implements the SSA earnings test — exempt amounts, countable earnings, grace year, months withheld, and the adjusted reduction factor that credits them back at FRA — and the survivor claiming window. The claiming-age search of Chapter 9 calls into this module once per candidate age rather than precomputing a benefit table.

`hfp_io.py`
: Household Financial Profile (HFP) workbook I/O. Reads and validates the Wages and Contributions sheet (one tab per individual) and the Fixed Assets and Debts tables, owns the person-sheet column list and the header validation, and returns the year-indexed tables consumed by `plan.py`.

`localsearch.py`
: The local search of Sec. (3.9), used when `mipStrategy="localsearch"`. Seeds a solution, then alternates restricted mixed-integer solves — one family of binaries freed at a time, and a local-branching ball around the Social-Security-taxability binaries — under per-step time and node caps.

`amorepair.py`
: Post-processing that repairs the mutually exclusive transactions once enforced by at-most-one binaries (Sec. (4.1.14)): a surplus deposit made in the same year as a taxable or tax-free withdrawal, and a Roth conversion made in the same year as a tax-free withdrawal. Replacing those binaries with this pass made the default plan a pure LP.

`pension.py`
: Pension income: per-person schedules, indexation, start ages, and the joint-and-survivor reduction.

`spending.py`
: The spending profile ξ_n — flat, smile, or a user-supplied series — and the subjective time-discount front-loading of `timePreference`.

`fixedassets.py`
: Models fixed-income assets (real estate, annuities, collectibles, etc.), computing year-by-year income, proceeds, and capital gains arising from assets listed in the Fixed Assets table.

`debts.py`
: Models debt obligations (loans, mortgages), computing annual payment schedules from the Debts table.

`export.py`
: Workbook and worksheet output: the year-indexed tables, the Summary, and the real- or nominal-dollar views written by the UI and by `owlclirun`.

`utils.py`
: Shared helpers, including the canonicalization of the mode-valued solver options (`normalize_mode_options`), which refuses a value that is none of the documented modes rather than silently solving another.

`progress.py`
: Progress reporting for the long sweeps.

`mylogging.py`
: The thread-aware logger, whose `tag` values map onto log levels.

`rates.py`
: Loads and pre-processes the historical market-return data from `data/rates.csv` (equity, bond, and inflation series from 1928 to present) and provides utilities for constructing rate scenarios.

`rate_models/`
: A pluggable system of rate model classes inheriting from `BaseRateModel`. The constant and deterministic methods (`trailing_30`, `optimistic`, `conservative`, `user`, `historical`, `historical_average`) and the parametric ones (`gaussian`, `lognormal`, `historical_gaussian`, `historical_lognormal`) live in `builtin.py`; each remaining method has its own module: `historical_bootstrap.py`, `vector_ar.py`, `garch_dcc.py`, `gmm.py`, `hmm.py`, `copula.py` (`historical_copula`), and `dataframe.py`. Shared machinery sits in `base.py` (the `BaseRateModel` interface), `_sampling.py`, `inflation_transform.py` (the skewness correction of Sec. (6.4.1)), `constants.py`, and `loader.py`, which discovers models at runtime. Custom models are discoverable if they subclass `BaseRateModel` and are importable.

`config/schema.py`
: Pydantic v2 models defining the full case configuration schema. All plan parameters (account balances, claiming ages, rate method, options, etc.) are validated here before being passed to `plan.py`.

`config/toml_io.py`
: TOML file loading and saving for `.toml` case files.

`config/legacy.py`
: Backward compatibility for older case-file formats.

`config/plan_bridge.py, config/ui_bridge.py`
: Bidirectional bridges between the validated configuration and, respectively, a `Plan` object and the Streamlit session state. `config/defaults.py` and `config/constants.py` hold the default values and the shared key names.

`data/`
: Tabulated data compiled into the package: the IRS Publication 590-B life expectancy tables (`irs_590b.py` for the Uniform Lifetime and Joint tables, `irs_590b_table_ii.py`), the CMS age-rating curve used to price a single enrollee from a couple's benchmark (`aca_age_rating.py`), the eight actuarial mortality tables of Sec. (7.3) (`mortality_tables.py`), the per-state tax parameters (`taxes_state.toml`), and the historical return series (`rates.csv`).

`plotting/` Visualization backends: `plotly_backend.py` for interactive plots in the Streamlit UI, and `matplotlib_backend.py` for static plots used in the CLI and Jupyter notebooks.

`cli/` A Click-based command-line interface registered as the `owlcli` entry point. Its commands are `list` (enumerate available case files), `run` (solve a case from a TOML file), `compare` (a case against a variant set by overrides), `explain` (describe a case without solving it), `frontier` (trace net spending against the bequest floor), `list-rates` (the available rate models), and `serve` (start the MCP server).

`assistant/` AI-assistant integration layer shared by the MCP server and the chat page. `tools.py` implements the MCP tool functions (case solving, comparison, stochastic frontiers, `explain_results`, ...); `explain.py` classifies the constraint tags (Sec. (C.4)) and turns a solved plan's shadow prices and primal solution into a structured, JSON-ready explanation; `explain_schema.py` holds the versioned Pydantic schema that validates the explanation payload; and `intake.py` provides the guided intake prompt and resources.

References

[1] M.-D. Lacasse, "Optimal Wealth Lab (Owl)," GitHub repository, 2025. https://github.com/mdlacasse/Owl.

[2] Social Security Administration, *Program Operations Manual System (POMS)*, RS 00615.003 (Full Retirement Age), RS 00615.015 (SSA age computation), https://secure.ssa.gov/poms.nsf/lnx/0300615003.

[3] Internal Revenue Service, *Publication 915: Social Security and Equivalent Railroad Retirement Benefits*, https://www.irs.gov/pub/irs-pdf/p915.pdf.

[4] Internal Revenue Code § 408A, "Roth Individual Retirement Accounts," https://www.law.cornell.edu/uscode/text/26/408A.

[5] Internal Revenue Service, *Publication 590-B: Distributions from Individual Retirement Arrangements (IRAs)*, https://www.irs.gov/pub/irs-pdf/p590b.pdf.

[6] Internal Revenue Code § 223, "Health Savings Accounts," https://www.law.cornell.edu/uscode/text/26/223.

[7] Internal Revenue Service, *Publication 969: Health Savings Accounts and Other Tax-Favored Health Plans*, https://www.irs.gov/pub/irs-pdf/p969.pdf.

[8] Internal Revenue Service, *Form 8889: Health Savings Accounts (HSAs)*, https://www.irs.gov/pub/irs-pdf/f8889.pdf.

[9] Internal Revenue Code § 36B, "Refundable credit for coverage under a qualified health plan," https://www.law.cornell.edu/uscode/text/26/36B.

[10] Internal Revenue Service, *Rev. Proc. 2024-35*, "Applicable Percentage Table for the Premium Tax Credit for Taxable Years Beginning in Calendar Year 2025," 89 Fed. Reg. 66278 (2024).

[11] Internal Revenue Service, *Rev. Proc. 2025-25*, "Applicable Percentage Table for the Premium Tax Credit for Taxable Years Beginning in Calendar Year 2026," https://www.irs.gov/pub/irs-drop/rp-25-25.pdf.

[12] Internal Revenue Service, *Rev. Proc. 2026-26*, "Applicable Percentage Table for the Premium Tax Credit for Taxable Years Beginning in Calendar Year 2027," sec. 3.01.

[13] *26 C.F.R.* § 1.401(a)(9)-9(d), "Joint and Last Survivor Table," https://www.ecfr.gov/current/title-26/section-1.401(a)(9)-9.

[14] Internal Revenue Code § 121, "Exclusion of gain from sale of principal residence," https://www.law.cornell.edu/uscode/text/26/121.

[15] Internal Revenue Code § 1014, "Basis of property acquired from a decedent," https://www.law.cornell.edu/uscode/text/26/1014.

[16] Internal Revenue Service, *Publication 15 (Circular E): Employer's Tax Guide*, https://www.irs.gov/pub/irs-pdf/p15.pdf.

[17] *20 C.F.R.* §§ 404.430–404.439, "Deductions because of work (the retirement earnings test)," https://www.ecfr.gov/current/title-20/part-404/subpart-E.

[18] Internal Revenue Service, *Notice 2025-67*, "Retirement plan contribution limits for tax year 2026."

[19] Internal Revenue Service, *Rev. Proc. 2025-32*, "Inflation-adjusted items for tax year 2026."

[20] U.S. Department of Health and Human Services, "Federal Poverty Guidelines" (annual), https://aspe.hhs.gov/poverty-guidelines.

[21] Centers for Medicare & Medicaid Services, "2026 Medicare Parts A & B Premiums and Deductibles," https://www.cms.gov/newsroom/fact-sheets/2026-medicare-parts-b-premiums-deductibles.

[22] Internal Revenue Service, "Topic No. 409: Capital Gains and Losses," https://www.irs.gov/taxtopics/tc409.

[23] Internal Revenue Code § 1411, "Imposition of Tax (Net Investment Income)," https://www.law.cornell.edu/uscode/text/26/1411.

[24] Internal Revenue Service, *Publication 17: Your Federal Income Tax*, `https://www.irs.gov/pub/irs-pdf/p17.pdf`.

[25] One Big Beautiful Bill Act (OBBBA), signed July 4, 2025. Extends and modifies individual income tax provisions; adds § 1002 senior deduction.

[26] SECURE 2.0 Act of 2022, Pub. L. No. 117-328, Division T, 136 Stat. 5966.

[27] R. J. Shiller, *Irrational Exuberance*, Princeton University Press, Princeton, NJ, 2000. Historical return data available at `http://www.econ.yale.edu/~shiller/data.htm`.

[28] J. C. Hull, *Options, Futures, and Other Derivatives*, 10th ed., Pearson, Hoboken, NJ, 2017. (Chapters 15–16 derive the log-normal / GBM return model and the arithmetic-to-log-space parameter conversion.)

[29] B. Efron, "Bootstrap methods: another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.

[30] H. R. Künsch, "The jackknife and the bootstrap for general stationary observations," *The Annals of Statistics*, vol. 17, no. 3, pp. 1217–1241, 1989.

[31] D. N. Politis and J. P. Romano, "A circular block resampling procedure for stationary data," in *Exploring the Limits of Bootstrap*, R. LePage and L. Billard, Eds. Wiley, New York, 1992, pp. 263–270.

[32] D. N. Politis and J. P. Romano, "The stationary bootstrap," *Journal of the American Statistical Association*, vol. 89, no. 428, pp. 1303–1313, 1994.

[33] J. Y. Campbell and L. M. Viceira, *Strategic Asset Allocation: Portfolio Choice for Long-Term Investors*, Oxford University Press, Oxford, 2002.

[34] H. Lütkepohl, *New Introduction to Multiple Time Series Analysis*, Springer-Verlag, Berlin, 2005.

[35] R. F. Engle, "Dynamic conditional correlation: a simple class of multivariate generalized autoregressive conditional heteroskedasticity models," *Journal of Business & Economic Statistics*, vol. 20, no. 3, pp. 339–350, 2002.

[36] T. Bollerslev, "Generalized autoregressive conditional heteroskedasticity," *Journal of Econometrics*, vol. 31, no. 3, pp. 307–327, 1986.

[37] J.P. Morgan/Reuters, *RiskMetrics — Technical Document*, 4th ed., New York: Morgan Guaranty Trust Company, 1996.

[38] G. J. McLachlan and D. Peel, *Finite Mixture Models*. Wiley, New York, 2000.

[39] A. P. Dempster, N. M. Laird, and D. B. Rubin, "Maximum likelihood from incomplete data via the EM algorithm," *Journal of the Royal Statistical Society, Series B*, vol. 39, no. 1, pp. 1–38, 1977.

[40] L. R. Rabiner, "A tutorial on hidden Markov models and selected applications in speech recognition," *Proceedings of the IEEE*, vol. 77, no. 2, pp. 257–286, 1989.

[41] A. Sklar, "Fonctions de répartition à n dimensions et leurs marges," *Publications de l'Institut de Statistique de l'Université de Paris*, vol. 8, pp. 229–231, 1959.

[42] L. E. Baum, T. Petrie, G. Soules, and N. Weiss, "A maximization technique occurring in the statistical analysis of probabilistic functions of Markov chains," *The Annals of Mathematical Statistics*, vol. 41, no. 1, pp. 164–171, 1970.

[43] H. M. Markowitz, "Portfolio selection," *The Journal of Finance*, vol. 7, no. 1, pp. 77–91, 1952.

[44] W. P. Bengen, "Determining withdrawal rates using historical data," *Journal of Financial Planning*, vol. 7, no. 4, pp. 171–180, 1994.

[45] P. Artzner, F. Delbaen, J.-M. Eber, and D. Heath, "Coherent measures of risk," *Mathematical Finance*, vol. 9, no. 3, pp. 203–228, 1999.

[46] R. T. Rockafellar and S. Uryasev, "Optimization of conditional value-at-risk," *Journal of Risk*, vol. 2, no. 3, pp. 21–41, 2000.